Syllabus

Module - 1 : Language Fundamentals

- Introduction
- Keywords
- Primitives and variables
- Operators
- Control Statements
- Methods
- Identifiers
- Scanner
- Arrays
- Strings (Introduction)

Module - 2 : Object oriented Programming

- Introduction
- Static and Non-static
- class and object
- Memory Management
- Blocks
- Constructors
- Inheritance
- Typecasting
- Method overloading
- Method overriding
- Polymorphism
- Abstraction
- Encapsulation

Module-3 : Core Java libraries

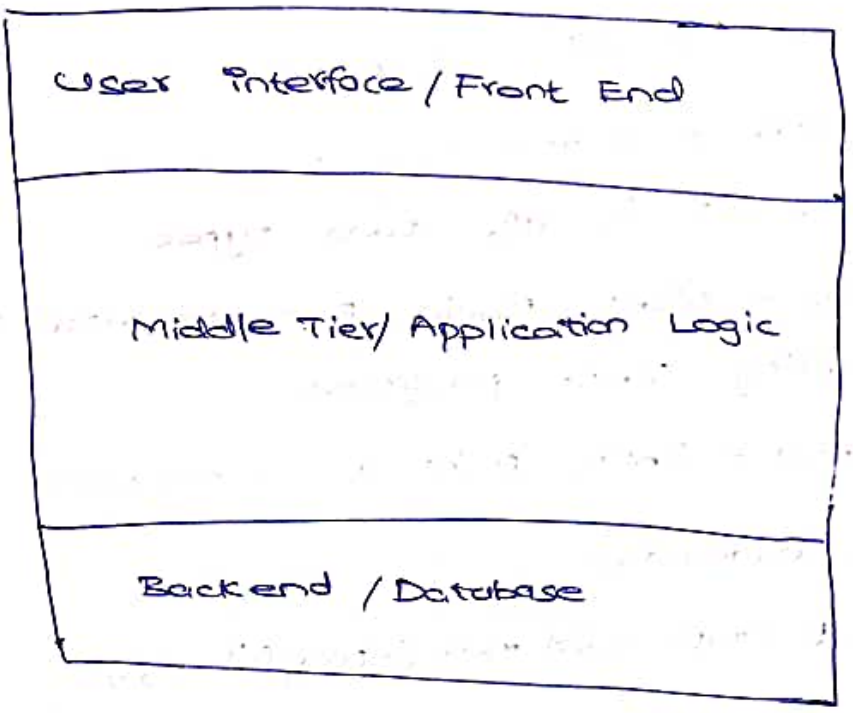
- Introduction
- Object class
- Exception Handling
- String class
- StringBuffer and StringBuilder
- Wrapper classes
- File Handling
- collection Framework
- Threads

22/3/22

Software :

A software is a collection of programs that works together to fulfill a Business Requirement.

Any software application in real world will contain 3 layers



1 - User interface are front-end is the layer of software application in which information is displayed and user interaction are captured.

2 - This layer of the S/W will generate data to be displayed based on the user interactions

3 - Backend / Database - It is the layer of S/W application in which all information is stored.

23/9/22

Types of software

1. Stand Alone Application
2. Client- Server Application
3. Web Based Application
4. Mobile Application
5. Embedded Application

() - Paranthisis
{ } - Braces
[] - Brackets

Basic Syntax

Syntax : Grammer for a programming Language. It is of two types

① Class - Most Basic Fundamental unit of writing Java programs

② Method - Entry point for execution.

```
class className
{
  Public static void main(String [] args)
  {

  }
}
```

First Java Program

```
class FirstJavaProgram
{
  public static void main(String[] args)
  {
    System.out.println("Welcome to Java");
  }
}
```

Write a Java prgm to display the name of cr BF/GF

```
Class BF
{
  public static void main(String[] args)
  {
    System.out.println("vijay");
  }
}
```

- Write
- Save
- compile javac Filename.java
- Execute java classname

24/3/22

System.out.println()

- It is a Java statement.
- ~~Display values~~, by using this statement a programmer can display values on the output screen
- String value have to be enclosed in double quotes eg: "java", "shylu shylu"
- Single characters have to be enclosed in single quotes eg: 'S', 'C'
- Numbers and Boolean values can be displayed directly.

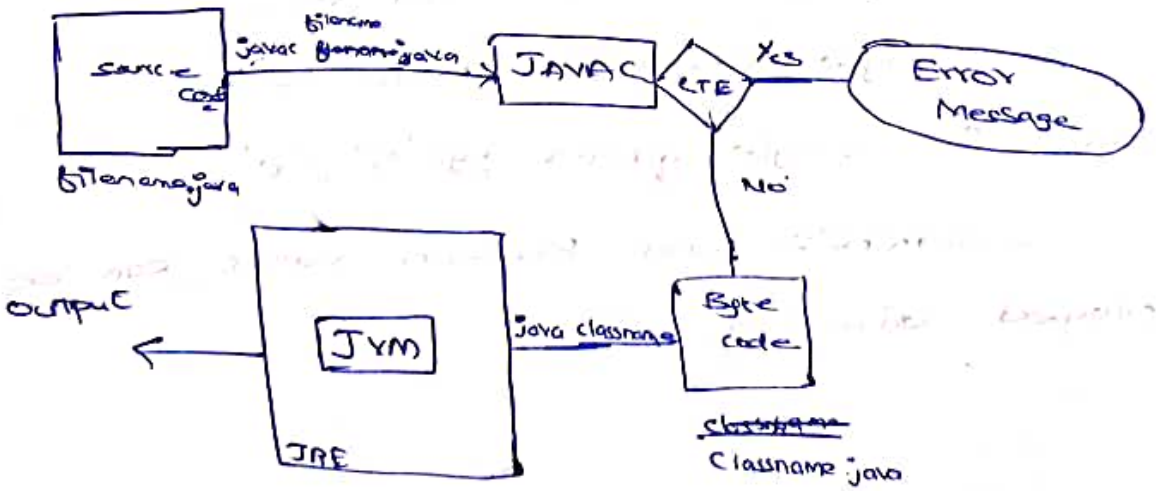
ex prgm:

```
class Program2
{
  public static void main(String[] args)
  {
    System.out.println("Jeraffin");
    System.out.println("23");
    System.out.println("73.4");
    System.out.println('S');
    System.out.println(true);
    System.out.println(false);
  }
}
```

Write a java prgm to display the following details a) Name b) YOP c) CGPA d) Gender e) Marital Sts

```
class Program3
{
  public static void main(String[] args)
  {
    System.out.println("Jeraffin");
    System.out.println("2021");
    System.out.println(74.5);
    System.out.println('M');
    System.out.println(false);
  }
}
```

Java Program Life cycle.



Life cycle (compilation):

The programmer creates the source code and save program with a filename and ".java" file extension.

The ".java" file is provided to the compiler which checks the syntax level mistakes made by the programmer

1. If mistakes are found the compiler displays appropriate error messages.

2. If no mistakes are found the compiler generates ".class" file which contains java bytecode

The Java bytecode is then executed by the JVM with the help of its subsystem in the JRE.

The programmer performs basic output analysis.

Bytecode:

*
- It is a highly optimized version of the Java source code
- It is generated by the compiler and is present in the ".class" file.
- The bytecode is not human readable.
- The JVM can understand bytecode and it can be considered as java executable code.

Javac :

※

- It is the name of the Java compiler.
- It takes the Java source code in ".java" file as input and checks for syntax level mistakes.
- If programmer has made mistakes it displays corresponding error messages.
- If there are no mistakes the compiler generates ".class" file with the bytecode.

JVM: Java Virtual Machine :

- This is the execution engine of Java language.
- The JVM is responsible for all run time activities in Java Language
- The JVM executes bytecode line by line and generates output
- It is a virtual machine and does not have a physical existence.

JRE: Java Runtime Environment :

The JVM uses several tools during execution like

1. Loaders
2. Linkers
3. Verifiers
4. Memory Managers
5. Event Handlers etc,

- This collection of tools necessary for execution is packaged as the JRE
- All activities in the JRE is controlled by the JVM.

History of Java

- Java was commercially released in the year 1995 by Sun Microsystems
- James Gosling is called as the father of Java programming language
- Initially named as "Oak" was later renamed as "Java"
- There have been several releases of the Java programming language.
- The current latest version of Java is Java18 or JavaSE18.
- However the stable version in use in Java8 or JavaSE8.
- Currently Sun Microsystems is a wholly owned subsidiary of Oracle corporation

29-3-22

The "+" Operator

The "+" operator in Java perform the following 2 operation

- Addition
- Concatenation (Joining)

Notes

String + Any value -> concatenation

Eg:

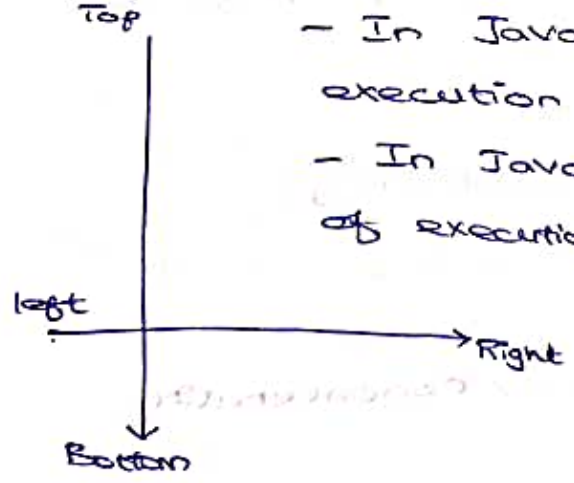
```
class Example1
{
    public static void main (String [] args)
    {
        System.out.println ("10+10");
        System.out.println ("4.2+2.5");
        System.out.println ("15+2.5");
        System.out.println ("3.4+9");
    }
}

class Example2
{
    public static void main(String[] args)
    {
        System.out.println("Hai" + "Bro");
        System.out.println("Boys" + 90);
        System.out.println("420 + "girls");
    }
}
```

ASCII code

A	- 65	a	- 97	0	- 48
↓		↓		↓	
Z	- 90	z	- 122	9	- 57

Order of Execution:



- In Java the vertical order of execution is top-bottom.
- In Java the horizontal order of execution is left-right.

eg:

```
Class Example3
{
    public static void main (String [] args)
    {
        System.out.println(20+20);
        System.out.println("The number is" + 20+20);
        System.out.println(20+20+" is the number");
        System.out.println("the number is "+(20+20));
    }
}
```

output:

```
40
The number is 2020
40 is the number
The number is 40
```

30.3.22

What is Java?

Java is a general purpose, high-level, object oriented, platform independent programming language.

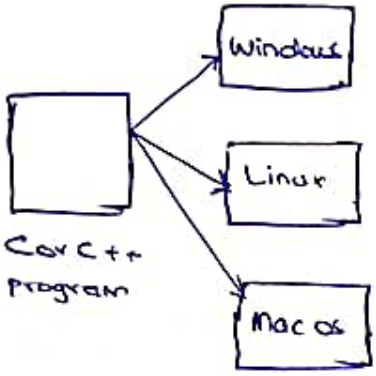
Let Break Down the key terms

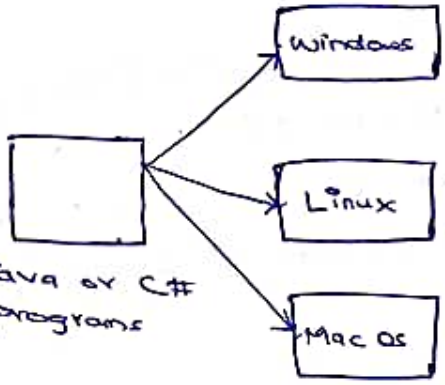
- Programming Language
- Platform Independent
- Object - oriented
- High Level
- Multi purpose

Platform Independent :

※

A platform is a combination of OS and Hardware





C (or) C++ programs will fail here as they are platform specific

Java (or) C# programs will succeeded here as they are platform independent

- Programs written in C (or) C++ is platform specific i.e: a program written in windows will only run in windows and so on.
- Programs written in Java is platform independent i.e:, a program once written in Java will work on any platform.
- This will reduce load on companies by avoiding concurrent development for different platforms.

High-Level :

Java code contains words derived from English language, making it easy to read and understand.

Object oriented :

Java supports 3 fundamental pillars of object-oriented programming which are

- Inheritance
- Encapsulation
- Abstraction

General Purpose :

Java programming can be used in all areas of software Testing and software Development

Comments in Java :

- Comments are notes made by programmer in Java source code
- Comments are generally used to Provide code description.
- In comments of Java is of 2 types

1. Single Line comments //
2. Multi Line comments /* */

- Comments do not have any impact on the execution

- Comments generally appear in green color.

Difference btwn JDK, JRE & JVM

JDK	JRE	JVM
Java Development kit	Java Runtime Environment	Java Virtual Machine
Software Application	Collection of Tools	Software component
All tools needed for Java Program Lifecycle	Tools needed in execution only	Java Execution Engine

Questions:

1. Why Java is platform Independent?
2. What is Bytecode?
3. Explain the role of Javac?
4. Diff btwn JDK vs JRE vs JVM

31.3.22

Keywords

Keywords are in-built words that have special meaning in Java language

As a programmer, we can use keywords but we cannot change the meaning.

i) lowercase
ii) Shade of blue color

There are 53 keywords in Java

byte, short, int, long, float, double, char, boolean, void, true, false, null
class, interface, enum, extends, implements, if, else, switch, case, break, default, do, for, while, continue, assert, return, new, instanceof, this, super, abstract, static, final, volatile, synchronized, StrictfP, native, transient, try, catch, finally, throws, throw, public, private, protected, package, import, goto, const

- Goto and const are 2 keywords that have no special meaning in Java

Primitives and Variables

Primitives and variables will help programmer to store values in a Java Program.

Primitives :

Primitives are in-built datatypes

It is used to ~~used~~ create variables

Java provides a total of 8 primitives

byte → 1 byte

short → 2 byte

int → 4 byte

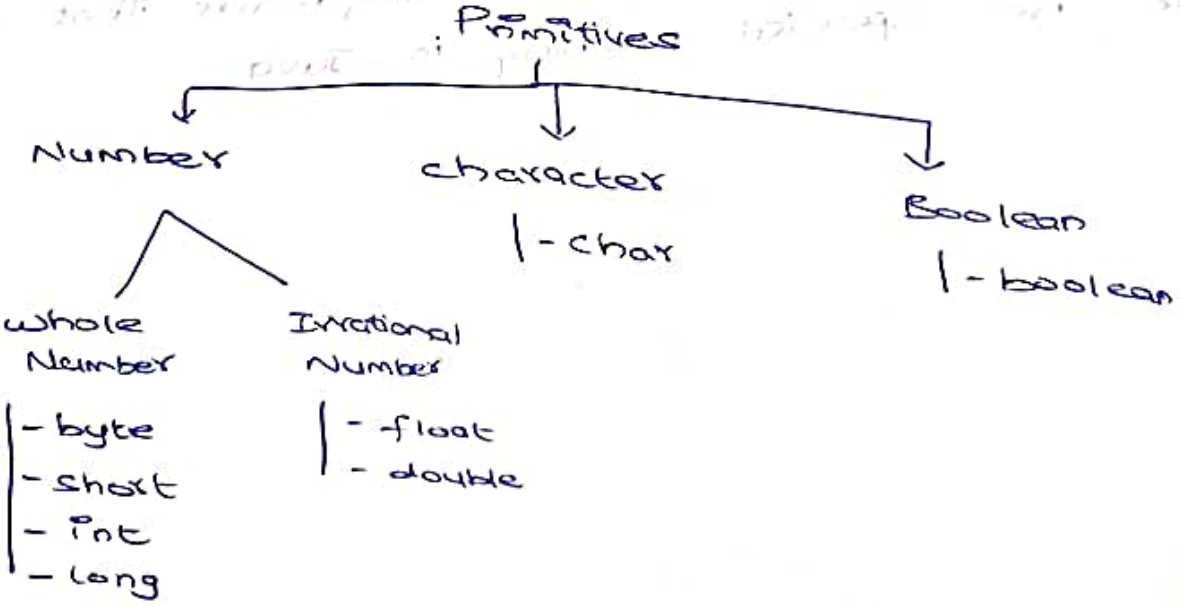
long → 8 byte

float → 4 byte

double → 8 byte

char → 2 byte

boolean → NA (1-bit)



Variables:

A Variable is a token/container used to hold a value

```
datatype VariableName; // Declaration
Variable = value; Initialization
```

```
Int age;
age = 24;

double cgpa;
cgpa = 7.42;

char initial;
initial = 'S'
```

```
datatype variableName = value;

Int age = 24;
double cgpa = 7.42;
char initial = 'S';
```

- The VariableName is always decided by programmer
- Variables should always be displayed directly.

Playing with long and float values

All long values should be suffixed with the alphabet "L" or "l".

All float values should be suffixed with the alphabet "F" or "f".

Example:

```
long aadhar = 123424241234fl;
      or
long aadhar = 12341234 1234L;

float val = 3.15F ; or  float pi = 3.14f
```

Eg:

```
Class Program3
{
    public static void main(String[] args)
    {
        float a = 3.14f;
        System.out.println(a);
        long b = 1234123412342l;
        System.out.println(b);
    }
}
```

Re-Initialization:

Replacing the old value in a variable with a new value is called as re-initialization.

Ex:

```
int a; // Declaration
a = 10; // Initialization
System.out.println(a); // a 10
a = 20; // Re-initialization
System.out.println(a); // 20
```

Final Variables:

- A variable created with final keyword is called as a final variable

Ex:

```
final int a = 10;
```

- If A variable is created with final keyword it cannot be re-initialized.

Ex:

```
final int val = 20;
val = 40; // CTE: Reinitialization
```

Questions:

1) What are keywords?
2) What are primitives?
3) What are variables?

Operators

Operators are symbols that are used to perform a particular task. Based on the type of operation, operators are classified into the following 5 families

Type	Operators
1. Arithmetic	+, -, *, /, %, =
2. Relational	>, <, >=, <=, ==, !=
3. Logical	&&, ||, !
4. Unary	++, --
5. Bitwise	&, |, ^, <<, >>

Arithmetic Operators:

Operator	Function
+	Addition / Concatenation
-	Subtraction
*	Multiplication
/	Division

% Modulus

= Equals(Assignment)

Eg:

```
class Program1
{
  public static void main(String[] args)
  {
    int a = 10;
    int b = 2;
    S.O.P (a+b); // 12
    S.O.P (a-b); // 8
    S.O.P (a*b); // 20
    S.O.P (a/b); // 5
    S.O.P (a%b); // 0
  }
}
```

Type conversion:

- precision
- memory size

byte → short → int → long → float → double

char → int

Eg:

```
class Program2
{
  P.S.V.m (String[] args)
  {
    int p=7;
    int q=2;
    S.O.P (p/q); // 3
    double i=7.0;
    double j=2.0;
```

```
        S.O.P (i/j); //3.5
        S.O.P (p/j); //3.5
        S.O.P (i/q); //3.5
    }
}
```

Operator Precedence :

[*, /, %] → ①

[+, -] → ②

[=] → ③

Eg:

```
class program3
{
    Public static void main(String[] args)
    {
        S.O.P (10+2*4+5); //23
        S.O.P (10*2+4*5); //40
        S.O.P (10/2*4+5); //25
        S.O.P (10/2*4*5); //100
        S.O.P (10*2/4*5); //25
        S.O.P (15%2-1*2); //-1
        S.O.P (11.5+1.5*2); //145
        S.O.P (8.5*7.8/2.7-10+(25+18)%5); //17.555
    }
}
```

Activity problems

— Write a Java prgm to find the square of an number

Eg:

```
class Program4
{
  Public static void main (String[] args)
  {
     int num = 5;
     int square = num * num;
     S.O.p ( num + " The root of " + square);
  }
}
```

→ Write a Java prgm the cube of a number

```
Class Program5
{
  Public static void main (String[] args)
  {
     Int num = 5;
     int cubic = num * num * num;
     S.O.p ( cubic + " The cubic root
                    of " + num);
  }
}
```

- Write a Java prgm to find area and perimeter of a square - given the side value is 10

Area = $side^2$
Peri = 4 * side

```
class Program6
{
  Public static void main(String[] args)
  {
     int side = 10;
     int area = side * side;
     int peri = 4 * side;
```

```
S.O.P ("Area of square is " + area);
S.O.P (" Perimeter of square is " + peri);
}
}
```

- Write a Java prgm to calculate area and circumference of a circle using the data given below : Radius : 4.5

Pi = 3.14 Area = πr^2 Circ = $2\pi r$

```
Class circle
{
  Public static void main (String[] args)
  {
    double r = 4.5;
    final double pi = 3.14;
    double area = pi * r * r;
    double circ = 2 * pi * r;
    S.O.P ("Area of circle : " + area);
    S.O.P ("Circ of circle:" + circ);
  }
}
```

- Write a Java prgm to find the simple interest using the data below

(p) The principle amt that we borrow - 10000

(r) Rate of Interest per Annum - 6.5 %

(t) Tenure - 2

Simple interest - PnR/100

```
Class Activity1
{
  Public static void main (String[] args)
  {
    double principle = 10000.0;
    double rate = 6.5;
```

```
int tenure = 2;
    double si = (principle * rate * tenure) / 100;
    S.O.P ("Simple Interest :" + SI);
  }
}
```

Mr. Iyer a resident of Chennai has taken a housing loan of 32,00,000.00 INR from State Bank of India at a rate of 8.5% per annum he has a tenure of 20 years and assuming he makes all payments in time and does not foreclose his loan plase calculate the monthly EMI ?

```
Class Activity2
{
   Public static void main(String[] args)
   {
      double loan = 3200000.0;
      double rate= 8.5;
        int tenure = 20;
      double interest = (loan * rate * tenure) / 100;
        double totalrepay = loan + interest;
          int tenuremonths = tenure * 12;
          double emp = totalrepay / tenuremonths;
           S.O.P ("EMI :" + emi);
     }
   }
```

Relational Operators

< → less than

> → greater than

<= → lessthan or equal to

>= → greater than or equal to

== → equal to

!= → Not equal to

- Relational Operators are used to compare two values
- After comparison it returns a boolean value

Eg:

```
class Program1
{
  public static void main(String[] args)
  {
    S.O.P (10<15); // true
    S.O.P (25<15); // false
    S.O.P (13>10); // true
    S.O.P (13>17); // false
  }
}
```

Eg:

```
class Program2
{
  public static void main(String[] args)
  {
    S.O.P (13<=15); // true
    S.O.P (18<=15); // false
    S.O.P (15<=15); // true
    S.O.P (12>=11); // true
    S.O.P (12>=16) // false
    S.O.P (12>=12)// true
  }
}
```

eg:

```
class Program3
{
    public static void main(String[] args)
    {
        S.o.p (10==10); // true
        S.o.p (10==11); // false
        S.o.p (14!=14); // true
        S.o.p (14!=14); // false
    }
}
```

Logical operators

&& ⟶ Logical AND

|| ⟶ Logical OR

! ⟶ Logical NOT

Logical operators will take boolean values as input and provides boolean values as output.

eg:

```
class Program1
{
    public static void main(String [] args)
    {
        S.o.p (false && false); // false
        S.o.p (false && true); // false
        S.o.p (true && false); // false
        S.o.p (true && true); // true
    }
}
```

```
Class Program1
{
    public static void Main (String[] args)
    {
        S.O.P (false || false); // false
        S.O.P (false || true); //true
        S.O.P (true || false); //true
        S.O.P (true || true); //true
    }
}
```

```
Class program3
{
    Public static void Main (String[] args)
    {
        S.O.P (!true); // false
        S.O.P (!false); // true
    }
}
```

Bitwise Operator

& — Bitwise AND

| — Bitwise OR

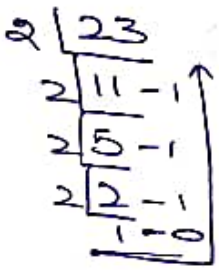
^ — Bitwise XOR

>> — Bitwise Right shift

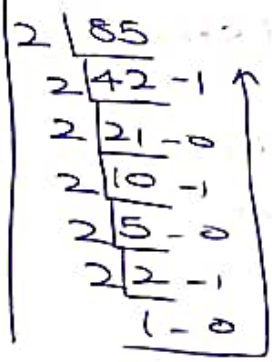
<< — Bitwise left shift

Decimals → Binary conversion

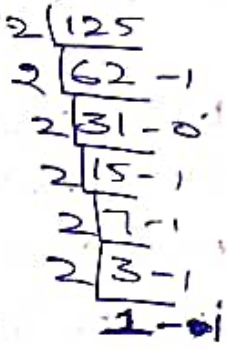
23 = 10111



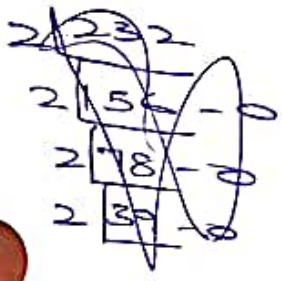
85 = 1010101



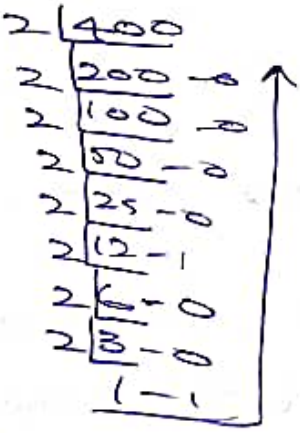
125 = 1011101



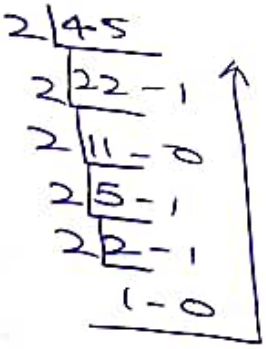
232 = 11101000



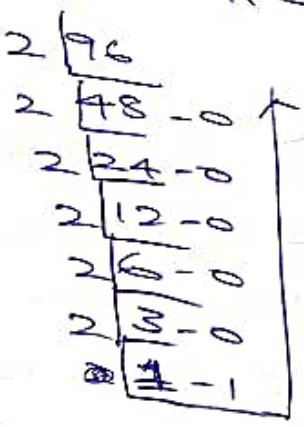
400 = 110010000



45 - 101101

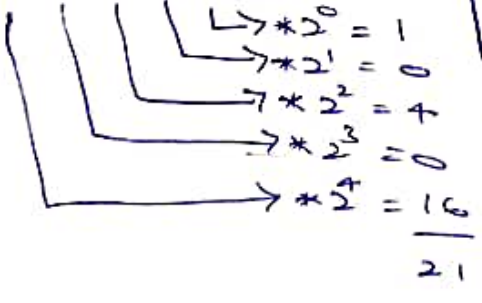


96 = 1100000

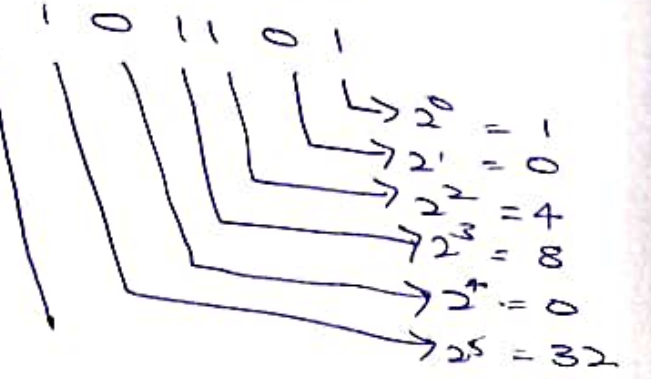


Binary → Decimal Conversion

10101 = 21



101101 = 45



11101 = 29

1 -1 1 01

$2^0 = 1$
$2^1 = 0$
$2^2 = 4$
$2^3 = 8$
$2^4 = 16$

29

1101 = 13

1101

$2^0 = 1$
$2^1 = 0$
$2^2 = 4$
$2^3 = 8$

13

1110011 = 115

1 1 1 0 0 1 1

$2^0 = 1$
$2^1 = 2$
$2^2 = 0$
$2^3 = 0$
$2^4 = 16$
$2^5 = 32$
$2^6 = 64$

115

110011 = 51

1 1 0 0 1 1

$2^0 = 1$
$2^1 = 2$
$2^2 = 0$
$2^3 = 0$
$2^4 = 16$
$2^5 = 32$

51

10101 = 21

1 0 1 0 1

$2^0 = 1$
2^1
$2^2 = 0$
$2^3 = 4$
$2^4 = 0$
$= 16$

21

Bitwise AND

I/P	I/P	O/P
0	0	0
0	1	0
1	0	0
1	1	1

Bitwise OR

I/P	I/P	O/P
0	0	0
0	1	1
1	0	1
1	1	1

Bitwise XOR

I/P	I/P	O/P
0	0	0
0	1	1
1	0	1
1	1	0

Q8

```
class Pgm1
{
  public static void main(String[] args)
  {
    int a = 11;
    int b = 13;
    S.O.P(a&b); // 9
    S.O.P(a|b); // 15
    S.O.P(a^b); // 6
  }
}

class Pgm2
{
  public static void main(String[] args)
  {
    int a = 27;
    int b = 24;
    S.O.P(a&b); // 24
    S.O.P(a|b); // 27
    S.O.P(a^b); // 3
  }
}

class Pgm3
{
  public static void main(String[] args)
  {
    int p = 35;
    int q = 22;
    S.O.P(a&b); // 3
    S.O.P(a|b); // 55
    S.O.P(a^b); // 52
  }
}
```

```
Class pgm4
{
  public static void main(String[] args)
  {
    int i = 38;
    int j = 27;
    S.o.p(i&j); // 3
    S.o.p(i|j); //61
    S.o.p(i^j);
  }
}
```

100110
011011
000010
111111

2|38 2|19-0 2|9-1 2|4-1 2|2-0 1-0

2|27 2|13-1 2|6-1 2|3-0 1-1

<< → Bitwise Left shift

```
Class Example1
{
  public static void main(String[] args)
  {
    int a=5;
    S.o.p(a<<1); //10

    int b=13;
    S.o.p(a<<1); //26

    int c=17;
    S.o.p(a<<1); //34
  }
}
```

10 5

26 13

34 17

```
int x=11;
S.o.p(a<<1); //22

int x=10;
S.o.p(y<<2); //40

int y=18
S.o.p(y2<<2); //72
```

22 11

40 20 10

72 36 18

1011

$2^0=1$, $2^1=2$, $2^3=8$, $2^4=16$

2|11 2|5-1 2|2-1 1-0

1011

$2^0=1$, $2^1=2$, $2^3=8$

22 | 11

```
int p = 9;
S.o.p(p<<3);
```

72 36 18 9

```
int q = 25;
S.o.p(q<<3);
```

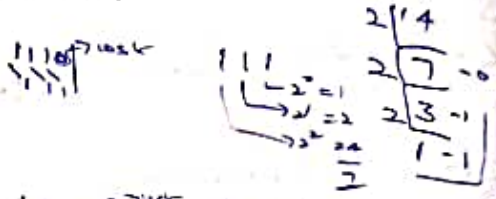
200 100 50 25

```
int r = 37;
S.o.p(r<<3);
```

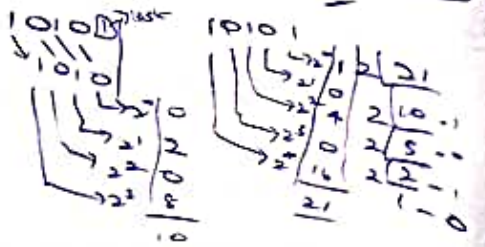
296 148 74 37

`>>` → Bitwise Right Shift

```
int a = 14;
S.o.p(a>>1); //7
```



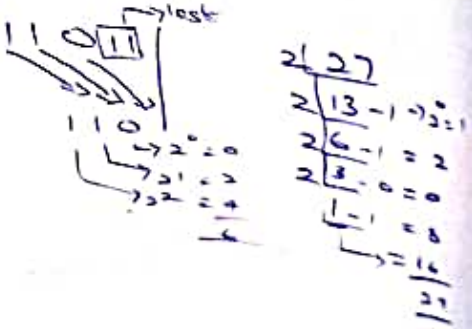
```
int b = 21;
S.o.p(b>>1); //10
```



```
int c = 37;
S.o.p(c>>1); //18
```

```
int x = 19;
S.o.p(x>>1); //9
```

```
int y = 27;
S.o.p(y>>2); //6
```



```
int z = 35;
S.o.p(z>>2); //8
```

```
int p = 13;
S.o.p(p>>3); //1
```

13 6 3 1

```
int q = 25;
S.o.p(q>>3); //3
```

25 12 6 3

```
int r = 46;
S.o.p(r>>3); //5
```

46 23 11 5

Unary Operators

Java provides the following two unary operators. There are 2 types of unary operators in Java

i) ++ (Increment)

ii) -- (decrement)

- The increment operator will increase value in a variable by 1.
- The decrement operator will decrease the value in a variable by 1.
- Increment and Decrement operations can be done in 2 ways
  * Post operation
  * Pre operation

Post - operation :

In case of post operation, the current value in the variable is used and then increment or decrement is done.

Syntax:

```
var++  | var--
```

```
int a = 7;
S.o.p(a++); //7
S.o.p(a); //8

int b = 5;
S.o.p(a--); // 5
S.o.p(a); 4
```

```
int x = 11;
S.o.p(a++); //11
S.o.p(a++); //12
S.o.p(a); //13

int y = 17;
S.o.p(y--); //17
S.o.p(y--); //16
S.o.p(y); //15
```

```
int a = 8 ; a 8
int b = a++ ⊕ a++ ;
                8   9
S.O.P (b) ; //17

int c = 11 ; c 9
int d = c-- ⊕ c-- ;
        11      10
S.O.P(d) ; //21

int p = 13 ; 14 15 16
int q = p++ ⊕ p++ ⊕ p++
        13    14    15
S.O.P (q) ; // 42

int r = 17 16 15 14
int s = r-- ⊕ r-- ⊕ r-- ;
        17    16    15
S.O.P (s) ; // 48

int m = 9 ; 10 11
int n = 7 ; 6 5
int o = m++ ⊕ n-- ⊕ m++ ⊕ n-- ;
        9     + 7   + 10    + 6
S.O.P(o) ; // 32

int i = 14 ; 15 12
int j = 16 ; 17 18
int k = i-- ⊕ j++ ⊕ i-- ⊕ j++ ;
        14    16    13    17
S.O.P (k) ; //60
```

Pre-operation :

In case of pre-operation, the current value of the variable is increased & decreased and then used

Syntax :

++var | --var

Eg:

```
int a = 6;          7
S.o.p(++a); // 7
S.o.p(a); // 7

int b = 9;
S.o.p(--b); // 8
S.o.p(b); // 8

int a = 10 ||
int b = ++a ⊕ ++a ;
          11     12
S.o.p(b); // 23

int c = 13 ;
int d = --c ⊕ --c ;
         12     12
S.o.p(d); // 23

int i = 18 ;
int j = 12 ;
int k = ++j ⊕ --i ⊕ -j ⊕ ++i ;
         14     17    13    18
S.o.p(k); // 62

int a = 12 ; 12 14
int b = 15 ;
int c = a++ ⊕ --b ⊕ a++ ⊕ b--;
        12     14    13     14
S.o.p(c); // 53

int i = 15 ; 16
int j = 17 ; 18
int k = i++ ⊕ ++j ⊕ --i ⊕ --j ;
        15     18     15     17
S.o.p(k) // 65

int x = 13 ; 14
S.o.p(++x); // 14
S.o.p(++x); // 15
S.o.p(x); // 15

int y = 17 ; 16
S.o.p(--y); // 16
S.o.p(--y); // 15
S.o.p(--y); // 14

int p = 13 ;
int q = 15 ;
int r = ++p ⊕ -q ⊕ ++p ⊕ --q ;
        14      4     15     13
S.o.p(r);
  = 56.
```

```
int p = 22;  23
int q = 25;  24
int r = p++ + q-- + ++p + --q;
        22    25    24    23
S.O.p (r); // 94
```

1] What are operators?

2] Diff. btwn = and == ?

3] Diff btwn Post operation & pre operation?

4] What is operator overloading?

Making same operator to do multiple tasks.

Java does not support operator overloading.

Control Statements

Control statements can be used by the programmer to alter the order of execution. inside the Java program.

Java provides the following 4 control statements

* If statement
* Switch statement
* For - Loop
* While - Loop

If Statement :

Version-1

```
if (condition)
{
  // if- body
}
```

Eg: 1

```
class prgm1
{
  public static void main (String[] args)
  {
    S.o.p ("Prgm starts");
    int a = 5
    if (a < 7)
    {
      S.o.p (" prgm is true");
    }
    S.o.p (" Prgm ends");
  }
}
```

O/P:

```
Prgm starts
Prgm is true
Prgm ends
```

eg 2:

```
class prgm2
{
  public class void main(String[] args)
  {
    S.O.P ("prgm starts");
    int a=10
    if (a>5 && a<15)
    {
      S.O.P ("prgm is true");
    }
    S.O.P ("prgm ends");
  }
}
```

O/P:

```
prgm starts
prgm is true
prgm ends
```

eg 3:

```
class prgm3
{
  public static void main(String[] args)
  {
    int num = 17;
    if (num%3==0 || num%5==0)
    {
      S.O.P (" prgm is true");
    }
  }
}
```

O/P:

eg 4:

```
class prgm4
{
  public static void main(String[] args)
  {
    boolean ischecked = true;
    if(ischecked == true)
    {
      S.O.P(" Proceed further");
    }
  }
}
```

O/P:

```
Proceed further
```

Version 2 :

```
if (condition
{
   //if-body
}
else
{
   // else-body
}
```

Eg 1 :

```
class pgm1
{
  public static void main (String[] args)
  {
    int x = 5
    if (x > 7)
     {
       S.O.P (" Jeera");
     }
    else (x < 7)
     {
       S.O.P ("Shylu");
     }
  }
}
```

O/P :

Shylu

Eg 2 :

```
class pgm2
{
  public static void main (String[] args)
  {
     int num = 10;
     if (num % 2 == 0)
     {
       S.O.P (" Even Number");
     }
     else
      {
        S.O.P ("Odd Number");
      }
  }
}
```

O/P :

Even Number

Q :3

```
class pgm3
{
  public static void main(String[] args)
  {
    int age = 23;
    if (age >= 18)
    {
      S.O.P("Major");
    }
    else
    {
      S.O.P("Minor");
    }
  }
}
```

O/P:

Major

Q 4 :

```
class pgm4
{
  public static void main(String[] args)
  {
    double accBal = 10000.0;
    int amt = 1000;
    if (amt <= accBal)
    {
      S.O.P("Withdrawal complete");
    }
    else
    {
      S.O.P("Insufficient Balance");
    }
  }
}
```

O/P:

Withdrawal complete

Version-3:

```
if (condition 1)
{

}
else if (condition 2)
{

}
else if (condition 3)
{
 }
 :
 else
 {

  }
```

Eg:1

```
class pgm1
{
  public static void main (String [] args)
  {
    int a = 5;
    if (a > 3)
    {
      S.o.p ("Qspiders");
    }
    else if (a == 7)
    {
      S.o.p ("Jspiders");
    }
    else
    {
      S.o.p ("PJspiders");
    }
  }
```

O/P:

Qspiders

Eg 2:

```
class prgm2
{
  public static void main(String[] args)
  {
    int age = 23;
    if (age > 0 && age <= 3)
    {
      S.o.p (" Free Travel");
    }
    else if (age > 3 && age <=10)
    {
      S.o.p (" Half Ticket");
    }
    else if (age > 10 && age <60)
    {
      S.o.p (" Full Ticket");
    }
    else
    {
      S.o.p (" Sr citizen Discount");
    }
  }
}
```

O/P : Full Ticket

Eg 3:

```
class prgm3
{
  public static void main (String[] args)
  {
    int val = 20;
    if (val %2 ==0 && val %5 == 0)
    {
      S.o.p("chennai");
    }
    else if (val %2 ==0)
    {
      S.o.p (" Bangalore");
    }
    else if (val %5 ==0)
    {
      S.o.p(" CBE");
    }
    else
    {
      S.o.p(" Tuty");
    }
  }
}
```

O/P : Chennai

eg 4 :

```
class prgm4
{
   public static void main(String[] args)
   {
      double expense = 500.0;
      if(expense >= 10000)
      {
         S.O.P(" 10% Discount");
      }
      else if (expense >= 5000 && expense < 10000)
      {
         S.O.P(" 5% Discount");
      }
      else
      {
         S.O.P("No Discount")
      }
   }
}
```

O/P :

No Discount

Version 4 : Nested if statement

```
if(condition)
{
    if(condition)
    {
    }
    else
    {
    }
}
else
{   if(condition)
    {
    }
    else
    {
    }
}
```

Eg 1:

```
class prgm1
{
   public static void main (String[] args)
   {
      int num = 5;
      if (num < 10)
      {
          if (num%2 == 0)
          {
            S.O.P ("Noodles");
          }
          else
          {
            S.O.P ("Biriyani");
          }
      }
      else
      {
          if (num%2 == 0)
          {
            S.O.P ("Prawns");
          }
          else
          {
            S.O.P ("Porotta");
          }
      }
   }
```

O/P: Biriyani

Eg 2:

```
class prgm2
{
   Public static void main (String[] args)
   {
  char gender = 'M';
       int age = 24;
       if (gender == 'M')
       {
           if (age >= 21)
           {
             S.O.P ("Eligible Bachelor");
           }
```

```
        else
        {
          S.o.p ("Not eligible");
        }
      else if (gender == 'F')
      {
        if (age >= 18)
        {
          S.o.p ("Eligible Spinster");
        }
        else
        {
          S.o.p ("Not eligible girl");
        }
      }
    }
```

Output:

Eligible Bachelor

Eg 3:

```
class prgm3
{
  public static void main (String[] args)
  {
    double accBalance = 10000.0;
    int amt = 250;
    if (amt >= accBalance)
    {
      if (amt % 100 == 0)
      {
        S.o.p ("Withdrawal complete");
      }
      else
      {
        S.o.p ("Invalid Denomination");
      }
    }
    else
    {
      S.o.p ("Insufficient Balance");
    }
  }
}
```

O/P:

Invalid Denomination

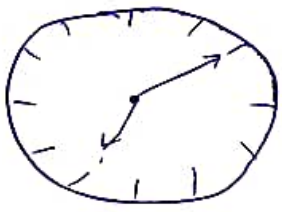
Write a java prgm to swap 2 numbers without using a third variable.

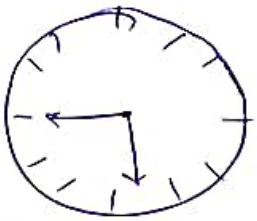
```
class prgm1
{
  Public static void main (String [] args)
  {
     int num1 = 10; // 25, 15
     int num2 = 15; // 10
     S.o.p (num1) ; //10
     S.o.p (num2) ; //15
     num1 = num1 + num2;   10+15 → 25
     num2 = num1 - num2;   25-15 → 10
     num1 = num1 - num2;   25-10 → 15
     S.o.p (num1) ;//15
     S.o.p (num2) ; //10
  }
}
```

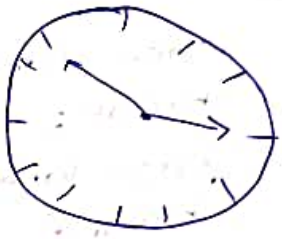
With using third variable

```
class prgm2
{
  public static void main (String [] args)
  {
    int num1 = 11;
    int num2 = 12;
     S.o.p (num1) ; // 11:
     S.o.p (num2) ; // 12
     int t = num1;
       num1 = num2;
       num2 = t ;
     S.o.p (num1); // 12
     S.o.p (num2); // 11
  }
}
```

Write a prgm to find the least angle bwh the hour hand and minute hand in a analog clock








```
Class prgm1
{
  public static void main (String[] args)
  {
    int hour = 11;
    int minute = 15;
    double hourangle = hour * 30 + minute x 0.5;
    double minuteangle = minute * 6;
    double angle;
    if (hourangle > minuteangle)
    {
      angle = hourangle - minuteangle;
    }
    else
    {
      angle = minuteangle - hourangle;
    }
    if (angle > 180)
    {
      angle = 360 - angle;
    }
    S.o.p("Least Angle :" + angle);
  }
}
```

Hour hand

12 hrs = 360°

1 hr = 30°

60 mins = 30°

1 min = 0.5°

hour angle = hour * 30 + minute * 0.5

Minute hand

60 mins = 360°

1 min = 6°

minute angle = minute * 6

O/P :

122.5

Switch Statement

Syntax:

```
Switch (choice)
{
   case label :
    break;
   case label :
    break;
    :
    :
   default :
}
```

Eg 1:

```
class prgm1
{
  Public static void main(String[] args)
  {
    int choice = 3;
    Switch (choice)
    {
      case 1 :
          S.O.P("Apple");
          break;
      case 2 :
          S.O.P("Grapes");
          break;
      case 3 :
          S.O.P("Banana");
          break;
      case 4 :
          S.O.P("Mango");
          break;
      default :
          S.O.P("Invalid choice");
    }
  }
}
```

O/P:

Banana

eg 2:

```
class prgm2
{
    public static void main(String[] args)
    {
        int choice = 1;
        switch(choice)
        {
          case 1 :
          S.o.p("Red");
          break;
          case 2 :
          S.o.p("Blue");
          break;
          case 3 :
          S.o.p("Orange");
          break;
          case 4 :
          S.o.p("Yellow");
          break;
          default :
          S.o.p("Black");
        }
    }
}
```

O/P:

Red

eg 3:

```
class prgm3
{
    public static void main(String[] args)
    {
        int option = 3;
        switch (option)
        {
          case 1: S.o.p ("UPI");
          break;
          case 2: S.o.p("COD");
          break;
          case 3: S.o.p("Card");
          break;
          case 4: S.o.p("Net Banking");
          break;
        }
    }
}
```

O/P:

Card

Eg 4:

```
class prgm4
{
  public static void main(String[] args)
  {
    int lang = 2
    switch(lang)
    {
      case 1: S.O.P ("English");
      break;
      case 2: S.O.P("Tamil");
      break;
      case 3: S.O.P("Hindi");
      break;
      default: S.O.P("Invalid selection");
    }
  }
}
```

O/P:

Tamil

Eg 5:

```
class prgm5
{
  public static void main(String[] args)
  {
    char grade = 'B';
    switch (grade)
    {
      case 'A': S.O.P ("Excellent");
      break;
      Case 'B': S.O.P("Good");
      break;
      Case 'C': S.O.P ("Average");
      break;
      default: S.O.P("Meet Class Teacher");
    }
  }
}
```

O/P:

Good

Repeated Execution

↓

Loops

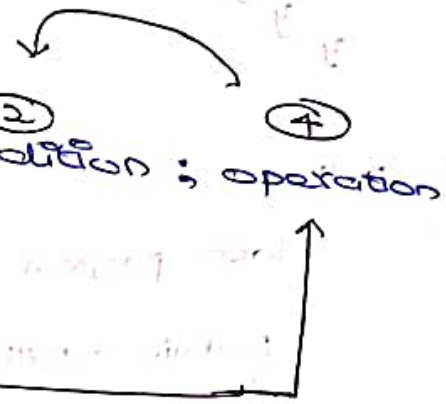
↙ ↘

For Loop — While Loop

For Loop :

Syntax

```
for (initialization ; condition ; operation
{
// for- loop body
}
```



Eg 1 :

```
class prgm1
{
   public static void main(String[] args)
   {
      S.o.p ("program starts");
      for(int i=1 ; i<=5 ; i++)
      {
         S.o.p ("Chennai ");
      }
      S.o.p ("program ends");
   }
}
```

O/P :

Program starts
Chennai
Chennai
Chennai
Chennai
Chennai
Program ends

Eg 2 :

```
class prgm2
{
   public static void main(String[] args)
   {
      for (int x = 5 ; x > 0 ; x--)
      {
         s.o.p ("@spiders");
      }
   }
}
```

O/P :

@spiders
@spiders
@spiders
@spiders
@spiders

eg:

```
class prgm3
{
  public static void main(String[] args)
  {
    for(a=1 ; a<=10 ; a=a+3)
    {
      S.o.p("9thara");
    }
  }
}
```

O/P:

9thara
9thara
9thara
9thara

eg 4:

```
class prgm4
{
  public static void main(String[] args)
  {
    for (int i=10; i>0; i=i-2)
    {
      S.o.p("Happy New Yr");
    }
  }
}
```

i=10	i>0	i=i-2
i=8	✓	
	HNY	
i=6	✓ HNY	
i=4	✓ HNY	
i=2	✓ HNY	
i=0	✓ HNY	

O/P:

Happy New Yr
Happy New Yr
Happy New Yr
Happy New Yr
Happy New Yr

eg 5:

```
class prgm5
{
  public static void main (String[] args)
  {
    int n=5;
    for(int i=0;i<n ; i++)
    {
      S.o.p("Hai");
    }
  }
}
```

i=0 i<5 i++ O/P =

i=1 ✓ Hai
i=2 ✓ Hai
i=3 ✓ Hai
i=4 ✓ Hai
i=5 ✗ Hai

Write a java prgm using for loop to display mom's name.

```
class Mom
{
   public static void main (String [] args)
   {
      int i;
      for (i=5; i>0; i--;)
      {
         S.o.p("Lalitha");
      }
   }
}
```

S.o.Pln("Hello");
Display content
↓
Move to nextLine

S.O.P ("Hello");
Display content

```
class eg1
{
   public static void main (String [] args)
   {
      int i, n=5;
      for (i=0; i<n; i++)
      {
         S.o.Pln("*");
      }
   }
}
```

O/P:

*
*
*
*
*

```
class eg2
{
  Public static void main (String[] args)
  {
    int i, n = 5;
    for (i=0; i<n; i++)
    {
      if (i%2 == 0)
      {
        S.o.p (i+" ");
      }
      else
      {
        S.o.p (0+" ");
      }
    }
  }
}
```

O/P :

1 0 1 0 1

i=0	i<n	Print
i=1	✓	1 0 1 0 1
i=2	✓	
i=3	✓	Note:
i=4	✓	%- divide
		Anything divide by 0 = 0

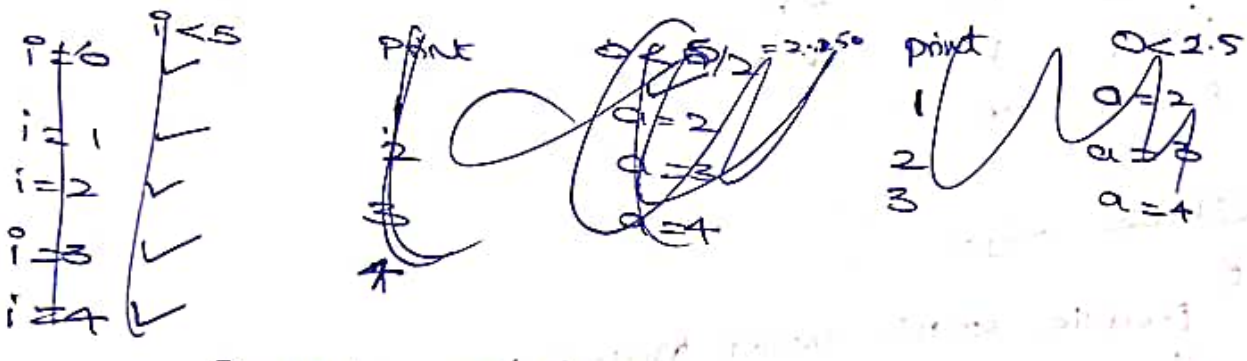
```
class eg3
{
  Public static void main (String[] args)
  {
    int i, a=0; (2) (3)
    int n = 5;
    for (int i=0; i<n; i++)
    {
      if (i%2 == 0)
      {
        S.o.p (a+" ");
        a++;
      }
      else
      {
        S.o.p ("*"+" ");
      }
    }
  }
}
```

H/W
1) ababa
2) a*b*c

i<n	Print
i=0 ✓	1 * 2 * 3
i=1 ✓	
i=2 ✓	
i=3 ✓	
i=4 ✓	

O/P:
1 * 2 * 3

```
class eg4
{
   public static void main (String[] args)
   {
      int i,n=5;
      int a=1;
      for (i=0;i<n;i++)
      {
         S.O.P(a+" ");
         if (i<n/2)
         a++;
         else
         a--;
      }
```



i<5	Print	i<2
i=0	1	a++ =a= 2
i=1	2	a++ =>a=3
i=2	3	a -- =>a=2
i=3	2	a -- =>a=1
i=4	1	

O/P:
12321

```
Class eg5
{
  Public static void main (String [] args)
  {
    int i, n=5;
    for (int=0; i<n; i++)
    {
      if (i%2==0)
      {
        S.O.P("a");
      }
      else
      {
        S.O.P ("b");
      }
    }
  }
}
```

	i<5	if	
i=0	✓	✓	a
i=1	✓	x	b
i=2	✓	✓	a
i=3	✓	x	b
i=4	✓	✓	a
i=5	*		

O/P:
abaaba

```
Class eg6
{
  Public static void main (String [] args)
  {
    int i, n=5;
    char a='a';
    for (i=0; i<n; i++)
    {
      if (i%2==0)
      {
        S.O.P(a+" ");
        a++;
      }
```

```
        else
        {
            S.O.P("*" + "  ");
        }
    }
}
```

O/P:

a*b*c

Square pattern

```
class Square1
{
    public static void main(String[] args)
    {
        int i, j, n = 5;
        for (i = 0; i < n; i++)
        {
            for (j = 0; j < n; j++)
            {
                S.O.P("*" + "  ");
            }
            S.O.println();
        }
    }
}
```

O/P:

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

Tracing

```
      j=0  j=1  j=2  j=3  j=4  j=5
i=0   *    *    *    *    *
```

again enter to 2nd loop after that loop reverts as j=0 j=1 or vice versa

```
i=1   *    *    *    *    *
      j=0  j=1  j=2  j=3  j=4
i=2   *    *    *    *    *
      j=0  j=1  j=2  j=3  j=4
i=3   *    *    *    *    *
      j=0  j=1  j=2  j=3  j=4
i=4   *    *    *    *    *

i=5
```

```
Class Square2
{
  Public Static void main (String[] args)
  {
    int i, j, n = 5;
    for (i=0; i<n; i++)
    {
      for (j=0; j<n; j++)
      {
        Scanner
        if (i%2==0)
        {
          S.o.p ("1" + " ");
        }
        else
        {
          S.o.p ("0" + " ");
        }
      }
      S.o.pln();
    }
  }
```

O/P =

```
1 1 1 1 1
0 0 0 0 0
1 1 1 1 1
0 0 0 0 0
1 1 1 1 1
```

Tracing

	j=0	j=1	j=2	j=3	j=4	j=5
i=0	1	1	1	1	1	
i=1	0	0	0	0	0	
i=2	1	1	1	1	1	
i=3	0	0	0	0	0	
i=4	1	1	1	1	1	
i=5						

```
class square3
{
  public static void main (String[] args)
  {
    int i,j,n=5;
    for (i=0;i<n;i++)
    {
      for (j=0;j<n;j++)
      {
        if (j%2==0)
        {
          S.O.P (1+" ");
        }
        else
        {
          S.O.P(0+" ");
        }
      }
      S.O.Pln();
    }
  }
}
```

Output:

```
1 0 1 0 1
1 0 1 0 1
1 0 1 0 1
1 0 1 0 1
1 0 1 0 1
```

```
class square4
{
  public static void main(String[] args)
  {
    int i,j,n=5;
    int a=1;
    for (i=0;i<n;i++)
    {
      for (j=0;j<n;j++)
      {
        S.O.P (a+" ");
      }
      S.O.Pln();
      a++;
    }
  }
}
```

O/P:

```
1 1 1 1 1
2 2 2 2 2
3 3 3 3 3
4 4 4 4 4
5 5 5 5 5
```

```
Class Squares
{
  public static void main (String[] args)
  {
     int i,j,n=5;
     int a=5;
     for(i=0;i<n;i++)
      {
        for(j=0;j<n;j++)
        {
          S.o.p(a+" ");
        }
        S.o.pln();
        a--;
      }
  }
}
```

O/P:

```
5 5 5 5 5
4 4 4 4 4
3 3 3 3 3
2 2 2 2 2
1 1 1 1 1
```

```
Class Squares
{
  Public static void main(String[] args)
  {
     int i,j,n=5;
     int a=1;
     for(i=0;i<n;i++)
     {
       for(j=0;j<n;j++)
       {
         S.o.p(a+" ");
         a++;
       }
       S.o.pln();
       a=1;
     }
  }
}
```

O/P:

```
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
```

```
Class square7
{
  public static void main (String[] args)
  {
    int i,j,n=5;
    int a=5;
    for(i=0;i<n;i++)
    {
      for(j=0;j<n;j++)
      {
        S.o.p(a+" ");
        a--;
      }
      S.o.println();
      a=5;
    }
  }
}
```

O/P:

```
5 4 3 2 1
5 4 3 2 1
5 4 3 2 1
5 4 3 2 1
```

```
Class squares
{
  public static void main (String[] args)
  {
    int i,j,n=5;
    for(i=0;i<n;i++)
    {
      for(j=0;j<n;j++)
      {
        if(i==0||i==n-1||j==0||j==n-1)
          S.o.p("*"+" ");
        else
          S.o.p("  "+" ");
      }
      S.o.println();
    }
  }
}
```

O/P:

```
* * * * *
*       *
*       *
*       *
* * * * *
```

```
class square9
{
  public static void main(String[] args)
  {
    int i,j, n=5;
    for (i=0;i<n;i++)
    {
      for (j=0;j<n;j++)
      {
        if (i==n/2 || j==n/2)
          S.O.P ("*"+" ");
        else
          S.O.P (" "+" ");
      }
      S.O.Pln();
    }
  }
}
```

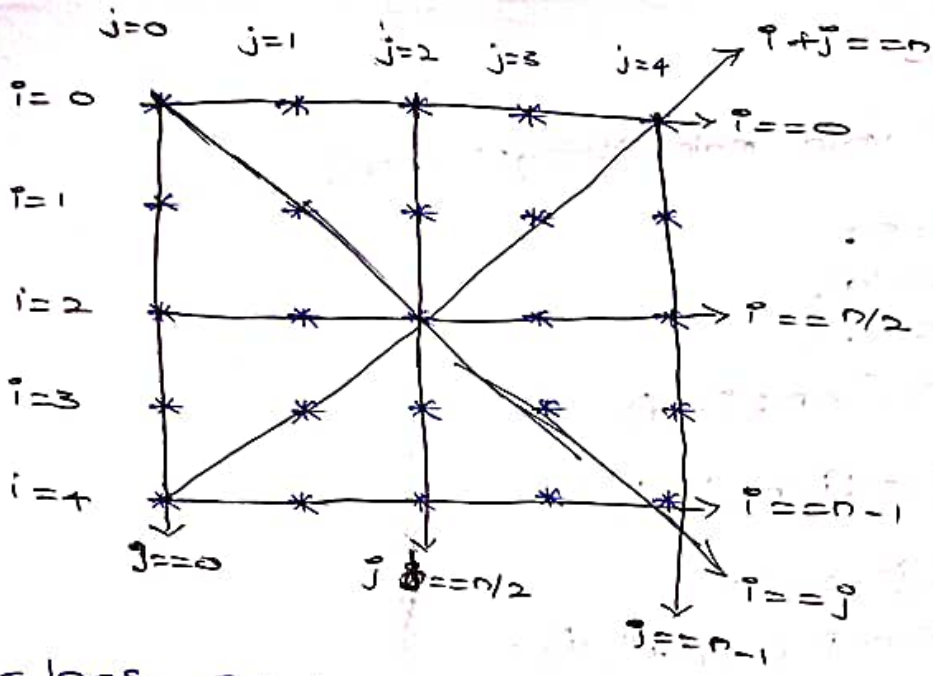
O/P:

```
    *
    *
    *
* * * * *
    *
    *
    *
```

```
class square10
{
  public static void main(String[] args)
  {
    int i,j,n=5;
    for(i=0;i<n;i++)
    {
      for (j=0; j<n;j++)
      {
        if (i==j || i+j ==n-1)
          S.O.P("*"+" ");
        else
          S.O.P (" "+" ");
      }
      S.O.Pln();
    }
  }
}
```

O/P:

```
*     *
  *  *
    *
  *  *
*     *
```




```
class Square11
{
  public static void main(String[] args)
  {
    int i, j, n = 5;
    char val = 'a';
    for(i=0; i<n; i++)
    {
      for(j=0; j<n; j++)
      {
        System.out.print(val+" ");
      }
      System.out.println();
      val++;
    }
  }
}
```

O/P :

a a a a a

b b b b b

c c c c c

d d d d d

e e e e e

```
class square12
{
  public static void main(String[] args)
  {
    int i,j,n=5;
    char val='a';
    char val2='b';
    for(i=0;i<n;i++)
    {
      for(j=0;j<n;j++)
      {
        if(j%2==0)
          S.o.p(val+" ");
        else
          S.o.p(val2+" ");
      }
      S.o.pln();
    }
  }
}
```

O/P:

```
a b a b a
a b a b a
a b a b a
a b a b a
a b a b a
```

```
class square13
{
  public static void main(String[] args)
  {
    int i,j,n=5;
    char val='a';
    char val1='b';
    for(i=0;i<n;i++)
    {
      for(j=0;j<n;j++)
      {
        if(i%2==0)
          S.o.p(val+" ");
        else
          S.o.p(val1+" ");
      }
    }
  }
}
```

O/P:

```
a a a a a
b b b b b
a a a a a
b b b b b
a a a a a
```

```
class square14
{
  public static void main(String[] args)
  {
    int i,j, n=5;
    char val='a';
    for(i=0;i<n;i++)
    {
      for(j=0;j<n;j++)
      {
        S.O.p(val+" ");
        val++;
      }
      S.O.println();
      val='a';
    }
  }
}
```

O/P:

```
a b c d e
a b c d e
a b c d e
a b c d e
a b c d e
```

```
class square15
{
  public static void main(String[] args)
  {
    int i,j, n=5;
    for(i=0;i<n;i++)
    {
      for(j=0;j<n;j++)
      {
        if(i%2==0)
        {
          if(j%2==0)
            S.O.p(1+" ");
          else
            S.O.p(0+" ");
        }
        else
        {
          if(j%2==0)
            S.O.p(0+" ");
          else
            S.O.p(1+" ");
        }
      }
      S.O.println();
    }
  }
}
```

O/P:

```
1 0 1 0 1
0 1 0 1 0
1 0 1 0 1
0 1 0 1 0
1 0 1 0 1
```

Alphapatterns [A-Z]

```
class alpha1
{
  public static void main(String[] args)
  {
    int i,j, n=5;
    for(i=0;i<n;i++)
    {
      for(j=0;j<n;j++)
      {
        if(i==0 || i==n/2 || j==0 || j==n-1)
          System.out.print("*"+" ");
        else
          System.out.print(" "+" ");
      }
      System.out.println();
    }
  }
}
```

O/P:

```
* * * * *
*       *
* * * * *
*       *
*       *
```

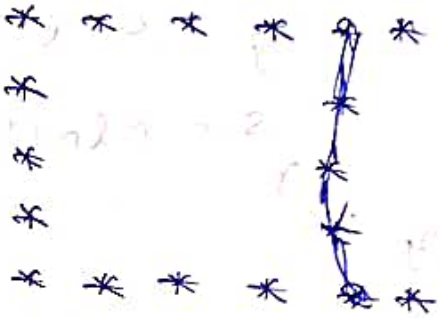
```
class alpha2
{
  public static void main(String[] args)
  {
    int i,j, n=5;
    for(i=0;i<n;i++)
    {
      for(j=0;j<n;j++)
      {
        if(i==0 || i==n/2 || i==n-1 || j==0 || j==n-1)
          System.out.print("*"+" ");
        else
          System.out.print(" "+" ");
      }
      System.out.println();
    }
  }
}
```

O/P:

```
* * * * *
*       *
* * * * *
*       *
* * * * *
```

```
class alpha3
{
  public static void main(String[] args)
  {
    int i,j,n=5;
    for(i=0;i<n;i++)
    {
      for(j=0;j<n;j++)
      {
        if(i==0||j==0||i==n-1)
          S.o.p("*"+" ");
        else
          S.o.p("  "+" ");
      }
      S.o.println();
    }
  }
}
```

O/P:



```
class alpha4
{
  public static void main(String[] args)
  {
    int i,j,n=5;
    for(i=0;i<n;i++)
    {
      for(j=0;j<n;j++)
      {
        if(i==0||j==0||i==n-1||j==n-1)
          S.o.p("*"+" ");
        else
          S.o.p("  "+" ");
      }
      S.o.println();
    }
  }
}
```

O/P:



```
class alpha5
{
  public static void main(String[] args)
  {
    int i, j, n=5;
    for(i=0; i<n; i++)
    {
      for(j=0; j<n; j++)
      {
        if(i==0 || j==0 || i==n-1 || i==n/2)
          S.o.p("*"+" ");
        else
          S.o.p(" "+" ");
      }
      S.o.println();
    }
  }
}
```

O/P:

```
* * * * *
*
* * * * *
*
* * * * *
```

```
class alpha6
{
  public static void main(String[] args)
  {
    int i, j, n=5;
    for(i=0; i<n; i++)
    {
      for(j=0; j<n; j++)
      {
        if(i==0 || j==0 || i==n/2)
          S.o.p("*"+" ");
        else
          S.o.p(" "+" ");
      }
      S.o.println();
    }
  }
}
```

O/P:

```
* * * * *
*
* * * * *
*
*
```

```
class alphar
{
  public static void main(String[] args)
  {
    int i,j, n=5;
    for(i=0; i<n; i++)
    {
      for(j=0; j<n; j++)
      {
        if(i==0||j==0||i==n-1 && j<=n/2||
            j==n/2 && i>=n/2||i==n/2 && j>=n/2
            ||j==n-1 && i>=n/2)
          S.O.P("*"+" ");
        else
          S.O.P(" "+" ");
      }
      S.O.Pln();
    }
  }
```

O/P:

```
* * * * *
*
*
*     * * *
*       *
* * * *   *
```

Basic Triangle

```
class triangle1
{
    public static void main(String[] args)
    {
        int i,j,n=5;
        for(i=0;i<n;i++)
        {
            for(j=0;j<n;j++)
            {
                if(i>=j)
                    S.o.p("*"+" ");
                else
                    S.o.p(" ");
            }
            S.o.pln();
        }
    }
}
```

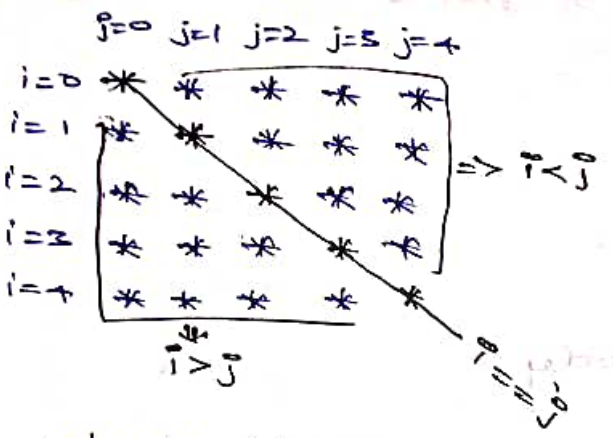
O/P:

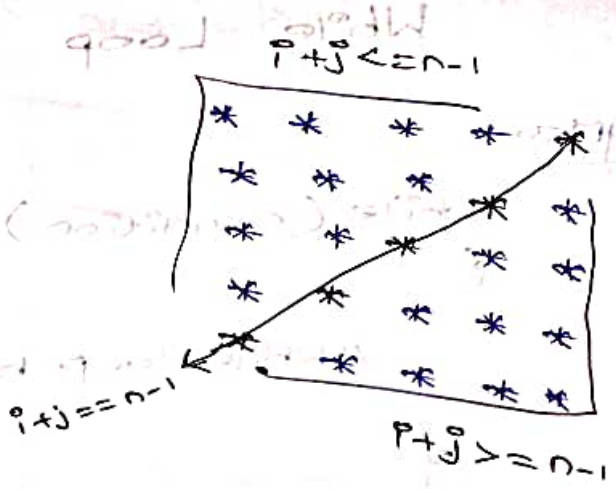
```
*
* *
* * *
* * * *
* * * * *
```

```
class triangle2
{
    public static void main(String[] args)
    {
        int i,j, n=5;
        for(i=0; i<n; i++)
        {
            for(j=0;j<n;j++)
            {
                if(i<=j)
                    S.o.p("*"+" ");
                else
                    S.o.p(" ");
            }
            S.o.pln();
        }
    }
}
```

O/P:

```
* * * * *
 * * * *
  * * *
   * *
    *
```






```
class triangle3
{
  public static void main (String[] args)
  {
    int i, j, n=5;
    for (i=0; i<5; i++)
    {
      for (j=0; j<n; j++)
      {
        if (i+j<=n-1)
          S.O.P("*"+" ");
        else
          S.O.P(" ");
      }
      S.O.Println();
    }
  }
}
```

O/P :

```
* * * * *
* * * *
* * *
* *
*
```

```
class triangle4
{
  Public static void main(String[] args)
  {
    int i, j, n=5;
    for (i=0; i<n; j++)
    {
      for (j=0; j<n; j++)
      {
        if (i+j>=n-1)
          S.O.P("*"+" ");
        else
          S.O.P(" ");
      }
      S.O.Println();
    }
  }
}
```

O/P :

```
    *
  * *
* * *
* * * *
* * * * *
```

While Loop

Syntax:

```
while (condition)
{
    // While loop body
}
```

Eg:

```
class prgm1
{
  Public static void main (String args[])
  {
    int a = 1;
    while(a <= 5)
    {
      S.O.P ("@spiders");
      a++;
    }
  }
}
```

O/P:

@spiders
@spiders
@spiders
@spiders
@spiders

Q 2:

```
class pgm2
{
  Public static void main (String args)
  {
    int x=5;
    while (x>0)
    {
      S.o.p ("chennai");
      x--;
    }
  }
}
```

O/P

Chennai
Chennai
Chennai
Chennai
Chennai

Diff btwn for loop and while-loop

For Loop	While-Loop
Initialization value is known No. of iterations is known	Initialization vale is not known No. of iterations is unknown

eg 3:

```
Class prgm3
{
  public static void main (String[] args)
  {
    int start = 1 ;
    int end = 5;
    while (start <= end)
    {
      S.O.P (start);
      start++;
    }
  }
}
```

O/P:

1
2
3
4
5

eg:4

```
Class pgm4
{
  public static void main (String[] args)
  {
    int start = 1;
    int end = 5;
    while (start <= end)
    {
      S.O.P(end);
      end--;
    }
  }
}
```

O/P:

5
4
3
2
1

Print all even numbers b/w 1-10

```
class prgms
{
  public static void main(String[] args)
  {
    int start = 1;
    int end = 10;
    while (start <= 10)
    {
      if (start % 2 == 0)
      {
        S.o.p (start);
      }
      start++;
    }
  }
}
```

O/P:

2
4
6
8
10

Print all odd numbers b/w 1-10

```
class prgms
{
  public static void main(String[] args)
  {
    int start = 1;
    int end = 10;
    while (start <= 10)
    {
      if (start % 2 != 0)
      {
        S.o.p (start);
      }
      start++;
    }
  }
}
```

O/P:

1
3
5
7
9

Class prgm6)

Write a java program to find square of all number from 1-10

```
class prgm7
{
  public static void main(String[] args)
  {
    int start = 1;
    int end = 5;
    while (start<=end)
    {
      int square = start * start;
      S.o.p (square);
      start++;
    }
  }
}
```

O/P:
1
4
9
16
25

WAJP to find cubic of all number from 1-10

```
class prgm8
{
  public static void main (String[] args)
  {
    int start = 1;
    int end = 10;
    while (start<=end)
    {
      int cube = start * start * start;
      S.o.p (cube);
      start++;
    }
  }
}
```

O/P:

Table 4th

```
class prgm 9
{
   public static void main(String[] args)
   {
      int num = 4;
      int start = 1;
      int end = 10;
      while (start <= end)
      {
         int prod = num * start;
         S.o.p(num+" * "+start+" = "+prod);
         start++;
      }
   }
}
```

O/P:

```
4 * 1 = 4
.
.
.
4 * 10 = 40
```

WAJP to find factorial of a number

```
class prgm 10
{
   P.S.V.M(S[] args)
   {
      int num = 5; // 4, 5, 2, 1, 0.
      int fact = 1; // 5, 20, 60, 120, 120
      while (num > 0)
      {
         fact = fact * num;
         num--;
      }
      S.o.p(fact);
   }
}
```

O/P =

```
120
```

The power of $5^3 = 5*5*5 \Rightarrow 125$

```
class prgm11
{
  P.S.V.M (String[] args)
  {
    int base = 5;
    int power = 3; 2, 1
    int res = 1; 5, 25, 125
    while(power >0)
    {
      res = res * base;
      Power--;
    }
    S.O.P (res);
  }
}
```

O/P:
125

Do While

Syntax:

```
do
{
  - body
}
while(condition);
```

ex:

```
class prgm1
{
  P.S.V.M (String[] args)
  {
    int a=1; // a=7;
    do
    {
      S.O.Pln("velachery");
      a++;
    }
    while(a<=5);
```

O/P:

velachery
velachery
velachery
velachery
velachery

O/P = (a=7)

velachery

```
class example1
{
  P.S.V.M(String[] args)
  {
    int x=0;
    do
    {
     S.O.P("Nayathara");
     x--;
    }
    while(x>0);
  }
}
```

O/P:

Nayathara

```
class example2
{
  P.S.V.M(String[] args)
  {
    int x=5;
    do
    {
      S.O.P("Sammy");
      x--;
    }
    while(x>0);
  }
}
```

O/P:

Sammy
Sammy
Sammy
Sammy
Sammy

Diff btwn while and do while:

While	Do While
Entry checked loop	Exit checked loop
Executes only when condition is true	Executes only atleast one, even when condition is false

num%10 == last digit

375 % 10 = 5

3974 % 10 = 4

29 % 10 = 9

64793 % 10 = 3

7 % 10 = 7

num/10 → Eliminate last digit

375 / 10 → 37

8974 / 10 → 897

29 / 10 → 2

7 / 10 → 0

Ex:

Pgm to count the numbers

```
int num = 45678; 4567, 456, 45, 4, 0
int count = 0; //1, 2, 3, 4, 5
do
{
   count++;
   num = num/10;
}
while(num>0);
S.O.P(count);
```

O/P = 5

Fibonacci series

```
int n = 5; //4
int fib1 = 0; //1.1
int fib2 = 1; //1.2
int fib3 = fib1+fib2; // 1.2
while(n>0) //5
{
   S.O.P(fib1); //0
   fib1 = fib2;
   fib2 = fib3;
   fib3 = fib1+fib2;
   n--;
}
```

O/P:

0

1

1

2

3

Reverse Number (If last num 0 not reverse)

```
int num = 8964;
int rev = 0;
while (num>0);
{
  int t= num%10;//4,6,9,8
  rev = rev * 10+t;//4,46,469,4698
  num = num/10; //896,89,8,0
}

S.O.P (rev);
```

O/P:

4698

Palindrome Number:

```
int num = 12321;
int rev = 0;
int copy= num;
while(num>0)
{
  int t= num%10;
  rev = rev * 10 + t;
  num = num/10;
}
if (copy == rev)
   S.O.P ("Palindrome");
else
   S.O.P ("Not a Palindrome");
```

O/P:

Palindrome

Palindrome - Dfn :

Palindrome number is a number that is same after reverse. ex: 545, 151, 171, 343 are Palindrome numbers. It can also be a string like LOL, MADAM etc.,

Fibonacci Series - Dfn :

In FS, next number is the sum of previous two numbers. for ex: 0, 1, 1, 2, 3, 5, 8, 13 etc. the first 2 numbers of fs are 0 and 1.

Java pgm to convert decimal to Binary

```
class pgm1
{
  public static void main(String[] args)
  {
    int num = 35;
    String bin = " ";
    while (num > 0)
    {
      int r = num % 2;
      bin = r + bin;
      num = num / 2;
    }
    S.O.P (bin);
  }
}
```

O/P:
100011

Java pgm to convert Octal to Binary

```
class pgm2
{
  Public static void main (String[] args)
  {
    int num = 35;
    String bin = " ";
```

```
while(num>0)
{
    int r = num%8;
    bin = r+bin;
    num = num/8;
}
System.out.println(bin);
```

O/P :

Perfect Divisors

Num	Divisors
18	1, 2, 3, 6, 9
24	1, 2, 3, 4, 6, 8
25	1, 5

```
class perfdivis
{
    P.S.V.M (String[] args)
    {
        int num = 18;
        int a = 1;
        while (a<=num/2)
        {
            if (num%a==0)
            {
                S.O.P (a);
            }
            a++;
        }
    }
}
```

O/P :

1
2
3
6
9

Perfect Numbers

A number whose sum of factors (excluding the number itself) is equal to the number is called a perfect number.

Ex:

Number → 28

Factors (or) Divisors → 1, 2, 4, 7, 14

Sum of Factors → 1+2+4+7+14 = 28

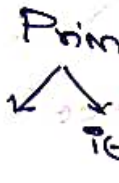
Sum of factors = Number

```
class perfect
{
  Public static void main (String[] args)
  {
    int num = 28;
    int a = 1;
    int sum = 0;
    while(a<=num/2)
    {
      if (num%a==0)
      {
        sum = sum+a;
      }
      a++;
    }
    if (sum == num)
      S.O.P ("Perfect");
    else
      S.O.P ("Not Perfect");
  }
}
```

O/P :

Perfect

Write a Java pgm to check the number is Prime or Not



Prime numbers are only divisable by 1 or the number itself. [0 and 1 are not prime number. 2 is the only prime Number]

```
Class prime
{
  Public static void main(String[] args)
  {
    Int num = 28;
    int  a=2;
    while (a<=num/2)
    {
      if(num%a==0)
      {
        break;
      }
      a++;
    }

    if(a>num/2)
       S.O.P (" Prime Number ");
    else
       S.O.P ("Not a Prime");
  }
}
```

Output:

Not a prime

Strong Number

A strong number is a special number that can be defined as an addition of factorial of each digit of the number, which is equal to the number itself.

Ex:
145

1! + 4! + 5! = 1 + 24 + 120 = 145

It's a strong Number

```
class strong
{
  public static void main(String[] args)
  {
    int num = 145;
    int sum = 0;
    int copy = num;
    while (num > 0)
    {
      int n = num % 10;
      int fact = 1;
      while (n > 0)
      {
        fact = fact * n;
        n--;
      }
      sum = sum + fact;
      num = num / 10;
    }
    if (sum == copy)
      S.O.P ("strong");
    else
      S.O.P ("Not strong");
  }
}
```

145

1! = 1

4! = 4 x 3 x 2 x 1 = 24

5! = 5 x 4 x 3 x 2 x 1 = 12

145

O/P :

strong

Armstrong Number

```
Class Armstrong
{
  Public static void main (String[] args)
  {
    int num = 153;
    int sum = 0;
    int copy = num;
    while (num > 0)
    {
      int t = num % 10;
      sum = sum + (t*t*t);
      num = num / 10;
    }
    if (sum == copy)
      S.O.P ("Armstrong");
    else
      S.O.P ("Not Armstrong");
  }
}
```

153

$1^3 = 1$

$5^3 = 125$

$3^3 = 27$

153

O/P:

Armstrong

Methods

A method is a named block of code. That is reusable in nature.

Method consists of 2 parts

- Method Declaration
- Method Definition

Syntax:

<access-modifier> <modifier> return type
methodName(<arguments>)
{

}

- A method should always be created in class body.
- A method gets executed only when it is called.
- The same method can be called multiple times.
- In a class, we can create n numb
of methods.

eg:

```
class prgm1
{
    static void test()
    {
      S.O.P("hi from test method");
    }
    Public static void main(String[] args)
    {
      S.O.P("program starts");
      test();
      S.O.P("program ends");
    }
}
```

O/P:

Program starts
hi from test method
Program ends.

eg 2:

```
class prgm2
{
    P.S.V.M(String[] args)
    {
      S.O.P("program starts");
      Play();
      Play();
      Play();
      S.O.P("program ends");
    }
    static void play()
    {
      S.O.P("Play() method executing");
    }
}
```

O/P:

Program starts
Play() method executing
"
"
Program ends

Q 3:

```
Class Pgm3
{
    static void start()
    {
    s.o.p ("start");
    static void run()
    {
      s.o.p ("run");
    static void disp()
    {
      s.o.p ("disp");
    Public static void main (String[] args)
    {
      run();
      disp();
      start();
    }
}
```

O/P:
run
disp
start

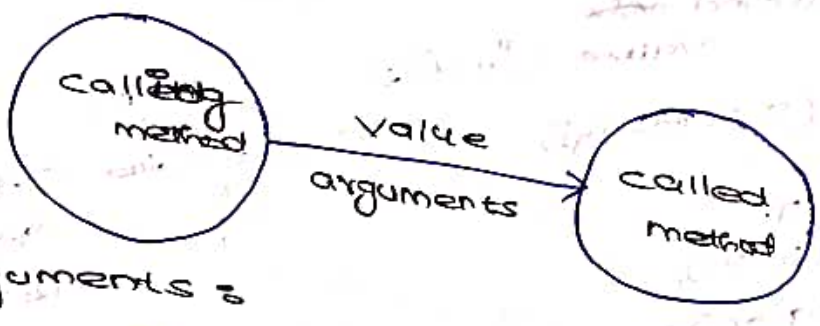
Q 4:

```
class Pgm4
{
  static void test()
  {
    s.o.p ("test start");
    s.o.p (" test ends");
  }
  static void disp()
  {
    s.o.p ("disp start");
    test();
    s.o.p ("disp ends");
  }
  public s. v. m (String[] args)
  {
    s.o.p ("main starts");
    disp();
    s.o.p ("main ends");
  }
}
```

O/P:
main starts
disp starts
test starts
test ends
disp ends
main ends

Passing values

Passing values is the concept of sending a value from calling method to called method.



Arguments :

- They are variables created in the method declaration.
- Arguments are used to pass value from calling method to called method
- We can declare multiple arguments separated by ",".

Eg :

```
class pgm1
{
  static void test(int a)
  {
    S.o.p("value : "+a);
  }
  P.S.V.m(String[] args)
  {
    test(10);
  }
}
```

O/P:

Value : 10

Eg: 2

```
class pgm2
{
  static void play(double d)
  {
    S.o.p ("Retrived value : "+d);
  }
  P.s.v.m (S.[] args)
  {
    play(7.89);
    play(9.8C);
    play(12.50);
  }
}
```

o/p:

Received value 7.89
" 9.8C
" 12.5C

Eg: 3

```
class pgm3
{
  P.S.V.M (String[] args)
  {
    for(int i=1;i<=10;i++)
    {
      square(i);
    }
  }
  static void square(int num)
  {
    int sq = num * num;
    S.O.P(num+" square is "+sq);
  }
}
```

o/p:

```
1 square is 1
2 square is 4
:  :  :
10 square is 100
```

Pg 4:

```
class prgm4
{
  public static void main(String[] args)
  {
    addition (10, 20);
    addition (300, 150);
  }
  static void addition (int num1, int num2)
  {
    int sum = num1 + num2;
    S.O.P ("Result : " +sum);
  }
}
```

O/P

Result : 30
Result : 450

Pg 5:

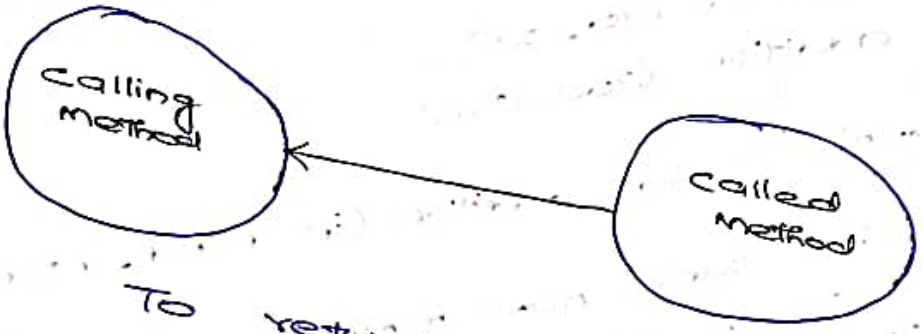
```
class prgm5
{
  static void favrat (int num, String color, boolean statue)
  {
    S.O.P (num);
    S.O.P (color);
    S.O.P (statue);
  }
  public static void main(String[] args)
  {
    favrat(10, "White", true);
  }
}
```

O/P:

10
White
true

Returning Values

Returning values is a concept of returning a value from called method to a calling method.



To return a value, we have to follow 3 steps.

- Declare returntype based on return value
- Return a compatible value using "return" keyword
- Capture the returned value in calling method.

* We use void as the return type when we do not have a return value.

* A method can only have one return type and can only return one value each time it gets called.

eg-1 :

```
class expl
{
  static int test()
  {
    return 15;
  }
  P.S.V.M (String[] args)
  {
    int val = test();
    S.O.P("Returned value : "+val);
  }
}
```

O/P :

Returned value : 15

eg-2 :

```
class exp2
{
  static double disp()
  {
    return 7.45;
  }
  P.S.V.M (String[] args)
  {
    double d = disp();
    S.O.P("Returned value : "+d);
    S.O.P (disp());
  }
}
```

O/P :

Returned value : 7.45

7.45

eg-3

```
class exp3
{
  static String play()
  {
    return "Biriyani";
  }
  P.S.V.M (String[] args)
  {
    String love = play();
    S.O.P("chicken "+love);
    S.O.P("Mutton "+play());
  }
}
```

O/P :

Chicken Biriyani

Mutton Biriyani

Eg-4

```
class exp4
{
    static int square(int num)
    {
        int sq = num * num;
        return sq;
    }
    p.s.v.m (String[] args)
    {
        int res = square(5);
        S.O.P("Result :" + res);
        S.O.P (square(7));
        S.O.P( square(6));
    }
}
```

O/P:

Result : 25
49
36

Eg-5

```
class exp5
{
    static double addition(double val1,
                           double val2)
    {
        double sum = val1 + val2;
        return sum;
    }
    P.S.V.M (String[] args)
    {
        double res = addition(1.2, 4.5);
        S.O.P (res);
        S.O.P (addition(2.5, 3.3));
    }
}
```

O/P:

5.7
5.8

ex-6:

```
class example
{
  static String run(int n)
  {
    if(n%2==0)
      return "Even";
    else
      return "odd";
  }
  p.s.v.m(String[] args)
  {
    S.O.P(run(12));
    S.O.P(run(15));
    S.O.P(run(18));
    S.O.P(run(23));
  }
}
```

O/P:

```
Even
odd
Even
odd
```

Questions

1. What are Methods?
2. Explain passing values?
3. Explain Returning values?
4. Does Java support pass by value (or) pass by reference?

- Pass by Value

Identifiers

Identifiers are names given by the programmers. The programmer is responsible for the following names.

- Classname
- Methodname
- Variablename

Rules for an identifier name

- The name should begin with an Alphabet.
- Numbers are allowed but not in the beginning.
- Special symbols "$" and "_" are allowed but not recommended.
- Keywords cannot be used as identifiers.
- Black spaces cannot be present in an identifier.

Industry standard Naming conventions

i) Classnames:

The first letter of each word in the class name should begin with an uppercase letter.

Ex: class Demo, class EmployeeProfile, class EmpSalProfile

ii) Variable and Method names:

The first word in a variable or method name is completely in lowercase and each subsequent word the first letter is in uppercase.

Ex: void test(), void generateResponse(); void updateEmployeeSalary()

int age, String dateOfBirth, long accNumber

Scanner

* Scanner is an inbuilt program present in the Java library
* The programmer can make use of scanner to read user input at the runtime.
* To make use of scanner, we have to follow 3 steps

- Input scanner from "java.util" package.
- Create an instance of scanner
- Use the inbuilt methods of scanner to read user input.

List of inbuilt Methods

Datatype	Methods Name
byte	nextByte()
short	nextShort()
int	nextInt()
long	nextLong()
float	nextFloat()
double	nextDouble()
boolean	nextBoolean()
String	next()

Note :-

Scanner doesn't provide any inbuilt methods to read "char" values directly.

Eg: 1

```
import java.util.*Scanner;
class exp1
{
  P.S.V.M(String[] args)
  {
    Scanner scn = new Scanner(System.in);
    S.o.p("Please enter a value");
    int val = scn.nextInt();
    S.o.p("value : "+val);
  }
}
```

O/P :-

Please enter a value
15
Value : 15

Eg : 2

```
import java.util.Scanner;
class exp2
{
  P.S.V.M(String[] args)
  {
    Scanner scn = new Scanner(System.in);
    S.o.p("Enter Number");
    int num = scn.nextInt();
    int square = num * num;
    S.o.p(num+" square is"+square);
  }
}
```

O/P :-

Enter Number
5
5 square is 25

Eg-3:

```
import java.util.Scanner;
class exp3
{
  public static void main(String[] args)
  {
    Scanner scn = new Scanner (System.in);
    S.O.P ("Enter Number1");
    int num1 = scn.nextInt();
    S.O.P ("Enter Number2");
    int num2 = scn.nextInt();
    int sum = num1+num2;
    S.O.P ("Result :" +sum);
  }
}
```

O/P:

```
Enter Number1
10
Enter Number2
5
Result : 15
```

Eg-4:

```
import java.util.Scanner;
class exp4
{
  P S.V.M (String[] args)
  {
    Scanner scn = new Scanner (System.in);
    S.O.P ("Enter String");
    String s = scn.next();
    S.O.P ("Enter double");
    double d = scn.nextDouble();
    S.O.P ("Enter Boolean");
    Boolean b = scn.nextBoolean();
    S.O.P ("String :"+s+" Boolean :"+b+" Double :"+d);
  }
}
```

O/P:

```
Enter String  Java
Enter double  10.55
Enter Boolean  true
String : Java  Boolean : true  double : 10.55
```

exp-5 :

```
import java.util.Scanner;
class exp5
{
  p.s.v.m (String[] args)
  {
    Scanner scn = new Scanner (System.in);
    S.O.P ("Enter the Number ");
    int n = scn.nextInt();
     if (n%2 == 0).
         S.O.P (" Even ");
      else
          S.O.P(" odd");
    }
}
```

O/P :

Enter the Number
15
odd

Ternary Operator - ?

Syntax :

Condition ? action on true : action on false;

We can use the ternary operator to replace simple if else statements

Eg:

```
int n = 15;
String s = n%2 == 0 ? " Even" : " odd";
S.O.P (s);
```

Eg :

```
int age = 26;
String s = age >= 18 ? "Major" : "Minor";
S.O.P (s);
```

Find the Difference

```
int n1 = 100;
int n2 = 120;
init diff = n1 > n2 ? n1 - n2 : n2 - n1;
S.o.P (diff);
```

Arrays

An array is a finite set of homogeneous values

finite : fixed in size

set : group of values

Homogeneous: same datatype

In Java, programmer can create arrays in 2 ways

- Using dimensions
- Using Array Initializers

Using Dimensions

In case of dimensions approach, the array is created by using the data type and size of the array.

Syntax:

```
datatype[] variablename = new datatype [int size];
```

Eg:

```
int[] arr1 = new int[10];
double[] arr2 = new double[7];
char[] arr3 = new char [5];
```

Using ArrayInitializers

In case of arrayinitializers, an array is created by providing the datatype and actual values.

Syntax:

```
datatype[] variablename = {value1, value2... valuen};
```

eg:

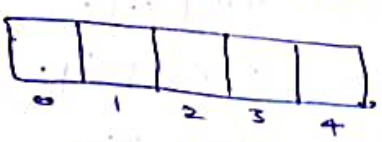
```
int[] nos = {0,2,4,6,8};
double[] cgpa = {7.5, 6.5, 7.2, 7.8, 7.7, 8.2};
char[] vowels = {'a','e','i','o','u'};
```

* Whenever an array is created, based on the datatype and size. the JVM allocats a contiguous block of memory.
* The block of memory allocated by the JVM is internally into segments of equal size.
* Each segment in the array can be accessed by using index values
  - index values always begins at 0
  - index values always ends at n-1 (n- size of the array)

eg:

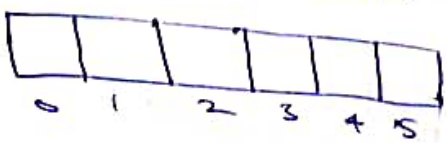
```
int[] arr = new int[5];
```

arr =>




```
int[] values = {13, 14, 15, 12, 17, 18};
```

values =>



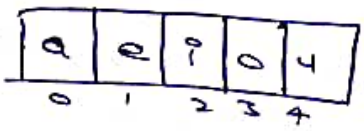
```
char[] ch = new char[7];
```

ch =>




```
Char[] vowels = {'a','e','i','o','u'}
```

vowels =>



Eg:

```
class prgm1
{
  P.S.V.m(String[] args)
  {
    in[] arr = new in[5];
      arr[0]=15;
      arr[1]=10;
      arr[2]=20;
      arr[3]=25;
      arr[4]=30;

      S.o.p(arr[0]);
      S.o.p(arr[1]);
      S.o.p(arr[2]);
      S.o.p(arr[3]);
      S.o.p(arr[4]);
```

O/P

15
10
20
25
30

```
  }
}
```

Eg.2:

```
class prgm2
{
  P.S.V.m(String[] args)
  {
    Ineffy
    Char[] ch = {'a','e','i','o','u'};
      S.o.p(ch[0]);
      S.o.p(ch[1]);
      S.o.p(ch[2]);
```

```
        S.o.P (ch[3]);
        S.o.P (ch[4]);
    }
}
```

O/P:
a
e
i
o
u

Array Length:

- Every array in java comes with a property known as length
- Using the length property, we can find size of the array

eg:

```
int[] arr1 = new int [7];
S.o.P (arr1.length); // 7
int[] arr2 = {10, 11, 13, 15, 17, 16, 12, 21, 23, 25};
S.o.P (arr2.length); // 10
char[] ch = {'a', 'c', 'v', 'j', 'e', 'i', 's', 'h'};
S.o.P( ch.length); // 8
```

ex:

```
class Pgm2
{
  P.S.V.m (String[] args)
  {
    char[] ch = {'a', 'e', 'i', 'o', 'u'};
    for (int i=0; i<ch.length ; i++)
    {
      S.o.P (ch[i]);
    }
  }
}
```

O/P:
a
e
i
o
u

ex

```
class pgm3
{
  P.S.V.M(String[]args)
  {
    int[] arr = new int[5];
    arr[0] = 15;
    arr[1] = 10;
    arr[2] = 7;
    arr[3] = 11;
    arr[4] = 12;
    for(int i=0; i<arr.length; i++)
    {
      S.O.P(arr[i]);
    }
  }
}
```

O/P:
15
10
7
11
12

ArrayIndexOutofBoundsException:

Whenever we try to access an invalid array index it causes arrayindex-outofBoundsException.

Q:

```
class expl
{
  P.S.V.M(String[] args)
  {
    int[] arr = {1,2,3,4};
    S.O.P(arr[1]);
    S.O.P(arr[2]);
    S.O.P(arr[6]);
  }
}
```

O/P:
2
3
ArrayindexoutofBounds Exception

1. Display the array elements in reverse order.

```
class pgm1
{
  public static void main (String[] args)
  {
    int[] arr = {25, 10, 12, 15, 11};
    for (int i = arr.length - 1; i >= 0; i--)
    {
      S.O.P(arr[i]);
    }
  }
}
```

O/P:
11
15
12
10
25

2. Display all the Array Elements

```
class pgm2
{
  P.S.V.M (String[] args)
  {
    int[] arr = {17, 12, 14, 13, 16, 15};
    for (int i = 0; i < arr.length; i++)
    {
      S.O.P (arr[i]);
    }
```

O/P:
17
12
14
13
16
15

3.

```
class pgm3
{
  P.S.V.M (String[] args)
  {
    char[] arr = {'T', 'i', 'g', 'e', 'r'};
    for (int i=0; i<ch.length; i++)
    {
      S.O.P (ch[i]);
    }
```

O/P:
T
g
e
r

4. Display only even numbers in the Arrays

```
class pgm4
{
  P.S.V.M(String[] args)
  {
    for(int i=0;
    int[] arr = {17,12,14,13,16,15};
    for (int i=0; i<arr.length; i++)
    {
      if (arr[i]%2 == 0)
      {
        S.o:P (arr[i]);
      }
    }
  }
}
```

O/P :
12
14
16

5. Display only odd numbers in the Array

```
class pgm5
{
  P.S.V.M(String[] args)
  {
    int[] arr = {17,12,14,13,16,15};
    for (int i=0; i<arr.length; i++)
    {
      if (arr[i]%2 != 0)
      {
        S.O.P (arr[i]);
      }
    }
  }
}
```

O/P:
17
13
15

Assignments

1) Display the prime Numbers using Array
2) Given an Array of integer to display the elements which are divisible by 5
3) Find sum of all the even numbers in array
4) Find sum of all the odd number in array
5) Find sum of all prime number

6. Find sum of all elements in array

```
class ex1
{
  P.S.V.m (String[] args)
  {
    int[] arr = {17,12,14,13,16,15};
    int sum = 0;
    for (int i=0 ; i<arr.length ; i++)
    {
      sum = sum+arr[i];
    }
    S.O.P ("sum : "+sum);
  }
}
```

O/P

sum : 87

7. Find the average value in array.

```
class ex2
{
  P.S.V.m (String[] args)
  {
    int[] arr = {17,12,14,13,16,15};
    int sum = 0;
    for (int i=0; i<arr.length ; i++)
    {
      sum = sum +arr[i];
    }
      int avg = sum/arr.length;
      S.O.P ("Average :"+ avg);
  }
}
```

O/P:

Average :

8. Sathya, Mirudhula, Jeyapriya, Jeba are 4 frds staying in chennai. Their CGPA are 9.4, 7.5, 8.4, 7.6 respectively. Pls find average Cgpa value

```
Class ex3
{
  P.S.V.M (String[] args)
  {
    double sum=0;
    double[] arr = {9.4, 7.3, 8.4, 7.0};
    for (int i=0 ; i<arr.length ; i++)
    {
      sum = sum + arr[i];
    }
    double avg = sum/arr.length ;
    S.O.P ("Average :" +avg);
  }
}
```

O/P :

Average : 8.125

9. Find largest Element in Array

```
class ex4
{
  P.S.V.M (String[] args)
  {
    int[] arr = {13,12,15,14,16,11};
    int large= arr[0];
    for(int i=1 ; i<arr.length ;i++)
    {
      if (arr[i] > large)
      {
        large = arr[i];
      }
    }
    S.O.P("Largest No :" +large);
  }
}
```

O/P :

Largest No : 16

10. Find the smallest element in Array

```
Class Pg1
{
  P.S.V.m(String[] args)
  {
    int[] arr = {13,12,15,14,16,11};
    int small = arr[0];
    for(int i=1; i<arr.length; i++)
    {
      if(arr[i] < small)
      {
        small = arr[i];
      }
    }
    S.O.P("Smallest :" +small);
  }
}
```

O/P :

Smallest : 11

11. Find the 2nd largest element in Array.

```
Class Pg2;
{
  P.S.V.m(String[] args)
  {
    int[] arr={13,12,15,14,16,11};
    int large = arr[0];
    int slarge = arr[1];
    for(int i=1; i<arr.length; i++)
    {
      if(arr[i] > large)
      {
        slarge = large;
        large = arr[i];
      }
      else if(arr[i] > slarge)
      {
        slarge=arr[i];
      }
    }
```

```
        S.O.P ("slarge :" + slarge);
    }
}
```

O/P

slarge : 15

12. Find the 2nd smallest in Array

```
class pg3
{
    P.S.V.m (String [] args)
    {
        int[] arr = {13, 12, 15, 14, 16, 11};
        int small = arr[0]; // 13, 12, 11
        int ssmall = arr[1]; // 12, 13, 12
        for (int i=1; i<arr.length; i++)
        {
            if (arr[i] < small)
            {
                ssmall = small;
                small = arr[i];
            }
            else if (arr[i] < ssmall)
            {
                ssmall = arr[i];
            }
        }
        S.O.P ("Sec small :" + ssmall);
    }
}
```

13. Find the sum of even numbers in Array:

```
class pg4
{
    P.S.V.m (String[] args)
    {
        int [] arr = {10, 15, 11, 16, 20};
        int sum = 0;
        for (int i=0; i<arr.length; i++)
        {
            if (arr[i] % 2 == 0)
            {
```

```
            sum = sum + arr[i];
        }
    }
    S.o.p("Sum of Even Numbers :" + sum);
}
```

O/P:

Sum of Even Numbers : 46

14. In a given array of integer, Display the elements which are divisible by 5.

```
class eg1
{
    p.s.v.m (String[] args)
    {
        int[] arr = {10, 15, 13, 12, 11, 14};
        for (int i=0; i<arr.length; i++)
        {
            if (arr[i] % 5 == 0)
            {
                S.o.p(arr[i]);
            }
        }
    }
}
```

O/P:

10
15

15. Display the prime numbers in an Array

```
class prime
{
    P.s.v.m (String[] args)
    {
        int[] arr = {12, 13, 14, 15, 16, 17};
        for (int i=0; i<arr.length; i++)
        {
            int num = arr[i];
            int a = 2;
            while (a <= num/2)
            {
                if (num % a == 0)
                {
                    break;
                }
                a++;
```

```
        if (a>num/2)                    O/P:
            S.O.P(num);
    }
  }
}
```

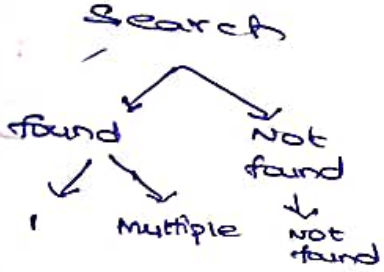
Display the prime Numbers in an Array using Methods

```
class example1
{
  P.S.V.M (String[] args)
  {
    int[] arr = {12,13,14,15,16,17};
    for(int i=0; i<arr.length; i++)
    {
      checkPrime(arr[i]);
    }
  }
  static void checkprime (int num)
  {
    int a=2;
    while (a<=num/2)
    {
      if (num%a==0)
      {
        break;
      }
      a++;
    }
    if(a>num/2
        S.O.P.(num);
    }
}
```

O/P:

Searching an element in an Array

```
Class Search
{ ps.v m (String[] args)
int[] arr = {12,13,14,12,16,17};
int element = 12;
boolean found = false;
for (int i=0; i<arr.length; i++)
{
    if (arr[i] == element)
    {
        found = true;
        S.o.p (element + "found @ index" + i);
    }
}
if (found == false)
    S.o.p ("Element Not Found";
}
}
```



O/P:
12 found @ index 0
12 found @ index 3

Sorting an element in an Array

```
Class Sort
{
    P.S.V.m(String[] args)
    {
        int[] arr = {13,12,15,16,14,17};
        S.o.p ("Before Sorting");
        for(int i=0; i<arr.length; i++)
        {
            S.o.p(arr[i]);
        }
```

```
for (int i=0; i< arr.length; i++)
{
  for (int j=i+1; j<arr.length; j++)
  {
    if (arr[i] > arr[j])
    {
      int temp = arr[i];
      arr[i] = arr[j];
      arr[j] = temp;
    }
  }
}

S.o.p ("After Sorting");
for (int i=0; i< arr.length; i++)
{
  S.o.p(arr[i]);
}
}
```

O/P:

Before Sorting

13 12 15 16 14 17

After Sorting

12 13 14 15 16 17

Reverse the order of elements in an Array

```
class arr
{
  P.S.V.m (String[] args)
  {
    int[] arr = {17, 16, 15, 19, 18};
    for (int i=0; i <= arr.length/2; i++)
    {
      int temp = arr[i];
      arr[i] = arr[arr.length-1-i];
      arr[arr.length-1-i] = temp;
    }
    for (int i=0; i<arr.length; i++)
    {
      S.o.p (arr[i]);
    }
  }
}
```

O/P:

18 19 15 16 17

Strings

String is an inbuilt class in Java, used to represent a sequence of characters.

Ex:

```
String S1 = "@spiders";
String S2 = "Software Testing";
```

String class provides several utility methods that can be used by the programmer to manipulate String values

Inbuilt Methods of String class

1. charAt()
2. length()
3. toCharArray()
4. indexOf()
5. lastIndexOf()
6. contains()
7. startsWith()
8. endsWith()
9. equals()
10. equalsIgnoreCase()
11. toUpperCase()
12. toLowerCase()
13. substring()
14. trim()
15. split()

1. charAt():

- This method returns the character at specified index from current String.
- If given index is invalid it causes StringIndexOutOfBoundException.

Ex:

```
String str = "computer";
             0 1 2 3 4 5 6 7
S.O.P (str.charAt(3)); // p
S.O.P (str.charAt(9)); // StringIndexOutOfBoundException
```

2. length():

- length() method returns the number of characters in current String

Ex:

```
String S1 = "Jeratin";
S.O.P (S1.length()); // 7
String S2 = "Shylas Jeratin";
S.O.P (S2.length()); // 14
```

length → Array - property
length → String - Method

3. toCharArray() :

- toCharArray() method creates an equivalent character Array for the current String.
- It does not convert String to character Array.

Ex:

```
String str = "software";
char[] ch = str.toCharArray();
for (int i=0; i<ch.length; i++)
{
   S.O.P(ch[i]);
}
```

O/P:

```
s
o
f
t
w
a
r
e
```

4. IndexOf() :

- Returns the index value in the current String where the character is present.
- indexOf() searches from left to right.
- We can search for multiple occurrences
- If the character is not present returns -1

Ex:

```
String str = "Developer";
             01234567 8
S.O.P (str.indexOf('l'));  // 4
S.O.P (str.indexOf('p'));  // -1
int a = str.indexOf('e');
int b = str.indexOf('e', a+1);
int c = str.indexOf('e', b+1);
S.O.P("First :"+a+" Seand :"+b+" Third :"+c);//1 3 7
```

5. lastIndexOf() :

- Returns the index value in the current String where the character is present
- lastIndexOf() searches from right to left.

Ex: ['e' and 4th point same as last one]

```
String str = "Developer";
             01234567 8
S.O.P (str.lastIndexOf('d'); // 5
S.O.P(str.lastIndexOf('m'); // -1
int a = str.lastIndexOf('e')
int b = str.lastIndexOf('e', a-1);
int c = str.lastIndexOf('e', b-1);
S.O.P("Last :"+a+"2nd :"+b+"3rd :"+c); // 7 3 1
```

6. contains():

- This method is used to check if current string contains a particular sequences of value.

Ex:

```
String str = "Butterfly";
S.o.p(str.contains("tter"); // true
S.o.P(str.contains("TT"); // false
S.o.P(str.contains("fly"); // false
```

7. startsWith():

- This method is used to check if current string begins with a particular sequences of value

8. endsWith():

- This method is used to check if current string ends with a particular sequences of value. characters

Ex:

```
String str = "Engineer";
S.o.P(str.startsWith("Eng"); // true
S.o.P(str.startsWith("eng"); // false
S.o.P(str.startsWith("ng"); // false
S.o.P(str.endsWith("neer"); // true
S.o.P(str.endsWith("hr"); // false
```

9. equals():

- This method is used to check for equality between 2 string values

10. equalsIgnoreCase():

- This method is used to check for equality between 2 string values while ignoring the case differences

Ex8

```
String str = "Amazon";
S.O.P (str.equals("Amazon")); // true
S.O.P (str.equalsIgnoreCase("Amazon")); // true
S.O.P (str.equals("AMAZON")); // false
S.O.P (str.equalsIgnoreCase("AMAZON")); // true
S.O.P (str.equals("Amazon Prime")); // false
S.O.P (str.equalsIgnoreCase("Amazon Prime")); // false
```

11. toUpperCase() :

This method returns an equivalent String where all alphabets are in uppercase

12. toLowerCase() :

This method returns an equivalent String where all alphabets are in lowercase

Ex :

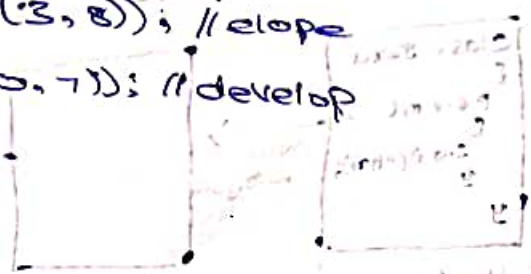
```
String str = "ChennaiIPL_125";
S.O.P(str.toUpperCase()); // CHENNAIIPL_123
S.O.P (str.toLowerCase()); // chennaiipl_123
```

13. substring() :

This method is used to derive a sequence of characters from the current String.

Ex:

```
String str = "Developer";
             0 1 2 3 4 5 6 7 8
S.O.P (str.substring(6)); // per
S.O.P (str.substring(4)); // loper
S.O.P (str.substring(3, 8)); // elope
S.O.P(str.substring(0, 7)); // develop
```



14. trim():

This method is used to eliminate unnecessary space from beginning, and ending of string

Ex:

```
String str = "  Tamil Nadu  ";
S.O.P (str.trim()); // Tamil Nadu
S.O.P (str); // Tamil Nadu
```

15. split():

This method is used to cut open the current string into several smaller strings.

Ex:

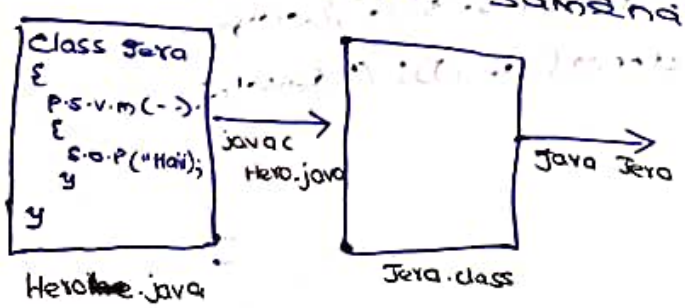
```
String str = "Software Testing is important in SDLC";
String[] arr = str.split(" ");
for(int i=0;i<arr.length;i++)
{
  S.O.P (arr[i]);
}
```

O/P:

Software
Testing
is
important
in
SDLC

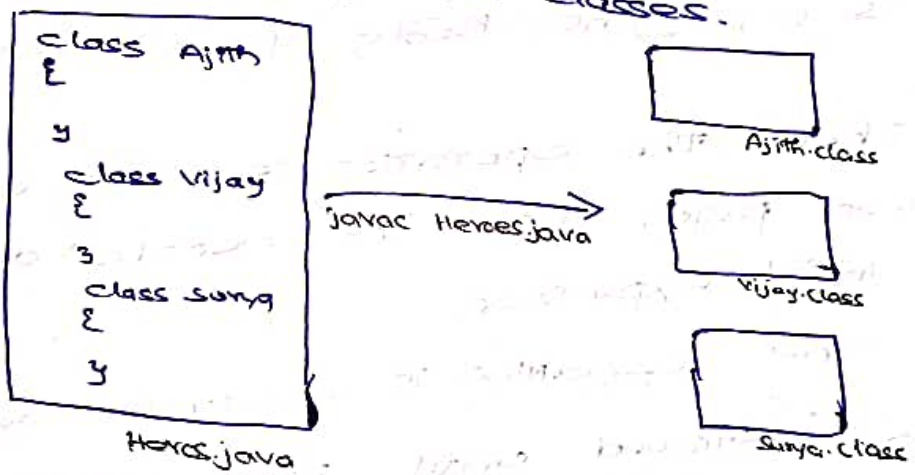
Tricks

- It is possible for the filename and classname to be different, in such scenario we should always compile with filename and execute with classname.
- The .class file generated by the compiler will always have same name as the classname



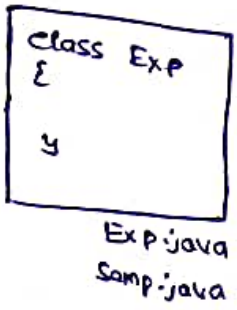
We can create, save and compile a class in java without the main method, however we cannot execute it.

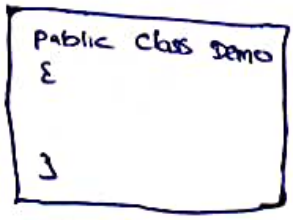
- We can create a single ".java" file with more than one class on it.
- In such a scenario the filename is decided by a programmer
- When we compile a ".java" file having more than 1 class a dedicated ".class" file is generated by the compiler for each class
- Each ".class" file that is generated will have a name as one of the classes.



Public class

- When creating a class we can prefix the keyword "public" which makes it public class
- A public class can only exist if the filename is same as the class name





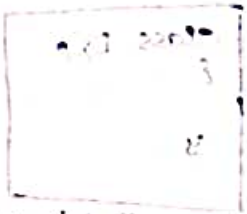
Infinite Loop

- It is a never ending loop that keeps executing without a endpoint.
- It is a programmers responsibility to prevent the formation of infinite loop.

Working of for Loop

* The initial expression initialized and declare variable and execute only once.
* The condition is evaluated, if the condition is true, the body of the loop is executed.
* Then, the operation is done according to the given prgm such increment(or) decrement value of initial expression.
* The condition is evaluated again. the prgm continued until the condition is false.





9-5-22

Static and Non-static Members

Members:

Any variable or method that is created directly in the class body is called as member.

Members are classified into two types

i) Static Members

ii) Non-static Members

Static Members:

Static members are created using "static" keyword.

Static members are accessed using the classname.

Ex:

```
Class Demo
{
    Static int val = 10;
    static void test()
    {
      S.o.p ("Executing test()...");
    }
}

Public class Program1
{
  Public static void main (String[] args)
    {
       S.o.P (Demo.val);
       Demo.test();
    }
}
```

Ex:

```
Class Sample
{
   Static double d = 4.75;
   Static char ch = 'J';
   Static void play()
   {
      S.O.P("Executing play()....");
   }
   static void disp()
   {
      S.O.P("Executing disp()....");
   }
}
public class Program2
{
   Public static void main(String[] args)
   {
      S.O.P (Sample.d);
      S.O.P (Sample.ch);
      Sample.play();
      Sample.disp();
   }
}
```

Ex:

```
Class Alpha
{
   Static int num = 25;
   Static String str = "QSpiders";
   Static void push()
   {
      S.O.P ("Executing push()....");
   }
   Static void run()
   {
      S.O.P ("Executing run()...");
   }
}
```

```
public class Program3
{
   public static void main (String[] args)
   {
      S.O.P (Alpha.num);
      S.O.P (Alpha.str);
      Alpha.push();
      Alpha.run();
   }
}
```

Non-static Members :

Non-static members are created without using "static" keyword.

Non-static members are accessed by creating an object.

Syntax for creating an object

```
Classname variable = new Classname();
```

Ex :

```
class Example
{
   int a=10;
   void test()
   {
      S.O.P ("Executing test()...");
   }
}
Public class Program4
{
   Public static void main (String[] args)
   {
      Example ex = new Example();
      S.O.P (ex.a);
      ex.test();
   }
}
```

Ex :

```
Class Delta
{
    boolean bool = true;
    char status = 'S';
    void watch()
    {
        S.o.P("Watching...");
    }
    void connect()
    {
        S.o.P("Connecting...");
    }
}
Public class Program4
{
    Public static void main(String[] args)
    {
        Delta ref = new Delta();
        S.o.P(ref.bool);
        S.o.P(ref.status);
        ref.watch();
        ref.connect();
    }
}
```

Ex :

```
Class Beta
{
    char ch = 'Q';
    double val = 9.75;
    void wish()
    {
        S.o.P("Executing wish()...");
    }
    void rush()
    {
        S.o.P("Executing rush()...");
    }
}
```

```
Public class Program6
{
  Public static void main(String[] args)
  {
     Beta bet = new Beta();
     S.O.P(bet.ch);
     S.O.P (bet.val);
     bet.wish();
     bet.rush();
  }
}
```

Ex :

```
Class Star
{
   static int a = 10;
   int b = 20;
   Static void test()
   {
      S.O.P("Executing test()....");
   }
   void disp()
   {
      S.O.P("Executing disp()....");
   }
}
Public class Program7
{
  public static void main(String[] args)
  {
     Star s = new Star();
     S.O.P (Star.a);
     S.O.P (s.b);
     Star.test();
     s.disp();
  }
}
```

Assignments

1. How to create static members?
2. How to access static members?
3. How to create non-static members?
4. How to access non-static members?
5. Syntax for creating objects?
6. Diff btwn static and non-static members

Static Members	Non-static Members
Created with "static" keyword.	Created without "static" keyword.
Accessed using classname.	Accessed using by creating an object.
Loaded to class area	Loaded to Heap area.
Loaded by classloader	Loaded directly by JVM.
Loaded during class loading	Loaded during object creation
only one copy	Multiple copies
Also known as classlevel members.	Also known as Instance members

Class and object

```
Class Student
{
   int id;
   String name;
   void study()
   {
     S.O.P("Job");
   }
   void sleep()
   {
     S.O.P("Rest");
   }
}

Student S1 = new Student();
S1.id = 101;
S1.name = "Jeraffn";
S1.sleep();

Student S2 = new Student();
S2.id = 102;
S2.name = "Vijay";
S2 study();

Student S3 = new Student()
S3.id = 103;
S3.name = "Vaishu";
S3.study();
S3.sleep();
```

Student properties

id
name
Study()
Sleep()

```
Class Employee
{
   String name;
   double ctc;
   String role;
   void work()
   {

   }
```

Propoties

name
ctc
role
work()
meeting()

```
    void meeting()
    {

    }
}

Employee e1 = new Employee();
e1.name = "Alok";
e1.ctc = 3.75;
e1.role = "Full stack Dev";
e1.work();
e1.meeting();
Employee e2 = new Employee();
e2.name = "Indhu";
e2.ctc = 3.0;
e2.role = "Test Engineer";
e2.meeting();
e2.work();
Employee e3 = new Employee();
e3.name = "Balaji";
e3.ctc = 2.5;
e3.role = "CEO";
e3.meeting();
e3.work();


class Actors
{
    String title;
    String name;
    void acting()
    {

    }
    void dancing()
    {

    }
}
```

Properties

title
name
acting
dancing

```
Actors a1 = new Actors()
a1.title = "Thala";
a1.name = "Ajith Kumar";
a1.acting();
Actors a2 = new Actors()
a2.title = "Thalaivar";
a2.name = "Rajinikanth";
a2.acting();

Class Mobile
{
   String model;
   double price;
   Void call()
   {
   }
   Void msg()
   {
   }
}

Mobile mob1 = new Mobile()
mob1.model = "Onet a";
mob1.price = 48000.0;
mob1.call();

Mobile mob2 = new Mobile()
mob2.model = "Samsung A51";
mob2.price = 21500;
mob2.msg();
mob2.call();

Mobile mob3 = new Mobile()
mob3.model = "Apple 13ProMax";
mob3.price = 145000.0;
mob3.call();
mob3.msg();
```

```
class Bike
{
  string name;
  double mileage;
  void drive()
  {
  }
  void clean()
  {
  }
}

Bike b1 = new Bike();
b1.name = "RE Continental GT 650";
b1.mileage = 20.0;
b1.drive();

Bike b2 = new Bike();
b2.name = "RX100";
b2.mileage = 28.0;
b2.drive();

Bike b3 = new Bike();
b3.name = "KTM Duke";
b3.mileage = 15;
b3.drive();
b3.clean();

class Biryani
{
    String type;
    int qty;
    double price;
    void eat()
    {
    }
    void prepare()
    {
    }
}
```

```
Biriyani b1 = new Biriyani();
b1.type = "Dindugal Biriyani";
b1.qty = 1;
b1.price = 200.0;
b1.eat();

Biriyani b2 = new Biriyani();
b2.type = "Cheerazha Sambha Biriyani";
b2.qty = 2;
b2.price = 450;
b2.prepare();
b2.eat();

Biriyani b3 = new Biriyani();
b3.type = "Hyderabad chicken Biriyani";
b3.qty = 5;
b3.price = 600.0;
b3.eat();

class LoginPage
{
    String username;
    String password;
    void login()
    {
    }
    void forgotpassword()
    {
    }
}

LoginPage l1 = new LoginPage();
l1.username = "Jero";
l2.password = "Jero123";
l1.login();

LoginPage l2 = new LoginPage();
l2.username = "Rocky-Vignesh";
l2.forgotpassword();
```

Properties:
username
Password
login()
forgot password()

```
Class InstaProfile
{
    String name;
    String bio;
    String dp;
    void follow()
    {

    }

    void msg()
    {

    }
}

InstaProfile ip1 = new InstaProfile();
ip1.name = "Shyla - Jerry";
ip1.bio = "Live Love Laugh";
ip1.dp = "my pic.jpeg";
ip1.follow();
ip1.msg();

InstaProfile ip2 = new InstaProfile();
ip2.name = "Rishika - Ritchu";
ip2.bio = "Vibe Alone";
ip2.dp = "Ritchu pic.jpeg";
ip2.msg();
ip2.follow();

class character
{
    String name;
    char gender;
    int level;
    void play()
    {

    }
    void emote()
    {

    }
void buy()
    {
    }
}
```

```
character c1 = new character();
c1.name = "Jeratin";
c1.gender = 'M';
c1.level = 11;
c1.play();
c1.emote();
c1.buy();
```

Class :

A class is an template or blueprint used to define the properties of an object

A class contains members. Members are of 2 types

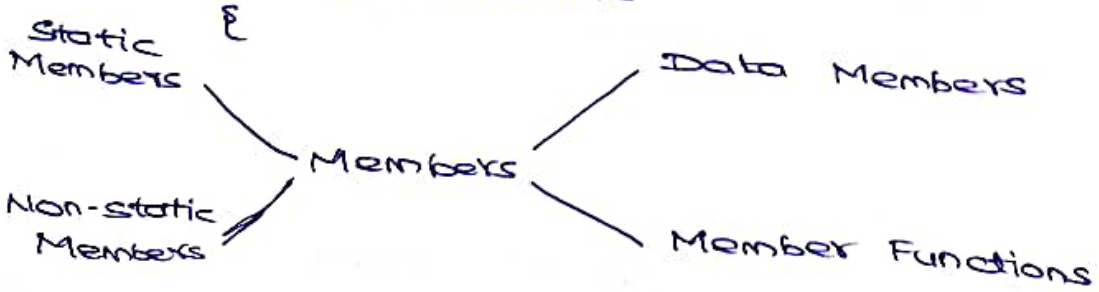
i) Data Members

ii) Member Functions

Data Members are variable that represents state of an object.

Member Functions are methods that represents behaviour of an object.

```
class classname
{
```



Object:

An object is a real world entity with its own states and behaviour

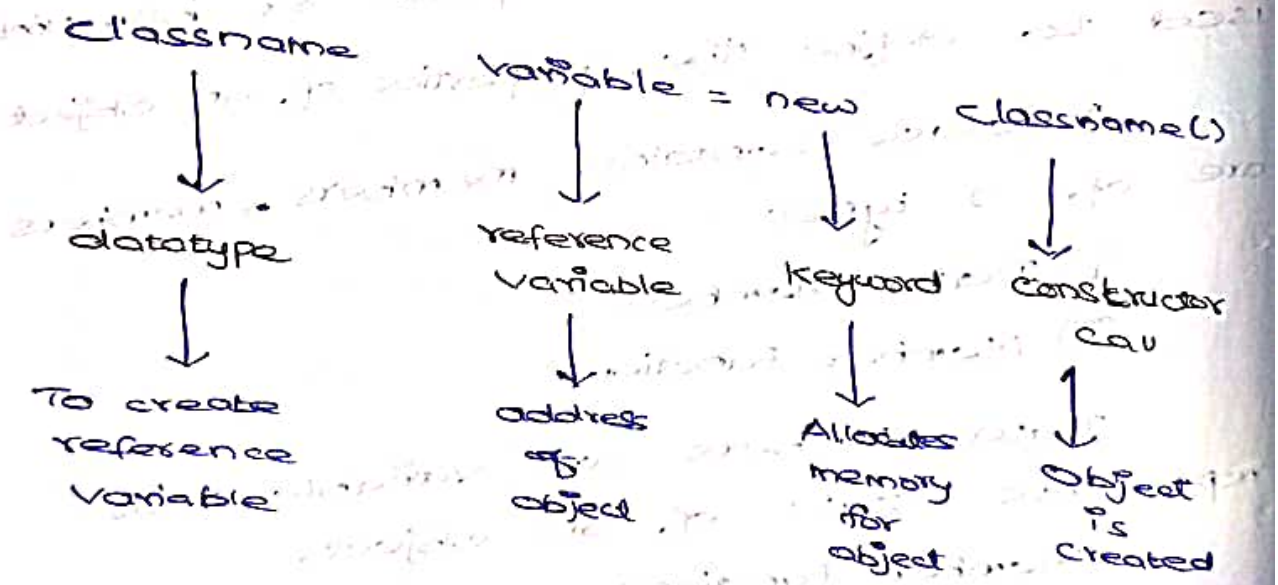
Syntax:

```
classname variable = new classname();
```

The non-static members in a class gets loaded to memory, when we create an object.

An object is nothing but a copy of a class that contains all the non-static members.

Syntax Decoding:



Deep copies and shallow copies:

Multiple reference variables of a class with each reference variable assign to a distant object.

In case of deep copies, changes made by one reference variable will not affect another one

Ex:

```
class Player
{
    String name = "Dhoni";
}

public class Program1
{
    P.S.V.M (S [] args)
    {
        Player p1 = new Player();
        Player p2 = new Player();
        S.O.P (p1.name); // Dhoni
        S.O.P (p2.name); // Dhoni
        p2.name = "Rohit";
        S.O.P (p1.name); // Dhoni
        S.O.P (p2.name); // Rohit
    }
}
```

Shallow copies:

Multiple reference variable of a class assign to a common object.

changes made by one reference variable with affect the others.

Ex:

```
Class Laptop
{
    String wallpaper = "9thara";
}
```

```
public class Program2
{
    public static void main (String[] args)
    {
        Laptop l1 = new Laptop();
        Laptop l2 = l1;
        S.o.p (l1.wallpaper); // athara
        S.o.p (l2.wallpaper); // athara
        l2.wallpaper = "Sammy";
        S.o.p (l1.wallpaper); // Sammy
        S.o.p (l2.wallpaper); // Sammy
    }
}
```

Deep copies	Shallow copies
Each reference variable will have distinct object	Each reference variable with same object,
changes made in one will not affect others.	changes made in one will affect the others

Reference Variable

Any variable that is created using classname as a datatype is called as a reference variable.

A reference variable in Java can contain either address of an object or null value.



Null :

Null is a keyword in Java language

Null meaning is Nothing

Null is default value for all reference variables.

Any reference variable assigned to null is called as null reference.

If we try any non-static members using null reference, it will cause "NullPointerException".

Ex :

```
class Actor
{
    void acting()
    {
        S.o.p("Actor is Acting");
    }
}

Public class Program3
{
    Public static void main(String[] args)
    {
        Actor a1 = new Actor();
        Actor, a2 = null;
        S.o.p (a1);
        S.o.p (a2);
        a1.acting();
        a2.acting();
    }
}
```

Questions =

1. What is a reference variable?
2. What is null?
3. When do we get NullPointerException?

Reverse a string

```
Class ReverseString
{
  Public static void main (String[] args)
  {
    String str = "chennai";
    char[] ch = str.toCharArray();
    str = "";
    for (int i = ch.length - 1; i >= 0; i--)
    {
      str = str + ch[i];
    }
    S.O.P (str);
  }
}
```

Eliminate Space

```
String str = "I love Java Coding";
charch = str.toCharArray();
str = "";
for(int i = 0; i < ch.length ; i++)
{
  if (ch[i] != ' ')
  {
    str = str + ch[i];
  }
}
S.O.P (str);
```

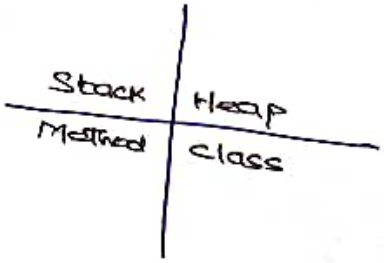
Memory Management in Java

In Java language, end to end memory management is handled by JVM in dynamic fashion.

During execution of Java program, the JVM makes use of following memory areas.

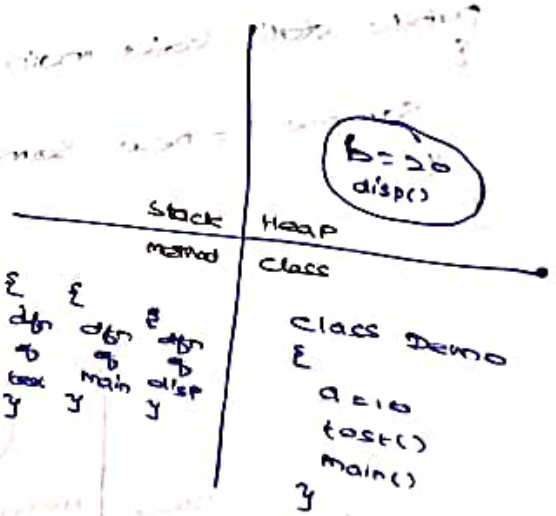
* Class area
* Heap area
* Method area
* Stack area

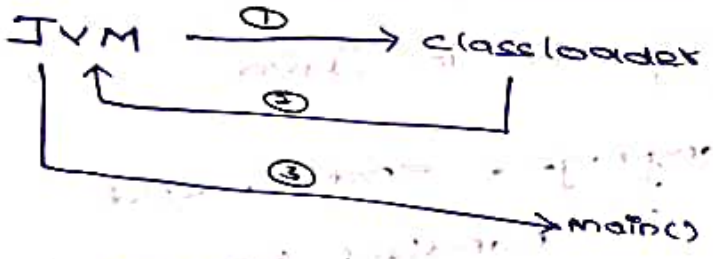
Memory Map



Ex:

```
class Demo
{
  static int a = 10;
  int b = 20;
  static void test()
  {
  }
  void disp();
  {
  }
  Public static void main(String[] args)
  {
    Demo ref = new Demo();
  }
}
```

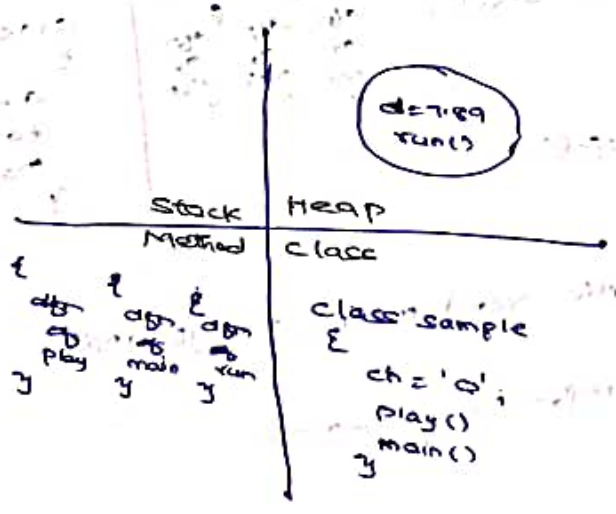




- Static members are executed with a help of classloader (one of subsystem of JVM) and stored it in a class area.
- Non static members are executed only when the object is created and stored in Heap area.
- Both static & non-members, Definition of a method function is stored in Method area.

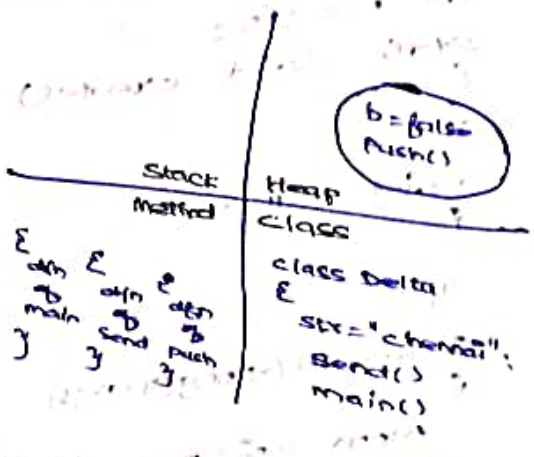
Ex:

```
class sample
{
  static char ch='Q';
  double d=7.89;
  static void play(){
  {
   }
   void run()
   {
   }
  public static void main(String[] args
  {
    Sample s = new Sample();
  }
}
```



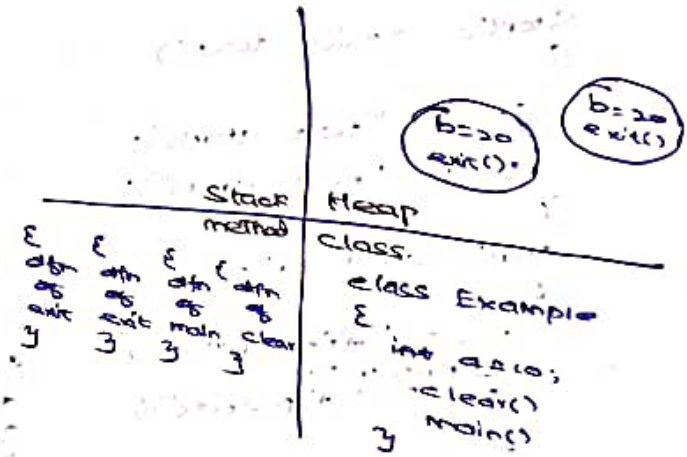
Ex :-

```
class Delta
{
  static String str = "chennai";
  boolean b = false;
  void push()
  {
  }
  static void send()
  {
  }
  P.S.V.M (String[] args)
  {
    Delta ref = new Delta();
  }
}
```



Ex :-

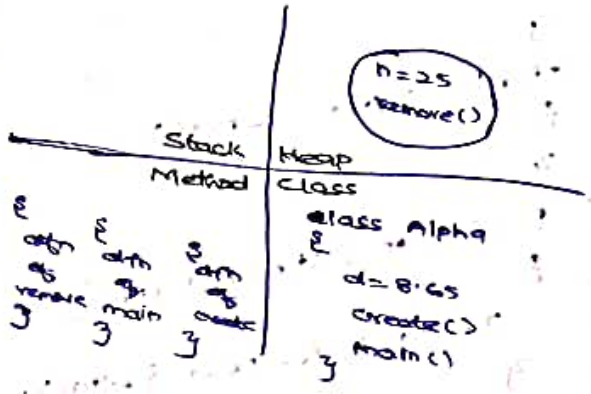
```
class Example
{
  static int a=10;
  int b=20;
  static void clear()
  {
  }
  void exit()
  {
  }
  P.S.V.M (String[] args)
  {
    Example ex1 = new Example();
    Example ex2 = new Example();
  }
```



Static members are stored only once in the area. But Non-static members are stored multiple time in the area; by creating many objects.

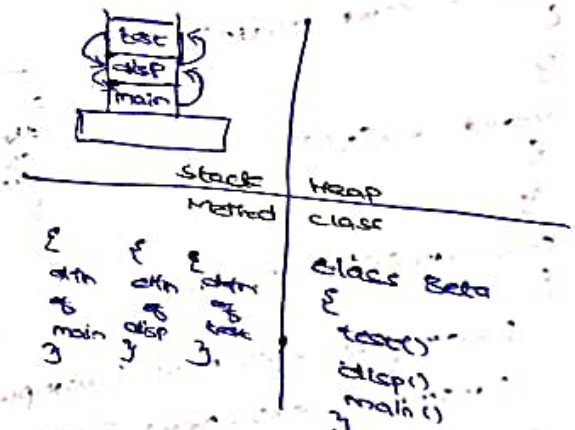
Ex :

```
Class Alpha
{
  static double d = 8.65;
  int n = 25;
  Static void create()
  {
  }
  void remove()
  {
  }
  P.S.V.M (String[] args)
  {
    Alpha obj = new Alpha();
  }
}
```



Ex :

```
Class Beta
{
  Static void test()
  {
    S.O.P("test starts");
    S.O.P("test ends");
  }
  static void disp()
  {
    S.O.P("disp starts");
    Beta.test();
    S.O.P("disp ends");
  }
  P.S.V.M (String[] args)
  {
    S.O.P("main starts");
    Beta.disp();
    S.O.P("main ends");
  }
```



Exit
① test
② disp
③ main

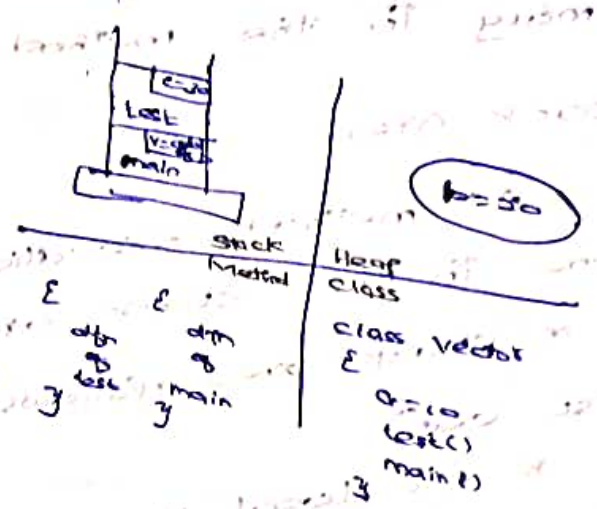
Entry
① main
② disp
③ test

Stack is a execution memory :

- All methods are executed here (Beta .static and non-static)
- It executes in LIFO (Last In First Out) Order.

Ex 8

```
class Vector
{
    static int a = 10;
    int b = 20;
    static void test()
    {   int c = 30;
        S.o.P("test()..." + c);
    }
    P.S.V.M (String[] args)
    {   Vector.test();
        Vector v = new Vector();
    }
}
```



Class Area

* All static members are allocated to memory in the class area.
* The static members are allocated to memory by the class loader.
* The static members are loaded to memory during class loading.
* The static members are allocated to memory only once.

Heap Area

* All objects created in Java are allocated memory in Heap area.
* The non-static members are loaded to heap area.
* The non-static members are allocated to memory during object creation.
* Each time we create an object a copy of non-static members are loaded to memory.

Method Area

All method definitions are allocated memory in the method area.

Stack Area

* All method executions in Java is done in the stack area.
* Stack area follows L.I.F.O (Last In First Out)
* All local variables are allocated memory in the stack area.
* Stack area is known as the execution memory

Pyramid

```
class Pyramid
{
  P.S.V.M (String[] args)
  {
    int n=5;
    int spc=n-1;
    int str=1;
    for(i=0; i<n; i++)
    {
      for(int j=0; j<spc; j++)
      {
        S.O.P (" ");
      }
      for(int k=0; k<str; k++)
      {
        S.O.P ("*");
      }
      S.O.Pln();
      spc--;
      str=str+2;
    }
  }
}
```

O/P

```
    *
   ***
  *****
 *******
*********
```

Initialization Blocks

Initialization Blocks are used to initialize Data Members.

We have 2 types of Initialization blocks

- Static Initialization Blocks
- Non-static Initialization Blocks.

Static Initialization Blocks :

* Static blocks are created using the static keyword.
* Static blocks are used to initialize static data members.
* We can initialize multiple static data members in a single static block.
* We can create multiple static blocks inside a java class.
* Static blocks are executed only once in its lifetime.

Ex 1 :

```
Class Demo
{
  Static int a;
  Static
  {
    a=10;
  }

  Public static Void main(String[] args)
  {
    S.O.P ("pgm starts");
    S.O.P (Demo.a);
    S.O.P ("pgm ends");
  }
}
```

Ex 28

```
Class Sample
{
   Static char ch;
   static double val;
   Static
   {
     ch= 'J';
     val = 6.3;
   }
   P.S.V.M (String[] args)
   {
     S.o.P ("pgm starts");
     S.o.P (Sample.ch); // J
     S.o.P(Sample.val); // 6.3
     S.o.P("pgm ends");
   }
}
```

Ex 29

```
Class Alpha
{
  Static int num;

  Static
  {  num= 10;
  }
  Static
  {
    num=20;
  }
  Static
  {
    num=30;
  }
  P.S.V.M(String[] args)
  {
    S.o.P ("pgm starts");
    S.o.P ( Alpha.num); // 30
    S.o.P ("Alpha. ends");
  }
}
```

Ex 4:

```
class Example
{
  static int x;
  static
  {
    S.o.P ("Executing static block");
    x = 25;
  }
  P.s.v.m (String[] args)
  {
    S.o.P ("Program start");
    S.o.P (Example.x); // 25
    S.o.P ("Program ends");
  }
```

Non-static Initialization Blocks:

* Non-static blocks are created without using "static" keyword.
* Non-static blocks are used to initialize non-static data members.
* We can initialize multiple non-static members in a single non-static block.
* It is possible to create multiple non-static blocks inside class.
* Non-static blocks are executed during object creation.
* The non-static blocks may execute multiple time.

Ex 1:

```
class Demo
{
  int a;
  {
    a = 10;
  }
  P.s.v.m (String[] args)
  {
    S.o.P ("Prgm starts");
    Demo ref = new Demo();
    S.o.P (ref.a); //10
    S.o.P ("Prgm ends");
  }
}
```

Ex 2 :

```
class Sample
{
    String str;
    boolean bool;
    {
      str = "chennai";
      bool = true;
    }
    P.S.V.M (String[] args)
    {
      S.O.P ("prgm starts");
      Sample s = new Sample();
      S.O.P (s.str); // chennai
      S.O.P (s.bool); // true
      S.O.P ("prgm ends");
    }
}
```

Ex 3 :

```
class Alpha
{
    int ml;
    {
      ml = 50;
    }
    {
      ml = 60;
    }
    {
      ml = 90;
    }
    P.S.V.M (String[] args)
    {
      S.O.P ("prgm starts");
      Alpha a = new Alpha();
      S.O.P (a.ml); // 90
      S.O.P ("prgm ends");
    }
}
```

Ex 4 :

```
Class Example
{
  int x;
  {
    S.O.P("Executing non-static block");
    x=25;
  }

  P.S.V.M(String[] args)
  {
    S.O.P("Pgm starts");
    Example ex = new Example();
    S.O.P(ex.x); // 25
    S.O.P("Pgm ends");
  }
}
```

Ex 5 :

```
Class Delta
{
  static
  {
    S.O.P("static block");
  }
  {
    S.O.P("Non-static block");
  }

  P.S.V.M(String[] args)
  {
    S.O.P("Pgm starts");
    Delta d1 = new Delta();
    Delta d2 = new Delta();
    S.O.P("Pgm ends");
  }
}
```

O/P :

```
Static block
Pgm starts
Non-static block
Non-static block
Pgm ends
```

When to make use of initialization blocks?

Whenever the data members in a class needs to be initialize through a input source (user input, database, resource) we should use initialization blocks.

Constructor

* Constructors are special members in a class used to create object.
* Every class in java will have a constructors.
* Constructor will always have the same name as classname.
* Constructors can be created either by the programmer (or) by the compiler.
* Constructors can also be used to initialize non-static data members.
* In Java, we have 3 types of constructors

i) Default constructor
ii) Zero Argument constructor
iii) Parameterized constructor.

Default constructor :

- The default constructor is created by the compiler.
- The compiler creates Default constructor when the programmer fails to create a constructor
- The default constructor provides default initialization for the non-static data members.

- The default constructor cannot accept arguments.

Ex =

```
Class Demo
{
  int a;
  Void test()
  {
    S.O.P("Executing");
  }
}
Public class Program1
{
  P.S.V.M (String[] args)
  {
    Demo ref=new Demo();
    S.O.P (ref.a);
    ref.test();
  }
}
```

javap Demo

```
class Demo
{
  int a;
  void test
  {
    S.O.P();
  }
  Demo();
}
```

O/P

```
0
Executing
```

Ex =

```
class Sample
{
  int num;
  double val;
  char ch;
}
Public class Program2
{
  P.S.V.M (String[] args)
  {
    Sample S= new Sample();
    S.O.P(S.num); // 0
    S.O.P (S.val); // 0.0
    S.O.P (S.ch); //
  }
}
```

Other Non-Primitive dataType → null

byte, short, int, long → 0
float, double → 0
boolean → false
char → [\u0000] → appears as empty space

Zero Argument Constructor:

- The ZAC is created by the programmer.
- We can initialize non-static data members in the body of zero argument constructor.
- As the name suggests, it does not accept any arguments.

All objects created with ZAC will have same initialization value.

Ex =

```
Class Alpha
{
  int a;
  void disp()
  {
    S.o.p("Executing ");
  }
  Alpha()
  {
    S.o.p("Alpha() constructor");
    a=10;
  }
}

public class Program5
{
  P.s.v.m(String[] args)
  {
    Alpha ref=new Alpha();
    S.o.p(ref.a); // 10
    ref.disp(); // Executing
  }
}
```

Ex :

```
class Beta
{
    String str;
    boolean bool;
    double doub;
    Beta()
    {
        str = "chennai";
        bool = true;
        doub = 6.9;
    }
}
public class Program4
{
    P.S.V.M (String[] args)
    {
        Beta b = new Beta();
        S.O.P(b.str); // chennai
        S.O.P(b.bool); // true
        S.O.P(b.doub); // 6.9
    }
}
```

Parameterized Constructor :

- The PC is created by the programmer
- In a PC, we can initialize non-static data members with the help of arguments.
- When using a PC, we can create objects for the same class with a different initialization values.

Ex :

```
class Square
{
  int side;
  void draw()
  {
  }
  Square(int arg)
  {
    side = arg;
  }
}
public class Programs
{
  P.S.V.M (String[] args)
  {
    Square S1 = new Square(10);
    S.O.P (S1.side); // 10
    Square S2 = new Square(20);
    S.O.P (S2.side); // 20
    Square S3 = new Square(15);
    S.O.P(S3.side); // 15
  }
}
```

Ex :

```
class Product
{
  int pid;
  double price;
  Product (int arg1, double arg2)
  {
    pid = arg1;
    price = arg2;
  }
}
public class Program6
{
  PS.V.M (String[] args)
  {
    Product P1 = new Product (8546, 25000.0);
    S.O.P (P1.pid); // 8546
    S.O.P (P1.price); // 25000.0
  }
}
```

Constructor Overloading

Creating more than one constructor in a class is called as constructor overloading

If we are doing constructor overloading, it is mandatory to vary the parameter for every constructor.

We can vary the arguments in 3 ways

1. Type of Arguments
2. Length of Arguments
3. Sequence of Arguments.

Constructor overloading can be used to create objects of the same class with different abilities

Type of Arguments:

Ex 1:

```
Class Hotel
{
  Hotel (int arg)
  {
    S.o.P("KFC");
  }
  Hotel (double arg)
  {
    S.o.P ("MCD");
  }
  Hotel (String arg)
  {
    S.o.P ("Wedding Biryani");
  }
}
```

```
Public class Program1
{
    P.S.V.M (String[] args)
    {
        Hotel h1 = new Hotel(10);
        Hotel h2 = new Hotel(1.5);
        Hotel h3 = new Hotel("ABC");
    }
}
```

O/P

KFC

MCD

Wedding Biryani

Length of Arguments.

Ex: 2:

```
Class city
{
    City(int arg1)
    {
        S.O.P("New York");
    }
    City(int arg1, int arg2)
    {
        S.O.P("London");
    }
    City(int arg1, int arg2, int arg3)
    {
        S.O.P("Rio");
    }
}
Public class Program2
{
    P.S.V.M()
    {
        City c1 = new City(10);
        City c2 = new City(10,11);
        City c3 = new City(12,13,14);
    }
}
```

O/P

New York

London

Rio

Sequence of Arguments

Ex 3 =

```
class Institute
{
  Institute(int a, String s)
  {
    S.O.P("Qspiders");
  }
  Institute(String s, int a)
  {
    S.O.P("Jspiders");
  }
}
public class InstituProgram3
{
  P.S.V.M (String[] args)
  {
    Institute I1 = new Institute(25, "Testing");
    Institute i2 = new Institute("Dev", 18);
  }
}
```

O/P

Qspiders

Jspiders

Ex 4 =

```
class Amazon
{
  Amazon()
  {
    S.O.P("Shopping");
  }
  Amazon(double ~~fees~~ args)
  {
    S.O.P("Shopping, Prime, Music");
  }
}
public class Program
{
  P.S.V.M (String[] args)
  {
    Amazon a1 = new Amazon();
    Amazon a2 = new Amazon(999.0);
  }
}
```

Ex 5 :

```
Class Employee
{
    Employee (double ctc)
    {
        S.o.p("Software Engineer");
    }
    Employee (double ctc, int bonus)
    {
        S.o.p ("Lead Engineer");
    }
    Employee (double ctc, int bonus, double share)
    {
        S.o.p ("Manager");
    }
}

Public class program
{
    P.S.V.M (String[] args)
    {
        Employee e1 = new Employee (3.5);
        Employee e2 = new Employee (8.5, 3);
        Employee e3 = new Employee (17.5, 5, 2.5);
    }
}
```

Ex 6 :

```
Class Account
{
    Account (double money, String pan, long
    {                                    aadhar)
        S.o.p ("Savings Acc");
    }
    Account (String offerletter, String pan, long
    {                                    aadhar)
        S.o.p ('Salary Acc");
    }
}
```

Passing and Returning objects

Ex :

```
Class Hotel
{
  static Biriyani prepare()
  {
    Biriyani b = new Biriyani();
    return b;
  }
  Static void display(Biriyani a)
  {
    a.eat();
  }
}

Public class Program
{
  P.S.V.M (String[] args)
  {
    Biriyani b1 = Hotel.prepare();
    Hotel.display(b1);
    Biriyani b2 = Hotel.prepare();
    Hotel.display(b2);
  }
}
```

```
Class Biriyani
{
  void eat()
  {
  }
}
```

Ex :

```
import java.util.Scanner;
Class HRDepartment
{
  static void displayEmployee(Employee emp)
  {
    S.O.P("Name :" + emp.name);
    S.O.P("CTC :" + emp.ctc);
    S.O.P("ID :" + emp.id);
  }
  Static Employee createEmployee()
  {
    Scanner scn = new Scanner(System.in);
    S.O.P("Enter the name");
    String name = scn.next();
    S.O.P("Enter the ctc");
    double ctc = scn.nextDouble();
    S.O.P("Enter the id");
    int id = scn.nextInt();
```

```
Employee emp = new Employee(name, ctc, id);
return emp;
}
}

Employee e1 = Hr.Department.createEmployee();
HrDepartment.displayEmployee(e1);
Employee e2 = Hr.Department.createElement();
HrDepartment.displayEmployee(e2);

class Employee
{
    String name;
    double ctc;
    int id;
    void work()
    {
    }
    Employee(String s, double d, int i)
    {
        name = s;
        ctc = d;
        id = i;
    }
}
```

Ex :-

```
class Product
{
    int bid;
    double price;
    String brand;
    Product(int i, double d, String s)
    {
        bid = i;
        price = d;
        brand = s;
    }
}
```

```
import java.util.Scanner;
Public class Flipkart
{
  static void displayProductDetails(Product p);
  {
    S.O.P("Pid : "+p.pid);
    S.O.P("Price : "+p.price);
    S.O.P("Brand : "+p.brand);
  }
  static Product purchase()
  {
    Scanner scn = new Scanner(System.in);
    S.O.P("Enter pid");
    int pid = scn.nextInt();
    S.O.P("Enter Price");
    double price = scn.nextDouble();
    S.O.P("Enter brand");
    String brand = scn.next();
    Product p = new Product(pid, price, brand);
    return p;
  }
}
Product p1 = Flipkart.purchase();
Flipkart.displayProductDetails(p1);
Product p2 = Flipkart.purchase();
Flipkart.displayProductDetails(p2);
```

Ex :

```
class Account
{
  long accNo;
  String pan;
  long aadhar;
  Account(long l, String p, long a);
  {
    accNo = l;
    pan = p;
    aadhar = a;
  }
}
```

```
import java.util.Scanner;
class IciciT
{
    void
    static Account createAccount()
    {
      Scanner scn = new Scanner(System.in);
       S.o.p("Enter the Acc No");
       long accNO = scn.nextLong();
       S.o.p("Enter the PAN");
       String pan = scn.next();
        S.o.p("Enter Aadhar");
        long aadhar = scn.nextLong();
         Account acc = new Account(accNO, pan,
                                   aadhar);
         return acc;
      }

      Static void displayAccountDetails(Account acc)
      {
        S.o.p("Acc Num :" + acc.accNO);
        S.o.p("PAN :" + acc.pan);
        S.o.p("Aadhar :" + acc.aadhar);
      }
    }

    Account a1 = Icici.createAccount();
    Icici.displayAccountDetails(a1);
```

① Add 2 number without using + operator

② If a number is divisible by 7 without using modulus or division operator.

Copy constructor

- It is a special type of a constructor used to create replica of an existing object.
- The copy constructor is created by the programmer.
- The copy constructor will take an existing object of the current class as an argument and creates a replica for it.
- The copy constructor should be created along with zero Argument (or) Parametized constructor.

Ex 1:

```
class circle
{
  int radius;
  Circle( int arg)
  {
    radius = arg1;
  }
  Circle (Circle c) // copy constructor
  {
    radius = c.radius;
  }
}
Public class Pgm
{
  P.S.V.M (String[] args)
  {
    Circle c1 = new Circle(15);
    Circle c2 = new Circle(c1);
    S.O.P (c2.radius);
  }
}
```

Ex 2:

```
Class Employee
{
  String name;
  int id;
  double etc;
```

```
Employee()
{
    name = "Jeru";
    id = 7;
    ctc = 7.0;
}
Employee(Employee emp) // copy constructor
{
    name = emp.name;
    id = emp.id;
    ctc = emp.ctc;
}
Employee e1 = new Employee();
S.o.p(e1.name+" "+e1.id+" "+e1.ctc);
Employee e2=new Employee(e1);
S.o.p(e2.name+" "+e2.id+" "+e2.ctc);
```

Ex 3 :

```
Class Mobile
{
    String brand;
    double price;
    String model;
    Mobile(String arg1, double arg2, String arg3)
    {
        brand = arg1;
        Price = arg2;
        model =arg3;
    }
    Mobile(Mobile mob) // copy constructor
    {
        brand = mob.brand;
        price =mob.price;
        model = mob.model;
    }
}

Mobile m1 = new Mobile("Samsung", 23, "A32");
S.o.p(m1.brand+" "+m1.price+" "+m1.model);
Mobile m2 = new Mobile(m1);
S.o.p(m2.brand+" "+m2.model+" "+m2.price);
```

Questions

1. What is a constructor?
2. Explain - constructor overloading?
3. What is a copy constructor
4. Diff btwn default, zero Argument & parameterized constructor?

Different ways to initialize non-static data members.

```
Class Product
{
  int pid = 101;
  double price = 750.0;
}
```

Initialization is done before object is created

```
Class Product
{
  Pnt pid;
  double price;
  {
    Pid = 101;
    Price = 750.0;
  }
}
```

Initialization is done during object creation

```
Class Product
{
  int pid;
  double price;
  Product(int arg1, double arg2)
  {
    pid = arg1;
    price = arg2;
  }
}
```

Initialization is done during object creation using arguments

```
class product
{
  int pid;
  double price;
}

Product P1 = new Product();
P1.pid = 101;
P1.price = 750.0;
```

Initialization is done after object creation

Add 2 nos without using + operator

```
int a,b,c;
a = 10;
b = 15;
c = a - (-b);
S.O.P (c);
```

Divide a number without using division or modulus operator

```
int num = 15;
int n = 5;
while (num > 0)
{
  num = num - n;
}
if (num == 0)
  S.O.P("It is divisible");
else
  S.O.P("Not divisible");
```

Reverse Pyramid

```
class ReversePyramid
{
  P.S.V.M (String[] args)
  {
    int i,j,k;
    int n = 5;
    int spc = n-1;
    int str = 9;
    for (i=0; i<n; i++)
    {
      for (j=0; j<spc; j++)
      {
        S.O.P(" " + " ");
      }
      for (k=0; k<str; k++)
      {
        S.O.P("*" + " ");
      }
      S.O.Pln ();
      spc++;
      str-=2;
    }
  }
}
```

O/P

```
* * * * * * * * *
 * * * * * * *
  * * * * *
   * * *
    *
```

Inheritance

- Inheritance is the concept of one class acquiring the members of an another class
- Inheritance is done using the keyword "extends".
- Inheritance is also known as Is-A Relationship.

```
Class A
{
}
```
- Super
- Base
- Parent

```
Class B extends A
{
}
```
- Sub
- Derived
- Child

Ex 1 :

```
Class Demo
{
  void test()
  {
    S.o.p("Executing test()...");
  }
}
Class Sample extends Demo
{
}
Public class Program1
{
  P.s.v.m(String[] args)
  {
    Sample ref = new Sample();
    ref.test();
  }
}
```

Ex 28

```
Class Alpha
{
  Int x = 10;
  Void run()
  {
    S.O.P ("Executing run()..");
  }
}
Class Beta extends Alpha
{
  Int y = 20;
  Void play()
  {
    S.O.P ("Executing play()..");
  }
}
Public class Program2
{
  P.S.V.M (String[] args)
  {
    Alpha ref = new Alpha();
    ref.run();
    S.O.P (ref.x);
    Beta obj = new Beta();
    obj.run();
    S.O.P (obj.x);
    obj.play();
    S.O.P (obj.y);
  }
}
```

Rules of Inheritance :-

- The sub class cannot inherit the following members of the superclass.
  1. Private
  2. Static → loophole in java programming.
  3. Constructors
  4. Initialization Block
- The super class cannot access the properties of sub class.

- If a class is final, it cannot have sub class

Uses of Inheritance :

* Code Reusability
* Software Enhancement
* Code Modification
* Generalization Vs Specialization

Ex 1 :

```
V1
class Insta1
{
  void uploadPic()
  {
  }
}

V2
class Insta2 extends Insta1
{
  void messenger()
  {
  }
}

V3
class Insta3 extends Insta2
{
  void story()
  {
  }
}
```

Ex 2 :

```
V1
class WA1
{
  void message()
  {
  }
}

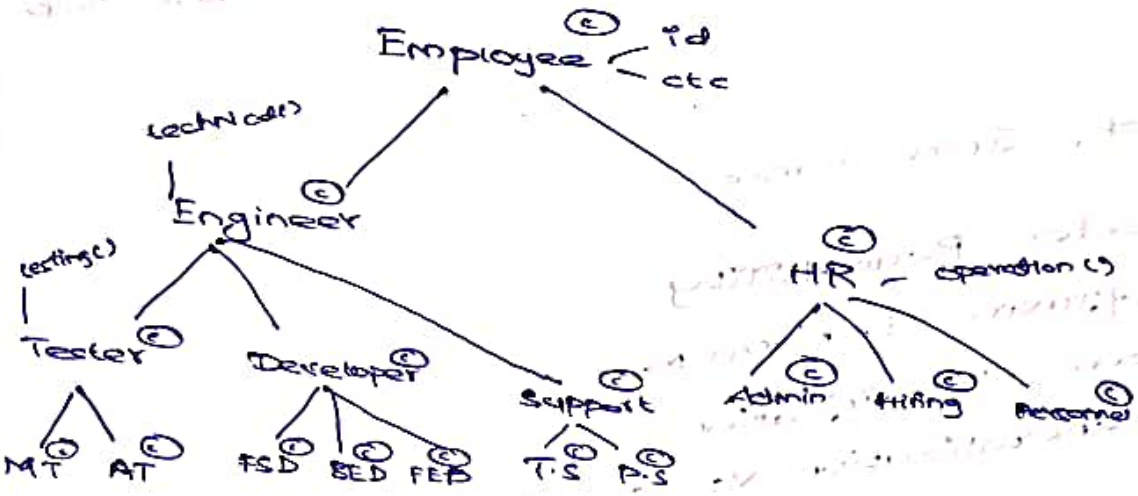
V2
class WA2 extends WA1
{
  void voicecall()
  {
  }
  void videoVidcocall()
  {
  }
}

V3
class WA3 extends WA2
{
  void status()
  {
  }
}

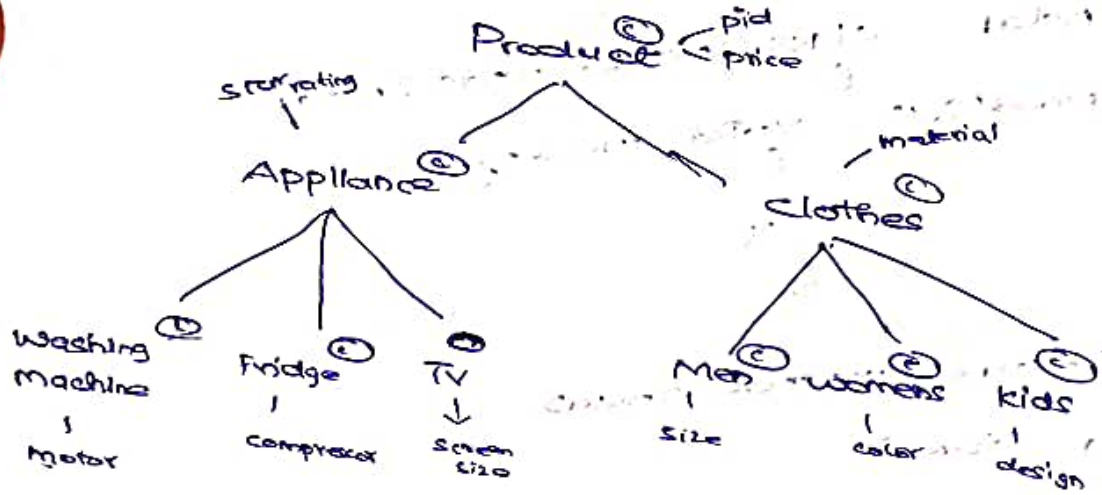
V4
class WA4 extendes WA3
{
  void payment()
  {
  }
}
```

Generalization Vs Specilization Ex 1 :

Hierarchy



Ex 2 :



Given a number n. find all the prime divisors of n.

Types of Inheritance

Single Level Inheritance:

In case of single level Inheritance, the properties of super class is derived by only one sub class and that sub class will have only one super class

Ex:

```
Class A
{
  void test()
  {
    S.o.p("test()...");
  }
}
class B extends A
{
  void disp()
  {
    S.o.p("disp()...");
  }
}
Public class prgm
{
  P.s.v.m(String[] args)
  {
    A a1 = new A();
    a1.test();
    B b1 = new B();
    b1.test();
    b1.disp();
  }
}
```

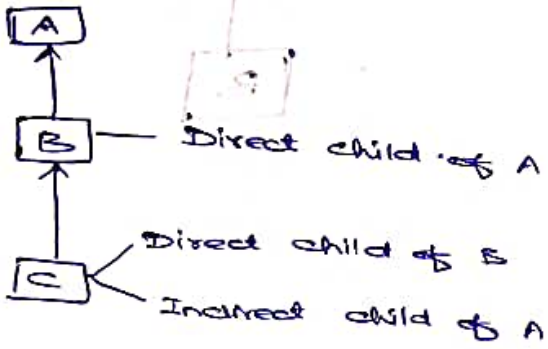


Multi Level Inheritance:

In case of multi level inheritance, the properties of a super class is inherited by more than one subclass in different levels

```
Class A
{
  void test()
  {
    S.O.P ("test()..");
  }
}
class B extends A
{
  void disp()
  {
    S.O.P ("disp()...");
  }
}
class C extends B
{
  void run()
  {
    S.O.P("run()...");
  }
}
```

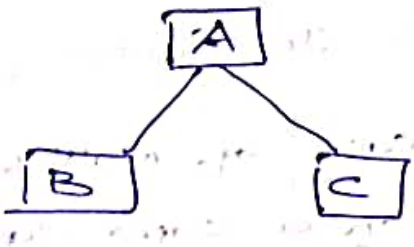
```
class
Public class pgm2
{
  P.S.V.M (String[] args)
  {
    A a1 = new A();
    a1.test();
    B b1 = new B();
    b1.test();
    b1.disp();
    C c1 = new C();
    c1.test();
    c1.disp();
    c1.run();
  }
}
```



Hierarchical Inheritance :

In case of hierarchical inheritance, the properties of a superclass is derived by more than one subclass in the same level.

```
Class A
{
  void test()
  {
    S.O.P ("test()...");
  }
}
class B extends A
{
  void disp()
  {
    S.O.P ('disp()...");
  }
}
```



```
class extends A
{
    void run()
    {
     S.O.P ("run()...");
    }
}
```

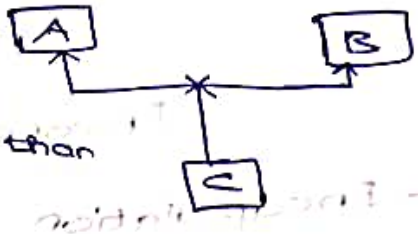
```
A a1 = new A();
a1.test();

B b1 = new B();
b1.test();
b1.disp();

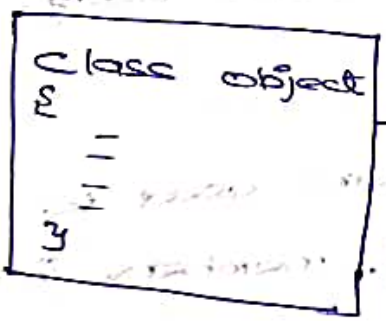
C c1 = new C();
c1.test();
c1.run();
```

Multiple Inheritance

- In case of multiple inheritance, a single sub class tries to directly inherit from more than one super class.



- Multiple inheritance is illegal and not supported in java using classes.
- Multiple inheritance is not supported as it leads to Jacobs Diamond Ambiguity Problem. (i.e) the sub class does not know from which super class it must acquire class object properties.



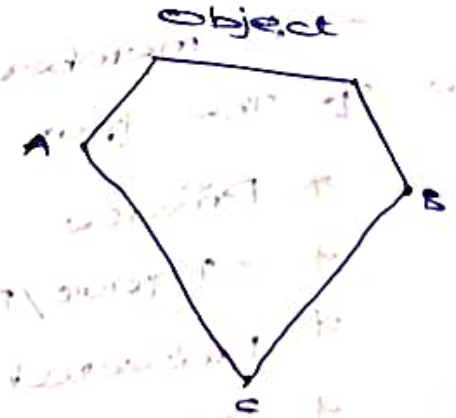

```
Class A
{
}
```

```
Class B
{
}
```

```
Class C extend A, B
{
}
```

CTE

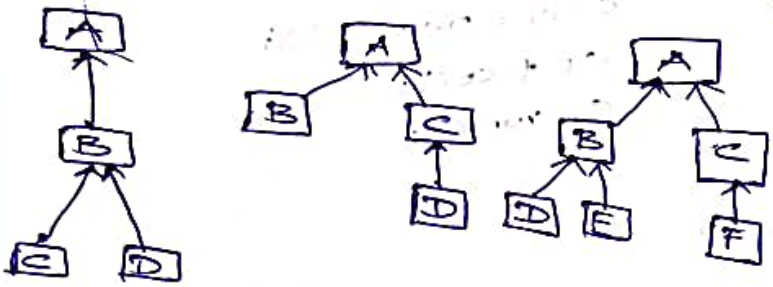
? confused -> Ambiguity



Jacobs Diamond Ambiguity Problem

Hybrid Inheritance:

Hybrid Inheritance is a combination of more than one form of inheritance.



Encapsulation

- Encapsulation is the concept of binding Data members and Member Functions into a class body.

- In Java, every class is an example for encapsulation as java syntax does not allow the programmer to create data members and (or) member functions outside a class body.

Access Modifiers:

- Access Modifiers are used to control the visibility of members inside a class

- A member in java can have one of the four access modifiers.

  * Private
  * <Default/Package>
  * Protected
  * Public

Private : => only that particular class

- Members with private access can only be accessed in that class.

* <Default/Package> => only that particular Package classes

- If a member is created without an access modifier it is considered as <default/Package> access.

- Members with <default/package>, access can only be accessed in that package.

Protected :

Members with protected access can be accessed directly in the same package

Access outside the package is controlled via inheritance

Public : -> Anywhere inside project directly

Members with public access can be directly accessed from anywhere.

Ex :

```
Package. encapsulation;
Public class Allstar
{
    Private static int a=10;
    static int b=20;
    Protected static int c=30;
    Public static int d = 40;

    P.S.V.M (String[] args)
    {
        S.O.P (Allstar.a); //10
        S.O.P (Allstar. b); //20
        S.O.P (Allstar. c); //30
        S.O.P (Allstar. d); //40
    }
}
```

Ex 2 :

```
Package.encapsulation;
Public class DeepDiver
{
  P.S.V.m (String[] args)
  {
    S.O.P (Allstar.b); //20
    S.O.P( Allstar.c); //30
    S.O.P( Allstar.d); //40
  }
}
```

Ex 3 :

```
package.intro;
import.encapsulation.Allstar;
Public class starplayer extends Allstar
{
  P.S.V.m (String[] args)
  {
    S.O.P (Allstar.c); //30
    S.O.P (Allstar.d); //40
  }
}
```

Access Modifiers on a class

A class or an interface can have either public or <default/ package> level access.

Composition - (Has-A Relationship)

- Composition is the concept of one class holding the object of an another class.
- Composition is also known as Has-A Relationship.

Ex 1 :

```
class Demo
{
    void test()
    {
    }
}
```

```
class Sample
{
    Demo ref = new Demo();
}

Public class pgm1
{
    P.S.V.M (String[] args)
    {
        Sample obj = new Sample();
        obj.ref.test();
    }
}
```

Ex. 2 :

```
Class Engine
{
    void power()
    {
    }
}
```

```
class Car
{
    Engine eng = new Engine();
}

Public class pgm2
{
    P.S.V.M (String[] args)
    {
        Car ref = new Car();
        ref.eng.power();
    }
}
```

Ex. 3 :

```
Class Camera
{
    void snap()
    {
    }
}

Class Speaker
{
    void audio()
    {
    }
}

class Microphone
{
    void record()
    {
    }
}
```

```
class Mobile
{
    Camera cam = new Camera();
    Speaker spk = new Speaker();
    Microphone mic = new Microphone();
}

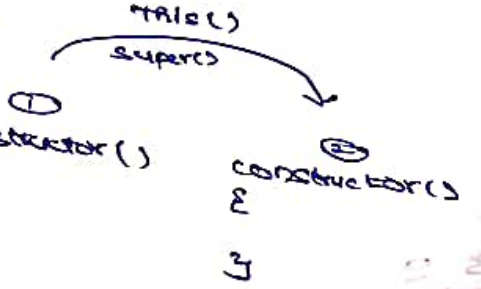
Public class pgm3
{
    P.S.V.M (String[] args)
    {
        Mobile mob = new Mobile();
        mob.cam.snap();
        mob.spk.audio();
        mob.mic.record();
    }
}
```

```
class Printer
{
    void print()
    {
    }
}
```

```
class Computer
{
    static Printer out =
        new Printer();
}
public class prgm4
{
    P.S.V.M(String[] args)
    {
        Computer.out.print();
    }
}
```

Constructor chaining

- Constructor chaining is a concept of one constructor calling an another constructor
- In Java, we can do constructor chaining using
  * this() statement
  * super() statement



this() statement :-

- this() statement is used to call another constructor in a same class.
- We have to write this() statement in the first line of the constructor.
- We can only write one this() statement in the constructor.
- this() statement can only be written by the programmer.

Ex 1 :

```
class Hotel
{
   Hotel()
   {
     S.O.P("Maruthi Bavan");
   }
   Hotel(Int a)
   {
     this();
     S.O.P("Ganga Sweets");
   }
   Hotel(double a)
   {
     this(10);
     S.O.P("A2B");
   }
}
Public class Program1
{
  P.S.V.M(String[] args)
  {
    Hotel h=new Hotel();
  }
}
```

O/P :

```
Maruthi Bavan
Ganga Sweets
A2B
```

Ex 2 :

```
class City
{
   City(Int a)
   {
     this(2.3);
     S.O.P("CSK");
   }
   City(double b)
   {
     this("ABC");
     S.O.P("MI");
   }
   City(String c)
   {
     S.O.P("RCB");
   }
}
```

```
Public class Program2
{
   P.S.V.M(String[], args)
   {
     City c=new City(9);
   }
}
```

O/P

```
RCB
MI
CSK
```

Super() Statement :

- Super() statement is used to call another constructor of the parent class.
- We have to write super() statement in the first line of the constructor.
- We can only write one super() statement in the constructor.
- super() statement can be written by the programmer or by compiler.

Ex. 1 :

```
class Demo
{
    Demo()
    {
      S.O.P("Demo ctr");
    }
}
class Sample extends Demo
{
    Sample()
    {
      super();
      S.O.P("sample ctr");
    }
}
public class Program
{
  P.S.V.m(String[] args)
    {
      Sample ref = new Sample();
    }
}
```

O/P

```
Demo ctr
Sample ctr
```

Ex. 2 :

```
class Alpha
{
    Alpha()
    {
      this(10);
      S.O.P("Red");
    }
```

```
Alpha(int a)
{
  S.O.P("Green");
}
}
class Beta extends Alpha
{
  Beta()
  {
    super();
    S.O.P("Pink");
  }
  Beta(int a)
  {
    this();
    S.O.P("Blue");
  }
}
```

```
Public class Program4
{
  P.S.V.M(String[] args)
  {
    Beta obj = new Beta(6);
  }
}
```

O/P

Green
Red
Pink
Blue

this()

this() statement is used to call constructor of same class

Should always be written by programmer.

super()

super() statement is used to call constructor of super class

can be written both by programmer and compiler

Rhombus or Diamond Prgm

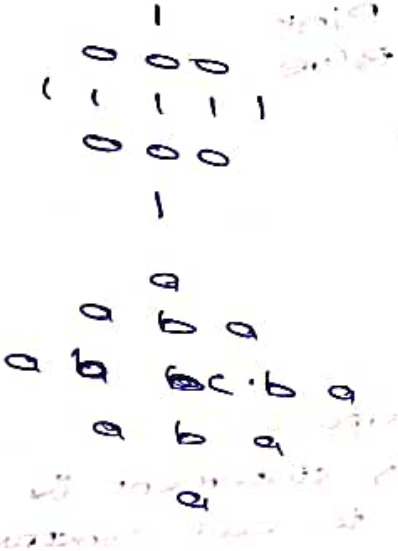
```
int n = 5;
int spc = n/2;
int str = 1;
for(int i=0; i<n; i++)
{
  for(int j=0; j<spc; j++)
  {
    S.O.P(" "+" ");
  }
  for(int k=0; k<str; k++)
  {
    S.O.P("*"+" ");
  }
```

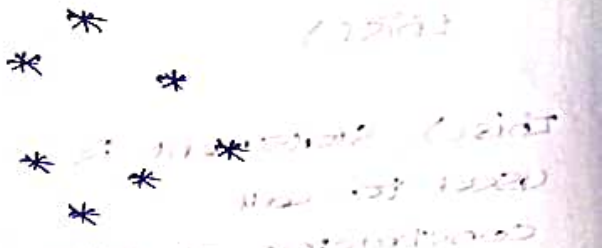
```
S.O.Pln(j);
if(i<n/2)
{
  spc--;
  str=str+2;
}
else
{
  spc++;
  str=str-2;
}
}
```

O/P

spc	str
2	1
1	3
0	5
1	3
2	1

Assignments :





Reverse the sentence

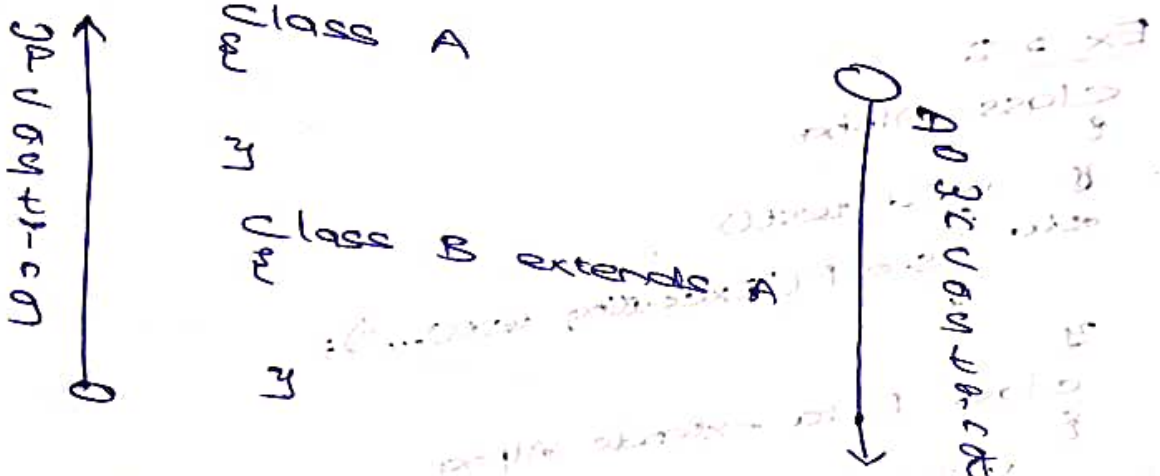
```
String str = "My Name is Rama";
String[] arr = str.split();
str = "";
for(int i=0; i<arr.length; i++)
{
  String s = arr[i];
  for(int j=s.length()-1; j>=0; j--)
  {
    str = str + s.charAt(j);
  }
  str = str + " ";
}
S.O.P(str);
```

O/P

yM emaN si emaR

Typecasting

- Typecasting is the concept of assigning the reference variable of one class to the reference variable of another class.
- To do typecasting, Inheritance is a pre-requisite.
- In Java, Typecasting is classified into two types
  * Upcasting
  * Downcasting



Upcasting :

- Upcasting is the concept of assigning the reference variable or object of sub class to the reference variable of super class.
- After upcasting we can only access the properties of super class.
- Upcasting can be done directly and is supported by the compiler.
- We can directly assign sub class object to super class reference variable

Ex 1.8

```
Class Demo
{
}
class Sample extends Demo
{
}
Public class Program1
{
  P.S.V.M (String[] args)
  {
    Sample s = new Sample();
    Demo d = s; //upcasting : sample -> Demo
    S.O.P (d);                 sub class -> super class
    S.O.P (s);
  }
}
```

Ex 2

```
Class Alpha
{
  void test()
  {
    S.O.P("Executing test()...");
  }
}
class Beta extends Alpha
{
  void disp()
  {
    S.O.P("Executing disp()...");
  }
}
Public class Program2
{
  P.S.V.M (String[] args)
  {
    Beta b = new Beta();
    b.test();
    b.disp();

    Alpha a = b;
    a.test();
    // a.disp(); // CTE
  }
}
```

Ex. 3 :

```
class A
{
  void test()
  {
    S.o.p ("test()");
  }
}
class B extends A
{
  void disp()
  {
    S.o.p("disp()");
  }
}
class C extends B
{
  void play()
  {
    S.o.p(" play()");
  }
}
public class Program3
{
  P.S.V.M (String[] args)
  {
    C c1 = new C();
    c1.test();
    c1. disp();
    c1. play();

    B b1 = c1 ; // upcasting : C -> B
    b1. test();
    b1. disp();

    A a1 = b1 ; //upcasting : B -> A
    a1. test();

    A a2 = c1 ; // upcasting : C -> A
    a2. test();
  }
}
```

Ex. 4 :

```
class Delta
{
  int a=10;
}
class Example extends Delta
{
  int b=20;
}
public class Program3
{
  P.S.V.M (String[] args):
  {
    Delta ref = new Example(); // Upcasting Example -> Delta
    S.O.P (ref.a);
  }
}
```

Downcasting :

- Downcasting is the concept of assigning the reference variable of super class to the reference variable of sub class.

- Downcasting process is explicit and is not supported by the compiler.

From

- For Down casting to work programmer has to
  - * Write casting statement
  - * Upcasting Before statement
- If we perform downcasting without upcasting it will cause ClassCastException

Ex. 1 :

```
class Demo
{
}
class Sample extends Demo
{
}
public class Program1
{
  P.S.V.m (String[] args)
  {
    Demo ref = new Sample();
    Sample obj = (Sample) ref; // Upcasting : Demo -> Sample
    S.o.p (ref);
    S.O.P (obj);
  }
}
```

Ex. 2 :

```
class Alpha
{
  void test()
  {
    S.o.p ("test()");
  }
}
class Beta extends Alpha
{
  void disp()
  {
    S.o.p ("disp()");
  }
}
public class Program2
{
  P.S.V.m (String[] args)
  {
    Alpha a = new Beta();
    a.test();
    Beta b = (Beta)a;
    b.test();
    b.disp();
  }
}
```

Ex: 3 :

```
class A
{
  void test()
  {
    S.o.p("test()");
  }
}
class B extends A
{
  void disp()
  {
    S.o.p("disp()");
  }
}
class C extends B
{
  void play()
  {
    S.o.P("play()");
  }
}
```

```
A a1 = new C(); //upcasting C -B-> A
a1.test();
```

(test() disp() run())

```
B b1 = (B)a1; //Downcasting : A -> B
b1.test();
b1.disp();

C c1 = (C) b1; // Downcasting : B -> C
c1.test();
c1.disp();
c1.play();

C c2 = (C) a1; // Downcasting A -B-> C
c2.test();
c2.disp();
c2.play();
```

```
A a1 = new B(); //Upcasting : B -> A
a1.test();
```

(test() disp())

```
B b1 = (B) a1; // Downcasting : A -> B
b1.test();
b1.disp();

C c1 = (C) b1; // Downcasting : B -> C;
```

↳ ClassCastException.

Ex.4 :

```
class Delta
{
  int a=10;
}
class Example extends Delta
{
  int b=20;
}
public class Program3
{
  P.S.V.M (String[] args)
  {
    Delta ref = new Example();
    S.o.P (ref.a); //10
    Example ex = (Example)ref; //downCasting : Delta -> Example
    S.o.P (ex.a); //10
    S.o.P (ex.b); //20
  }
}
```

```
Class Product
{
}

Class Television extends Product
{
}

Class Refrigerator extends Product
{
}

Class Mobile extends Product
{
}

Class Lab extends Product
{
}

Class Flipkart
{
  P.S.V. addToCart(Product p)
  {
    if (p != null)
    {
      S.o.p("Product Added");
    }
  }
}
```

```
Mobile mob = new Mobile();
Flipkart.addToCart(mob);

Television tv = new Television();
Flipkart.addToCart(tv);
```

Method overloading

- Developing multiple methods in a class with a same name and different arguments is called as method overloading.
- We can vary the arguments in three ways
  - * Type of Arguments
  - * Length of Arguments
  - * Sequence of Arguments
- Method overloading is used to perform same task with different input arguments.

- We can overload static, private, and final methods.

- Method overloading used to achieve compile time / polymorphism.

Ex 1:

```
class Biryani
{
  void eat(int a)
  {
    S.o.p("Mutton Biryani");
  }
  void eat(double d)
  {
    S.o.p("Chicken Biryani");
  }
  void eat(String s)
  {
    S.o.p("Fish Biryani");
  }
}
```

```
Public class Program1
{
  P.S.V.m(String[] args)
  {
    Biryani b = new Biryani();
    b.eat(15);
    b.eat(2.5);
    b.eat("Java");
  }
}
```

O/P:

Mutton Biryani
Chicken Biryani
Fish Biryani

Ex 2:

```
class Game
{
  void play(int a)
  {
    S.o.p("Pubg");
  }
  void play(int a, int b)
  {
    S.o.p("COD");
  }
  void play(int a, int b, int c)
  {
    S.o.p("Free Fire");
  }
}
```

```
Public class Program2
{
  P.S.V.m(String[] args)
  {
    Game g = new Game();
    g.play(10);
    g.play(1,2);
    g.play(1,4,5);
  }
}
```

O/P

Pubg
COD
Free Fire

Ex 3 :

```
class Demo
{
    void test(int a, String s)
    {
        S.O.P ("Testing");
    }
    void test(String s, int a)
    {
        S.O.P("Development");
    }
}

Public class Program3
{
    P.S.V.M (String[] args)
    {
        Demo d=new Demo();
        d.test(25, "ABC");
        d.test("XYZ", 34);
    }
}
```

O/P:

```
Testing
Development
```

Ex-4 :

```
class Flipkart
{
    void payment()
    {
        S.O.P ("COD");
    }
    void payment(long card)
    {
        S.O.P("Card. Payment");
    }
    void payment(String upi)
    {
        S.O.P("UPI");
    }
    void payment (String bank, long mobile)
    {
        S.O.P ("Netbanking");
    }
}
```

Ex. 5 :

```
class Irctc
{
    void search(int train number)
    {
    }
    void search(String src, String dest)
    {
    }
    void search(String pnr)
    {
    }
}
```

```
class Facebook
{
    void login (String un, String pwd)
    {
    }
    void login (long mob, String pwd)
    {
    }
}
```

Method Overriding

Method overriding is the concept of sub class inheriting a method from the super class and changing the method definition while keeping the same declaration.

- To do overriding, inheritance is mandatory
- After overriding and upcasting latest implementation will be given.
- Method overriding is used to upgrade existing features in the software.
- Method overriding is used to achieve runtime Polymorphism.
- We cannot override static, private and final methods.

Ex 1:

```
class Demo
{
    void display()
    {
        S.o.p("Madras");
    }
}
class Sample extends Demo
{
    @Override
    void display()
    {
        S.o.p("Chennai");
    }
}
```

```
Public class Program1
{
    P.S.V.M (String[] args)
    {
        Demo ref = new Demo();
        ref.display();
        Sample obj = new Sample;
        obj.display();
    }
}
```

O/P:

Madras
Chennai

Ex-2 :

```
class Parent
{
   void watch()
   {
      S.o.P("News/Serial");
   }
}
class child extends Parent
{
   @override
   void watch()
   {
      S.o.P("Music/Movies");
   }
}
```

```
Parent p = new Parent();
p.watch();

child c = new child();
c.watch();

Parent ref = new child();
ref.watch();
```

Ex-3 :

```
class FacebookOld
{
   void reaction()
   {
      S.o.P("Like");
   }
}
class FacebookNew extends FacebookOld
{
   @override
   void reaction()
   {
      S.o.P("Like, Wow, Angry, Sad, Love");
   }
}
```

```
FacebookOld fb = new FacebookOld();
fb.reaction();

FacebookNew ref = new FacebookNew();
ref.reaction();

FacebookOld obj = new FacebookNew();
obj.reaction();
```

Ex-4 :

```
class WhatsApp1
{
   void receipt()
   {
      S.o.P("Sent");
   }
}
class WA2 extends WA1
{ @override
   void receipt()
   {
      S.o.P("sent-Delivered");
   }
}
```

```
class WA3 extends WA2
{
   @override
   void receipt()
   {
      S.o.P("Sent & Delivered & Seen");
   }
}
```

Ex-5 :

```
class GmailOld
{
    void composeMail()
    {
        S.o.P("create a new tab in browser");
    }
}
class GmailNew extends GmailOld
{
    @Override
    void composeMail()
    {
        S.o.P("Create PopUp Box");
    }
}
```

Ex - 6

```
class Windows7
{
    void start()
    {
        S.o.P("ListView");
    }
}
class Windows8 extends Windows7
{
    @Override
    void start()
    {
        S.o.P("Tiles View");
    }
}
```

static private

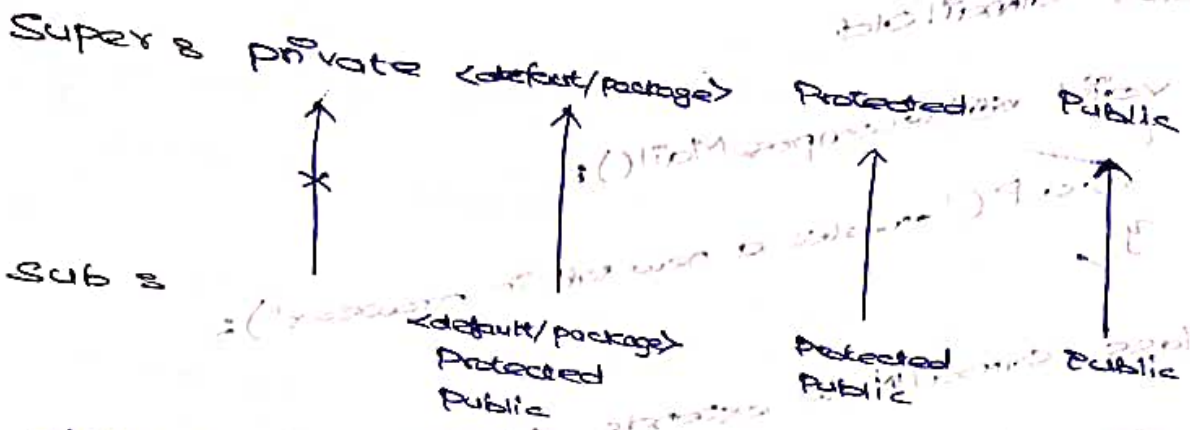
We cannot inherit hence we cannot override.

final → Method

Nobody can change/ modify

final → variable : no-reinitialization
class : no-sub class
method : No-overriding

Access Modifiers:




```
class Delta
{
   int val = 25;
}
class Sample extends Delta
{
   int val = 45;
}
```

```
Delta d = new Delta();
S.o.p(d.val); // 25
Sample s = new Sample();
S.o.p(s.val); // 45
Delta ref = new Sample();
S.o.p(ref.val); // 25
```

This is only method overloading & method overriding not variable overriding.

byte → short → int → long → float → double

char → int

```
int x = 98;
Char ch = (char)x;
S.o.p(ch); // b
```

upcasting.

```
char ch = 'a';
int x = ch;
S.o.p(x); // 97
```

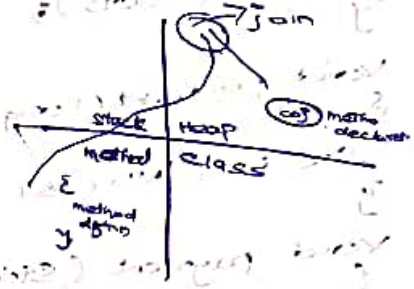
Downcasting

Method Binding :

* It is the concept of joining method declaration and method definition loading to STACK area.

* Method binding is done by JVM @ runtime.

* Method binding decision can be taken either by i) compiler @ compile time
ii) JVM @ Run time



Polymorphism

- Polymorphism is the concept of one object showing different behaviours in its life time

- In Java, polymorphism is of 2 types

* Compile time polymorphism
* Run time polymorphism

Compile time Polymorphism :

- This form of polymorphism using method overloading.

- In this scenario method binding decisions are done by the compiler at compiling ~~compiler~~ time.

- Compiler take decision based on the arguments

- Once compiler takes a decision it cannot be changed hence it is also known as static polymorphism (or) static binding.

- As binding decision was taken at compile time before execution it is also known as early binding.

Ex-1 :

```
Class Flipkart()
{
  void payment()
  {
    S.o.p("COD");
  }
  void payment(Long card)
  {
    S.o.p("No cost EMI");
  }
  void payment(String UPI)
  {
    S.o.p("Cashback");
  }
  void payment(String bank, Long mobile)
  {
    S.o.p("Discount");
  }
}
```

```
Flipkart f = new flipkart();
f.payment();
f.payment(1234321123443211);
f.payment("Gpay");
f.payment("ICICI", 87609912485);
```

Ex-2 :

```
class Irctc
{
  void search(int trainno)
  {
    S.o.p("Schedule");
  }
  void search(String src, String des)
  {
    S.o.p("List");
  }
  void search(String PNR)
  {
    S.o.p("Ticket Info");
  }
}
```

```
Irctc ref = new Irctc();
ref.search(12001);
ref.search("KSR", "MAS");
ref.search("BLRCH1234");
```

Ex-3 :

```
class TestScript
{
  void test(int a)
  {
    S.o.p("Sanitary Testing");
  }
  void test(double d)
  {
    S.o.p("IT");
  }
  void test(String s)
  {
    S.o.p("Regression Tests");
  }
};
```

```
TestScript t = new TestScript();
t.test(15);
t.test(3.45);
t.test("XYZ");
```

Runtime Polymorphism :

- This form of polymorphism is achieved using Method overriding.
- In this scenario ~~method binding~~ decisions are done by the JVM.
- JVM takes decision based on the current instance.
- Once JVM takes a decision it can be changed hence it is also known as Dynamic ~~plot~~ polymorphism or dynamic binding
- As binding decision was taken at run time during execution it is also known as late binding

Ex - 1 :

```
class Card
{
    void swipe()
    {
     S.o.p ("Please wait");
    }
}
class CreditCard extends Card
{
    @override
    void swipe()
    {
     S.o.p("Due Increases");
    }
}
class DebitCard extends Card
{
    @override
    void swipe()
    {
     S.o.p("Balance Reduces");
    }
}
```

```
Card ref;
ref = new CreditCard();
ref.swipe();
ref = new DebitCard();
ref.swipe();
```

Ex - 2 :

```
class Facebook
{
   void response()
   {
    S.o.p("Loading");
   }
}
class FacebookMobile extends Facebook
{
   @override
   void response()
   {
    S.o.p("Response in App View");
   }
}
class FacebookWebsite extends Facebook
{
   @override
   void response()
   {
    S.o.p("Response in Browser View");
   }
}
```

```
Facebook fb;
fb = new FacebookMobile();
fb.response();
fb = new FacebookWebsite();
fb.response();
```

Diff btwn compile time & run time polymorphism:

Parameter	Compile time Polymorphism	Run time Polymorphism
Achieved by	Method overloading	Method overriding
Binding Decision by	Compiler @ Compile Time	JVM @ Run time
Decision based on	Arguments	Current Instance
Nature of Binding	Static	Dynamic
Binding	Early	Late

Diff btwn Static & Dynamic Binding

Parameter	Static Binding	Dynamic Binding
Decisions taken by	Compiler	JVM
Decisions can change	NO	Yes

Ex - 5 :

```
class Adapter
{
    void display()
    {
    }
}
```

```
Class TVAdapter extends Adapter
{
    @override
    void display()
    {
      S.O.P("Display in TV");
    }
}
Class ProjectorAdapter extends Adapter
{
    @override
    void display()
    {
      S.O.P("Display in Projector");
    }
}
Class computer
{
    static void view(Adapter ap)
    {
      ap.display();
    }
}
TV Adapter tv = new TVAdapter();
ProjectorAdapter pa = new ProjectorAdapter();
computer.view(pa);
computer.view(tv);
```

Palindrome in String

```
String str = "MADAM";
boolean status = true;
for(int i=0; i<str.length()/2; i++)
{
    if(str.charAt(i) != str.charAt(str.length()-(i+1)))
    {
      status = false;
      break;
    }
}
if(status == true)
    S.O.P("Palindrome");
else
    S.O.P("Not a Palindrome");
```

```
String str = "MADAM";
String rev = "";
for(int i=str.length()-1; i>=0; i--)
{
  rev = rev + str.charAt(i);
}
if(str.equals(rev))
  S.o.p("Palindrome");
else
  S.o.p("Not a Palindrome");
```

Abstraction

* Abstraction is the concept of hiding implementation details and exposing features.
* Abstraction in Java is the concept of hiding method definition and exposing method declaration.

In Java, we can achieve abstraction using abstract methods.

Abstract Methods

- An abstract method contains only Method Declaration.
- It is created using "abstract" keyword.
- An abstract method can only be created inside an abstract class or an interface.
- Abstract methods cannot be "static", private and final.

Abstract class

- An abstract class is created using the keyword "abstract".

Syntax:

```
abstract class classname
{
}
```

- Abstract class does not define any restrictions on its members.
- We cannot create an object for an abstract class.
- However, we can access properties of an abstract class through inheritance and overriding.

Ex-1:

```
abstract class Demo
{
  abstract void test();
  void disp()
  {
    S.o.P("Executing");
  }
}
class Sample extends Demo
{
    @override
    void test()
    {
      S.o.P ("Executing test");
    }
}
Public class Pgm1
{
  P.S.V.m (String[] args)
  {
     Demo ref = new Sample();
     ref.test();
     ref.disp();
  }
}
```

Interface

Interface created using keyword "interface".

Syntax:

```
Interface interfacename
{
    methods          &    variables
      ↓                     ↓
    public                public
      &                     &
    abstract              static
                            &
                          final
}
```

- All data members in an interface are public, static, final.
- All member functions in an interface are public and abstract.
- We cannot directly create an object for an interface.
- We can access properties of an interface through inheritance and overriding.
- We can inherit from an interface using the keyword "implements".
- Using interface, we can do multiple inheritance.

Ex - 2:

```
abstract class Alpha
{
  abstract void test();
  abstract void disp();
}

class Beta extends Alpha
{
    @override
    void test()
    {
     S.o.P ("test()");
    }
    @override
    void disp()
    {
     S.o.P ("disp()");
    }
}
```

```
Public class Pgm2
{
 P.S.V.M (String[] args)
 {
  Beta
  Alpha a = new Beta();
  a.test();
  a.disp();
 }
}
```

Ex-3 :

```
abstract class Delta
{
  void test()
  {
    S.o.p("Executing test");
  }
  void disp()
  {
    S.o.p("Execute disp");
  }
}
class Example extends Delta
{
}
```

```
public class Program3
{
  P.S.V.m (String[] args)
  {
    Delta ref = new Example();
    ref.test();
    ref.disp();
  }
}
```

Ex-4 :

```
abstract class Hotstar
{
  void login()
  {
  }
  abstract void watch();
}
class HotstarFree extends Hotstar
{
  @override
  void watch()
  {
    S.o.p("Limited Time, Ads");
  }
}
class HotstarVip extends Hotstar
{
  @override
  void watch()
  {
    S.o.p("Regional Content, IPL, Ads");
  }
}
class HotstarPremium extends Hotstar
{
  @override
  void watch()
  {
    S.o.p("Full Access, No Ads");
  }
}
```

```
Hotstar ref;
ref = new HotstarFree();
ref.login();
ref.watch();
```

```
ref = new HotstarVip();
ref.login();
ref.watch();
ref = new HotstarPremium();
ref.login();
ref.watch();
```

Ex-5 :

```
abstract class Android
{
    void call()
    {
    }
    void message()
    {
    }
    abstract void ui();
}
class @Oxygenos extends
                    Android
{
    @Override
    void ui()
    {
     S.O.P("Welcome to one+");
    }
}
class Miui extends Android
{
    @Override
    void ui()
    {
     S.O.P ("Welcome to Redmi");
    }
}
class FunTouchOs extends Andriod
{
    @Override
    void ui()
    {
     S.O.P("Welcome to VIVO");
    }
}

Android obj = new OxygenOs();
obj.call();
obj.message();
obj.ui();

obj = new Miui();
obj.call();
obj.message();
obj.ui();

obj = new FunTouchOs();
obj.call();
obj.message();
obj.ui();
```

Ex-6 :

```
abstract class MasterScript
{
    void execute()
    {
    }
    void generateReports()
    {
    }
    abstract void testScenarioReader();
}
class TestScript1 extends MasterScript
{
    @Override
    void testScenarioReader()
    {
    }
}
class TestScript2 extends MasterScript
{
    @Override
    void testScenarioReader()
    {
    }
}
```

```
class TestEngine
{
    P.S.V. runTests(MasterScript obj)
    {
        obj.execute();
        obj.generateReports();
    }
};
```

```
TestScript TS1 = new TestScript();
TestEngine.runTests(TS1);
TestScript TS2 = new TestScript();
TestEngine.runTests(TS2);
TestScript TS3 = new TestScript();
TestEngine.runTests(TS3);
```

Why Abstract class is always considered as partial abstract body ?

(1) An abstract class will contain both abstract & concrete methods

ex:

```
abstract class Demo
{
    abstract void test();
    void disp()
    {
    }
}
```

(2) Abstract class will contain concrete methods inherited from class object.

ex:

```
abstract class Example
{
    abstract void run();
    abstract void play();
}
```

Class object
{
// concrete methods
}

(3) We can achieve 100 percent abstraction using Interface.

```
interface interfacename
{
    body;
}
```

Ex-1 :

```
Interface Demo
{
    void test();
}
class sample implements Demo
{
    @Override
    Public void test()
    {
        S.O.P("test()..");
    }
}
```

```
Public class Prgm1
{
    P.S.V.M(String[] args)
    {
        Demo ref = new Sample();
        ref.test();
    }
}
```

Ex-2 :

```
interface Delta
{
    void play();
    void run();
}
Class Example implements Delta
{
    @Override
    Public void play()
    {
        S.O.P("Execute play()...");
    }
    @Override
    Public void run()
    {
        S.O.P("Execute run()...");
    }
}
```

```
Public class Prgm2
{
    P.S.V.M(String[] args)
    {
        Delta d = new Example();
        d.play();
        d.run();
    }
}
```

① Does java support Multiple Inheritance

- No, with respect to class
- Yes, with respect to Interface.

Ex - 3 :

```
interface Alpha
{
  void teach();
}
interface Beta
{
  void beat();
}
class Gamma implements Alpha, Beta
{
  @override
  Public void beat()
  {
    S.o.p ("Execute beat()");
  }
  @override
  Public void teach()
  {
    S.o.p ("Execute teach()");
  }
}
```

```
Public class Programs
{
  P.S.V.M (String[] args)
  {
    Gamma ref = new Gamma();
    ref.teach();
    ref.beat();
  }
}
```

Ex - 4 :

```
interface Animal
{
  void noise();
}
class Dog implements Animal
{
  @override
  Public void noise()
  {
    S.o.p("Barking");
  }
}
class Cat implements Animal
{
  @override
  Public void noise()
  {
    S.o.p ("Meow");
  }
}
class Lion implements Animal
{
  @override
  Public void noise()
  {
    S.o.p("Roars");
  }
}
class Cow implements Animal
{
  @override
  Public void noise()
  {
    S.o.p ("Maa Maa");
  }
}
class AnimalSimulator
{
  P.S. Animal create (String s)
  {
    if(s.equalsIgnoreCase("Dog"))
      return new Dog();
    if(s.equalsIgnoreCase("Cat"))
      return new Cat();
    if(s.equalsIgnoreCase("Lion"))
      return new Lion();
    if(s.equalsIgnoreCase("Cow"))
      return new Cow();
    else
      return null;
  }
```

```
p. s. v. makeNoise (Animal a)
{
  if (a != null)
  a.noise();
  }
}
Public class Prgm4
{
p.s.v.m (String[] args)
{
  Scanner scn = new Scanner (System.in);
  S.o.p ("Enter Animal");
  String name = scn.next();
  Animal a = AnimalSimulator.create(name);
  AnimalSimulator.makeNoise(a);
  }
}
```

Ex-5 :

```
interface Shape
{
  void draw();
}
class circle implements Shape
{
  @Override
  Public void draw()
  {
  }
}
class Square implements Shape
{
  @Override
  Public void draw()
  {
  }
}
class Triangle implements Shape
{
  @Override
  Public void draw()
  {
  }
}

class ShapeTool
{
  P.S Shape createShape (String s)
  {
    if (s.equalsIgnoreCase ("circle"))
      return new Circle();
    else if (s.equalsIgnoreCase ("square"))
      return new Square();
    else if (s.equalsIgnoreCase ("Triangle"))
      return new Triangle();
    else
      return null;
  }
  public static void displayShape
                    (Shape s)
  {
    if (s != null)
      s.draw();
  }
}

Public class Prgm5
{
  P.S.V.m (String[] args)
  {
  Scanner scn = new Scanner
                    (System.in);
  S.o.p ("Enter Shape");
  String name = scn.next();
  Shape s = ShapeTool.
                createShape(name);
  ShapeTool.displayShape(s);
  }
}
```

Diff btwn

Abstract class	Interface
Abstract	Interface
Partial Abstract body	100% Abstract Body.
No restrictions on data members	All data members are public, static & final
No restrictions on member functions	All member functions are public and abstract
extends	implements
does not support multiple inheritance	supports multiple inheritance

```
interface Demo
{
    void test();
}

class Example implements Demo, Sample
{
    @Override
    public void test()
    {
        ...
    }
}
```

```
interface Sample
{
    void test();
}
```

```
Class Demo
{
    public void run()
    {
        ...
    }
}

class Example extends Demo implements Sample
{

}
```

```
interface Sample
{
    void run();
}
```

Note:

From version 1.8, interface allows
- Default Methods
- Static Methods

This provisional was made to support Functional programming.

Main Method Syntax Explain:

public static void main (String args)

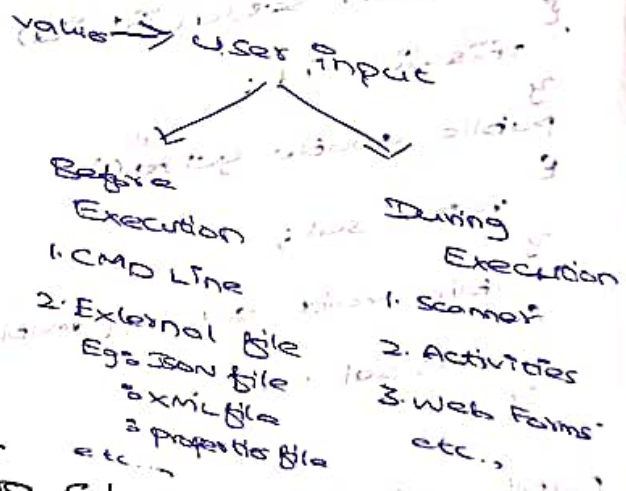
public - Universal Access

static - So that JVM can access using classname
To avoid unnecessary object creation

void - no calling method; directly called by JVM

main - Naming out of convention following basics of C/C++.

String[] args - Read CMD Line variable



Java Bean class / Pojo class:
[Plain old Java object].

- Java Bean class is used to 100% encapsulation.

- Objects of Java Bean Class are used to transfer data btwn prgms in an application.

Rules for creating Java Bean class :

- All data members must be private
- Access to private data members via Public getter() and setter() methods.
- No Methods except the getters() and setters().
- Class must have only default constructor.
- class must Implement Serializable Interface.

Ex-1 :

```
import java.io.Serializable;
Public class EmployeeBean implements Serializable
{
  private int id;
  private double sal;
  public int getId()
  {
    return id;
  }
  public void setId(int id)
  {
    this.id = id;
  }
  public double getSal()
  {
    return sal;
  }
  Public void setSal(double sal)
  {
    this.sal = sal;
  }
}
Public class Mainclass
{
  P.S.V.M (String[] args)
  {
    EmployBean eb = new EmployeeBean();
    eb.setId(101);
    eb.setSal(4.5);
    S.O.P ("ID:" + eb.getId());
    S.O.P ("Sal :" + eb.getSal());
  }
}
```

Ex-2 :

```
import java.io.Serializable;
Public class AccountBean Implements Serializable
{
  private String username;
  private String password;
  Public String getUsername()
  {
    return username;
  }
  Public void setUsername(String username)
  {
    this.username = username;
  }
  Public String getPassword()
  {
    return password;
  }
  Public void setPassword(String password)
  {
    this.password = password;
  }
}
```

```
Import java.util.Scanner;
Public class Gmail
{
  P.S.V.M (String[] args)
  {
    Scanner scn = new Scanner(System.in);
    S.O.P("Enter Username");
    String username = scn.next();
    S.O.P("Enter Password")
    String password = scn.next();
    AccountBean ab = new AccountBean();
    ab.setUsername(username);
    ab.setPassword(password);
    GoogleAccountValidator.verifyAccount(ab);
  }
}
```

```
Public class GoogleAccountValidator
{
  P.S.V.. verifyAccount(AccountBean ab)
  {
    if(ab.getUsername().equals("Rama") && ab.getPassword().
                                    equals("Rama4STRA"));
      S.O.P("Loggin success");
    else
      S.O.P("Invalid credentials");
  }
}
```

Java Bean class objects are called as Data Transfer objects (DTO).

Singleton class :

Any Java class for which there can exists only one object in memory is called as a Singleton class

Ex - 18

```
Public class Connection
{
  Private static Connection con = null;
  Public static Connection getConnection()
  {
    if (con == null)
    {
      con = new Connection();
    }
    return con;
  }
  Private Connection()
  {
    S.O.P ("Creating Connection Object");
  }
}

Public class SigletonDriver
{
  P.S.V.M (String[] args)
  {
    Connection con1 = Connection.getConnection();
    Connection con2 = Connection.getConnection();
    Connection con3 = Connection.getConnection();
    S.O.P (con1);
    S.O.P (con2);
    S.O.P (con3);
  }
}
```

Module - 5

Libraries

- A library is a collection of readymade or predefined programs.
- The main purpose of a library is to save time and efforts of the programmers.
- A library is generally organized into several packages
- Each package in a library contains a subset of programs with similar functionality.

Jar File :

- Jar stands for "Java Archive File".
- All jar files will have .jar file extension
- Jar files are used to transport java pgms or applications from one location to an another.
- jar files are safe secure and reliable mechanism for the transport of java pgms

Class object :

It is an inbuilt java library class.

The object class is present in the "java.lang" Package.

It is the super most class in java and every class is either directly or indirectly a child of object class.

It defines some of the most common methods needed by all Java programs.

Methods of object class :

toString()	notify()
hashCode()	notifyAll()
equals()	finalize()
wait()	clone()
wait(long a)	getClass()
wait(long a, int b)	

Can't override

toString Method

Method Declaration :

public String toString()

* The toString() method provides the String representation of the object.

String Representation :

FullyQualifiedClassname + @ + HexadecimalPseudoAddress

* It is possible for the programmer to override the toString() method and change the implementation.

Ex :

```
class Child1
{
}
class Child2
{
  @Override
  public String toString()
  {
    return "Kedi Payan";
  }
}
public class Prgm2
{
  P.S.V.M(String[] args)
  {
    Child1 c1 = new Child1();
    Child2 c2 = new Child2();
    S.O.P(c1.toString());
    S.O.P(c2.toString());
  }
}
```

O/P

```
com.qspiders.classobjectExample.Child1@15db9742
Kedi Payan
```

hashCode() :

Method Declaration : Public int hashCode()

- The hashCode() Method provides the unique integer in of the object assigned by the JVM.
- this integer IN is the decimal representation of the hexadecimal Pseudo address.
- It is possible for the programmer to override the hashcode() method and change the implementation.

Ex :

```
class child1
{
}
class child2
{
  @override
  Public int hashcode()
  {
    return 420;
  }
}
```

```
Public class Prgm2
{
  P.S.V.M (String[] args)
  {
    child1 c1=new child1();
    child2 c2= new child2();
    S.O.P(c1.hashcode());
    S.O.P(c2.hashcode());
  }
}
```

equals() Method :

Method declaration : public boolean equals(object obj)

- The equals() Method is used to compare the content btwn 2 objects.
- We will have to override this method to perform equality comparison as by default it works the same as "==" operator

Ex :

```
class Employee
{
  private int empId = 101;
  @override
  public boolean equals(object obj)
  {
```

```
    Employee emp = (Employee) obj;
    if (this.empid == emp.empid)
    return true;
    else
    return false;
  }
}
Public class Prgm2
{
  P.S.V.m ()
  {
    Employee e1 = new Employee();
    Employee e2 = new Employee();
    S.O.P (e1.equals(e2));
  }
}
```

O/P:
True

```
                    ↳ (object obj)
```

Ex-2:

```
Class product
{
  int pid = 437;
  double price = 15000;
  @Override
  Public boolean equals(object obj)
  {
    Product p2 = (Product) obj;
    if (this.pid == p.pid && this.price == p.price)
    return true;
    else
    return false;
  }
}
Public class Program1
{
  P.S.V.m ()
  {
    Product p1 = new product();
    product p2 = new product();
    S.O.P (p1.equals(p2));
  }
}
```

Ex-3 =

```
class Square
{
  private int side = 5;
  @Override
  Public boolean equals(Object obj)
  {
    Square s = (Square) obj;
    if (this.side == s.side)
    return true;
    else
    return false;
  }
}

Public class Pgm5
{
  P.S.V.M()
  {
    Square s1 = new. Square();
    Square s2 = new Square();
    S.O.P (s1. equals (s2));
  }
}
```

wait(), wait(long a), wait(long a, int b)

- wait() method is overloaded in 3 versions in object class

Method Declarations =

1. Public final void wait()
2. Public final void wait (long a)
3. public final void wait (long a, int b)

- wait() method is used internally by the JVM to pause program execution till the occurrence of an event or for an certain time interval.

notify() Method:

Method Declaration: public final void notify()

The notify() method is used to notify a particular thread about the occurance of an event.

notifyAll() Method:

Method Declaration: public final void notifyAll()

The notifyAll() method is used to notify all the threads about the occurance of an event.

finalize() Method:

Method Declaration: protected void finalize()

Before destroying an object and removing it from the heap area the Garbage collector executes the finalize() method.

The finalize() method contains clean up or closure code that has to be executed before an object is destroyed.

Garbage Collector (GC)

- System of JVM
- Managing memory in Heap Area
- Identifies & removes unwanted objects from Heap Area.
- Before destroying and removing an object from memory Garbage Collection will execute "finalize()" method.

Ex :

```
class Connection
{
  @Override
  protected void finalize() throws Throwable
  {
    S.O.P("Connection Object Destroyed");
  }
}
public class prgm6
{
  P.S.V.M (String[] args)
  {
    Connection con = new Connection();
    System.gc();
    con = null;
    System.gc();
  }
}
```

O/P
Connection Object Destroyed

clone() Method :

Method Declaration : protected object clone()

- The clone() Method is used to create an exact replica of the current object.
- Clone is restricted proces and has to be controlled.

1. Cloning works only one objects of the type cloneable.
2. It is mandatory to perform downcasting on all. cloned copies.
3. Cloning is risky and may cause "CloneNotSupportedException".

Ex1:

```
Public class circle Implements Cloneable
{
    int radius;
    P.S.V.M (String[] args) throws CloneNotSupported
    {                                    - Exception
        Circle c = new circle();
        C.radius = 25;
        Circle rep = (circle) c.clone();
        S.O.P (rep.radius);
    }
}
```

Ex-2:

```
Public class Account Implement Cloenable
{
    String username;
    String password;
    P.S.V.M (String[] args) throws CloneNotSupported
    {                                    Exception
        Account laptop = new Account();
        laptop.username = "Jeva";
        laptop.password = "Jeva123";
        Account mobile = (Account) laptop.clone();
        S.O.P (mobile.username);
        S.O.P (mobile.password);
    }
}
```

Getclass() method :

Method Declaration : public final class getclass()

The getclass() method returns the fully qualified class name of the current instance.

Ex:

```
import java.util.Scanner;
class Demo
{
}
class Sample extends Demo
{
}
public class Prgm1
{
  P.S.V.M (String[] args)
  {
    Object o1 = new Demo();
    Object o2 = new Sample();
    Object o3 = new Scanner (System.in);
    Object o4 = new Object();
    S.O.P(o1.getClass());
    S.O.P(o2.getClass());
    S.O.P(o3.getClass());
    S.O.P(o4.getClass());
  }
}
```

O/P:

class.com.qspiders.ClassObjectExample.Demo

" " " " .Sample

class java.util.Scanner

class java.lang.Object

Questions:

1. Diff btwn == and equals()
2. Explain the use of finalize()?
3. What is the role of Garbage Collector?
4. What is class object
5. What is the use of clone() method?
6. What all methods are present in class Object?

Exception Handling.

Parameter	Compile Time Error	Exception	Runtime Error
Reason	Syntax Level Issue	Logical issues	System Level Issue
Time of Issue	Compile Time	Run Time	Run Time
How to overcome	Code Rewriting	Exception Handling	Not a Programmer issue
Ex:	; expected Cannot find symbol etc.	Arithmetic Exception NullPointer Exception ClassCast Exception	VmOutOfMemory IOError LinkageError

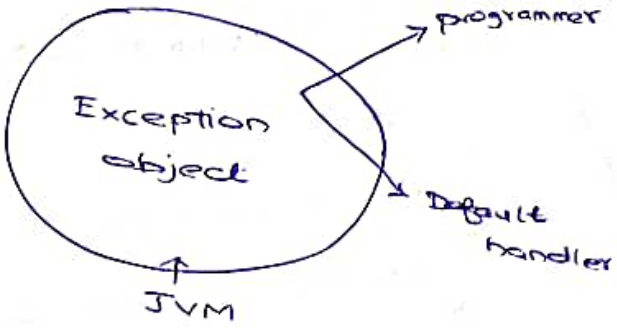
Exception:

* An exception is an unexpected event that occurs at runtime and causes the program to terminate abnormally
* Exception are classified into 2 categories
  - Checked Exception
  - Unchecked Exception

Checked Expression	Unchecked Expression
Detected by compiler @ Compile Time	Undetected by compiler @ compile time
Must be handled	Not mandatory to handle
occurs @ runtime	occurs @ runtime
Eg: CloneNotSupportedException IOException	Eg: ArithmeticException NullPointerException

What happens when Exception occurs

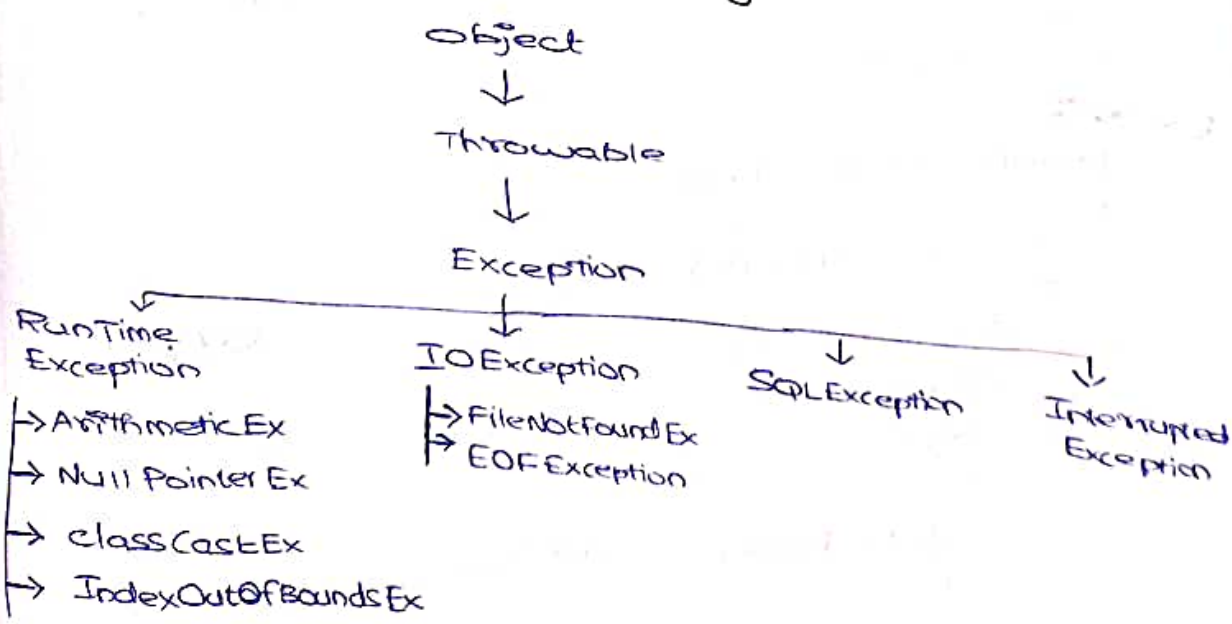
```
Class Prgm
{
    .....
    ..... → Risky code
    .....
}
```



When the risky code executes it may cause an Exception and where the Exception occurs, the JVM creates an object to represent the Exception.

- An object is created by the JVM to represent the Exception.
- There is an inbuilt library class to represent each and every Exception in Java.
- The JVM then looks at the programmer to handle the Exception.
- If the programmer fails to handle the Exception JVM calls the default handler

Exception Hierarchy



Ex-1:

```
Public class Program1
{
  P.S.V.M (String[] args)
  {
    S.O.P(" Program starts");
    try
    {
      S.O.P(10/0);
    }
    catch (ArithmeticException e)
    {
      S.O.P(" / by zero");
    }
    S.O.P ("Program ends");
  }
```

O/P:

```
Prgm starts
/ by zero
Prgm ends
```

Ex-2:

```
Public class Circle
{
  int radius;
  P.S.V.M (String[] args)
  {
    Circle c=new Circle();
    c.radius =25;
    try
    {
      Circle rep = (Circle) c.clone();
      S.O.P (rep.radius);
    }
    catch(CloneNotSupportedException e)
    {
      S.O.P("Object is not Cloneable");
    }
  }
}
```

O/P:

```
Object is not Cloneable
```

Ex-3:

```
Public class Prgm2
{
  P.S.V.M (String[] args)
  {
    S.O.P ("prgm starts");
    String str = "Java";
    try
    {
      S.O.P(Str.charAt(20));
    }
```

O/P:

```
Invalid Index
Accessed
```

```
        catch (NullPointerException e)
        {
          S.O.P("Error Operating with Null Reference");
        }
        catch (StringIndexOutOfBoundsException e)
        {
          S.O.P("Invalid Index Accessed");
        }
        S.O.P ("Prgm Ends");
      }
    }
```

Ex-12

```
class Delta
{
  void test()
  {
    S.O.P("Executing test()...");
  }
}
Public class Program3
{
  P.S.V.M (String[] args)
  {
    try
    {
      Delta d = (Delta) Class.forName("com.qspiders.
                exceptionHandling.Delta").newInstance();
      d.test();
    }
    catch(ClassNotFoundException e)
    {
      S.O.P("No such class exists");
    }
    catch(IllegalAccessException e)
    {
      S.O.P("Access Denied");
    }
    catch(InstantiationException e)
    {
      S.O.P("Failed to create object");
    }
  }
}
```

Ex-5 :

```
Public class Prgm 4
{
  P.S.V.M (String[] args)
  {
    try
    {
      S.o.p ("starts");
      S.o.p (10/0);
      S.o.p ("ends");
    }
    catch (ArithmeticException e)
    {
      S.o.p ("/ by zero");
    }
    finally
    {
      S.o.p ("Executing finally bro");
    }
  }
```

O/P :

```
Starts
/ by zero
Executing finally
block
```

Ex-6 :

```
import java.util.InputMismatchException;
import java.util.Scanner;
Public class Programs
{
  P.S.V.M (String[] args)
  {
    Scanner scn = new Scanner (System.in);
    S.o.P("Enter Value");
    try
    {
      int val = scn.nextInt();
      S.o.P ("value : "+val);
    }
    catch (InputMismatchException e)
    {
      S.o.P("Invalid Input value");
    }
    finally
    {
      scn.close();
      S.o.P("Scannex closed");
    }
  }
}
```

O/P

```
Enter Value
15
Scanner closed
```

Try Block

- The Java instructions in our program that are risky is written in the try block
- Every try block is accompanied by at least one catch block.
- The control travels from try to catch block as soon as there is an Exception.

Catch Block

- The catch block in Java is written after the try block.
- The catch block executes only when there is an Exception.
- The catch block takes the Exception object created by JVM as an argument.
- We can write multiple catch blocks after a single try block.

Finally Block

- Finally is a special block used in Exception Handling.
- It is written after try and catch block.
- Code inside finally block will execute irrespective of the Exception.
- Finally block contains code that has to mandatorily execute.

Default Handler

* The JVM calls the default handler when the programmer fails to handle an exception.

* The Default handler performs the following actions.

  - Stops Program Execution
  - Performs Stack trace
  - Displays Details
    - x Name of the Exception.
    - x Message (if available)
    - x Stack trace info

Throws keyword

- Throws keyword is used to delegate the task of Exception handling to the calling method.
- throws keyword used in the method declaration.
- We can delegate multiple exceptions using the throws keyword.

Note: Throws keyword does not handle the exception it forword its responsibility to the calling method.

Ex-1:

```
public class Circle
{
   int radius;
   static void copy() throws CloneNotSupportedExcep
   {
      Circle c = new Circle();
      c.radius = 25;
      Circle rep = (Circle)c.clone();
      S.O.P(rep.radius);
   }
   P.S.V.M(String[] args) throws CloneNotSupported
   {                                              Excepti
      Circle.copy();
   }
}
```

Ex-2 :

```
class Delta
{
    void test()
    {
        S.o.p("Hai");
    }
}
public class Program3
{
    static void createInstance() throws ClassNotFoundEx,
                                        IllegalAccessEx, InstantiationEx
    {
        Delta d=(Delta)Class.forName("com.qspiders.exceptionHandling.
                                      Delta").newInstance();
        d.test();
    }
    P.S.V.m(String[] args) throws ClassNotFoundException,
                                  IllegalAccessEx, InstantiationException
    {
        createInstance();
    }
}
```

Creating Custom Exceptions

- Create a class that extends Exception
- Develop Trigger Logic for Exception Event
- Handle the Exception

Ex-1 :

```
class UpmaException extends Exception
{
}
public class Program7
{
    static void trigger(int num) throws UpmaException
    {
        if(num%2==0 && num%3==0)
            throw new UpmaException();
        else
            S.O.P("Not Upma");
    }
    P.S.V.m(String[] args)
    {
        try
        {
            trigger(25);
            trigger(18);
        }
        catch(UpmaException e)
        {
            S.O.P("Upma Preparation Detected");
        }
    }
}
```

O/P

```
Not Upma
Upma Preparation
Detected.
```

Ex-28

```
class InvalidNumberException extends Exception
{
}
public class Program8
{
  static void verifyNumber(String mobile) throws
                                    InvalidNumberException
  {
    if (mobile.length() != 10)
      throw new InvalidNumberException();
    else
      S.O.P("Valid Number");
  }
  P.S.V.M (String[] args)
  {
    try
    {
      verifyNumber("8760092487");
      verifyNumber("860092487");
    }
    catch(InvalidNumberException e)
    {
      S.err.P("Invalid Number Found");
    }
  }
}
```

Throws	Throw
Method Declaration	Method Definition
Used to delegate the task of Exception handling to calling method	Used to manually throw an Exception object
We can delegate multiple exceptions with throws	We can only throw one Exception at a time

Class String:

- It is an inbuilt class of the java library.
- It is present in java.lang package
- String class is used to represent sequence of characters
- We can represent string values using String class in the following two ways
  1. Literals
  2. new keyword
- In both literals and new keyword internally an object will be created

Ex-1:

```
Public class Program
{
  P.S.V.M (String[] args)
  {
    String s1 = "Qspiders"; // literals
    String s2 = "new String ("Jspiders"); // new keyword
    S.O.P (s1);
    S.O.P (s2);
  }
}
```

- All string objects in java allocated memory a special part of the Heap Area known as String Pool.
- The string Pool is divided into 2 sections
  1. Constant Pool.
  2. Non Constant Pool.
- All string objects created as a literals will be allocated memory in the constant pool.

- The JVM does not allow duplicates in the constant pool.

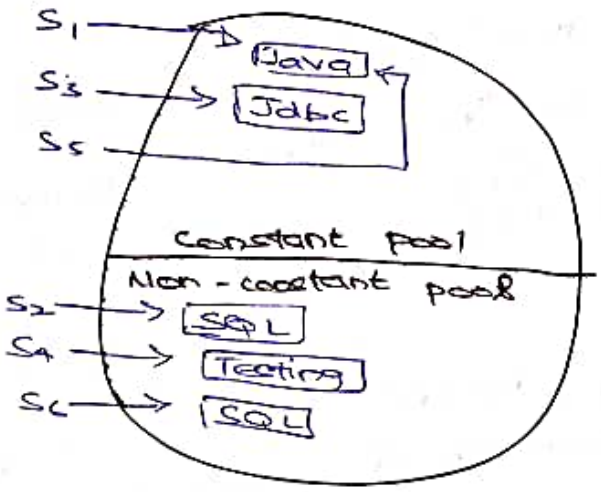
- All string objects created using new keyword will be allocated memory in non-constant pool.

- The Non-constant pool may have duplicate values.

Ex-1:

```
String s1 = "Java";
String s2 = new String("SQL");
String s3 = "Jdbc";
String s4 = new String("Testing");
String s5 = "Java";
String s6 = new String("SQL");
```

String pool



- String class override the following 3 methods

1. toString() = current String value
2. hashCode() = Unicode value
3. equals() = content comparison btwn 2 strings

Ex-1:

```
Public class program
{
    P.S.V.M (String[] args)
    {
        String s1 = "Tiger";
        S.O.P(s1.toString());
        String s2 = "A";
        S.O.P(s2.hashcode());
        String s3 = "Java";
        String s4 = new String ("Java");
        S.O.P(s3.equals(s4));
    }
}
```

O/P:

```
Tiger
65
true
```

String class implements compareable interface hence all objects are compareable objects

-Ve → Given string after current string
+Ve → Given string before current string
0 → Given string equals current string

Ex:

```
String str = "Dhanush";
S.O.P(str.compareTo("Ajith"));
S.O.P(str.compareTo("Vijay"));
S.O.P(str.compareTo("Dhanush"));
```

O/P:

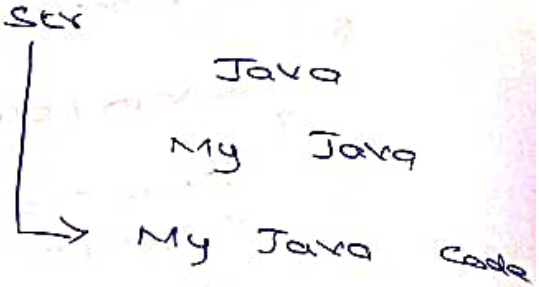
```
3
-18
0
```

We can compare the string object using compare 2 methods inherited from compareable methods

- String class objects are immuteable objects because once it is created any attempt to modify or change will result in the creation of a new object.

Ex:-

```
String str = "Java";
S.O.P (str);
str = "My" + str;
S.O.P (str);
str = str + "code";
S.O.P(str);
```



String Buffer and String Builder

- Both string Buffer and String Builder are inbuilt class in Java library 'java.lang' package.
- String buffer & String builder can be used as an alternate for String class.
- String buffer & String builder object can only be created with new keyword.

Ex-1:-

```
Public class Prgm1
{
  P.S.V.m(String[] args)
  {
    String Buffer sb1 = new StringBuffer ("@spiders");
    String Buffer sb2 = new StringBuffer ("Iqspiders");
     S.O.P (sb1);
     S.O.P (sb2);
  }
}
```

- Both String Buffer and String builder are final classes and cannot have a subclass
- String buffer and String builder overrides only the toString method of class object

Ex-1:

```
Public class prgm2
{
  P.S.V.M (String[] args)
  {
    StringBuffer sb1 = new StringBuffer("A");
    S.O.P (sb1.toString()); // A
    S.O.P (sb1.hashCode()); // 366711264
    StringBuilder sb2 = new StringBuilder("a");
    S.O.P(sb2.toString()); // a
    S.O.P (sb2.hashCode()); // 18291647
  }
}
```

String buffer and string builder are mutable object.

2 object created

```
String str = "Java";
S.O.P (str);
str = str + "J2EE";
S.O.P(str);
```

```
StringBuffer buffer =
 new StringBuffer("Java");
S.O.P(buffer);
buffer.append ("J2EE");
S.O.P(buffer);
```

```
StringBuilder builder = new StringBuilder("Java");
S.O.P (builder);
builder.append ("J2EE");
S.O.P (builder);
```

Version	String Buffer	String Builder
Version	V1.0 (Legacy class)	V1.5 (Non legacy class)
Thread Safety	Thread safe and Synchronized	Non thread safe and Non - Synchronized
Source	more secure	less secure
Speed	slow	fast

Version	String	StringBuffer	StringBuilder
Version	V1.0	V1.0	V1.5
Legacy	Yes	Yes	Yes
immutable	Yes	No	No
Overriden methods of class object	toString() hashCode() equals()	toString()	toString()
Comparable	Yes	No	No
Literals	Yes	No	No
Thread safe	Yes	Yes	No

Diff btwn final, finally and finalize?

Final	Finally	Finalize()
Keyword	Block	Method
create Constants	used in ExceptionHandling	Used in garbage collection
a) class No-subclass	Code inside finally block will execute irrespective of Exception	Code inside finalize() method executed by GC before object is destroyed.
b) Method No overriding		
c) Variable No-reinitialization		

Wrapper class :

- Wrapper classes are inbuild classes java library that are used to support primitive data types.

- For each primitive data type, there is inbuild class in java library.

Primitive	Wrapper
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Note: Java is object oriented but not 100%, because it support primitive values.

* Using wrapper class we can perform the following 3 operation

1. Boxing
2. UnBoxing
3. Parsing

Boxing:

It is a process of representing a primitive value as a wrapper class object.

Ex-1:

```
Public class program1
{
   P.s.v.m (String[] args)
   {
      int a = 15;
      Integer ref = new Integer (a);
      S.o.P (ref);
      char c = 'a';
      Character ch = new character (c);
      S.o.P (ch);
      double b = 3.45;
      Double obj = b;
      S.o.P (obj);
   }
}
```

O/P:
15
a
3.45

UnBoxing:

It is a process representing wrapper class object as a primitive value.

Ex:

```
Public class Prgm2
{
   P.s.v.m (String[] args)
   {
      Integer ref = 35;
      int a = ref;
      S.o.P (a);
      Boolean bool = false;
      boolean b = bool;
      S.o.P (b);
      Float val = 3.45f;
      float c = val;
      S.o.P (c);
   }
}
```

O/P
35
false
3.45

Parsing :

- Extracting primitives values from the string is called as parsing.
- Parsing can be done using parser methods present in wrapper class.
- All the parser methods are static.

Wrapper	Parser
Byte	parseByte()
Short	parseShort()
Integer	parseInt()
Long	parseInt()
Float	parseFloat()
Double	parseDouble()
Character	Not Available
Boolean	parseBoolean()

Eg :

```
Public class Pgm3
{
  P.s.v.m (String[] args)
  {
    String s1 = "90";
    int a = Integer.parseInt(s1);
    S.o.p(a);
    String s2 = "3.45";
    double b = Double.parseDouble(s2);
    S.o.p(b);
    String s3 = "true";
    boolean s3 = "true";
    boolean c = Boolean.parseBoolean(s3);
    S.o.p(c);
  }
}
```

O/P:
90
3.45
true

Why character not available in parser (See previous page)

```
String str = "Tiger";
             01234
Char ch = str.charAt();
```

Parser

File Handling

File Handling is the process of manipulating the files and folders present in a file system.

File :

- A file is a container in which information is available.
- Every file in a computer will have a file extension.

Folder / Directory :

A folder is a container inside which files are available

File system :

The file system is an arrangement of files and folders inside a computer.

- The java library provides extensile support for file handling via inbuilt prgms
- All inbuilt java prgms related to file handling can be found in "java.io" package

Java prgm to create a folder :

```
import java.io.File;
Public class Program1
{
  P.S.V.m (String[] args)
  {
    File ref = new File("D:/3CCN10/FILE10");
    boolean bool = ref.exists();
    if (bool == false)
    {
      ref.mkdir();
      S.o.P("Folder is created");
    }
```

```
    else
    {
      S.O.P ("Folder exists");
    }
  }
}
```

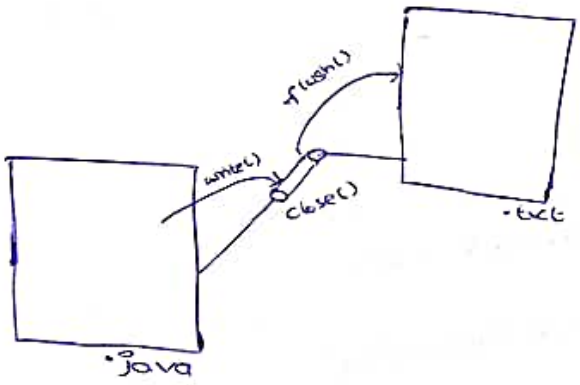
Prgm to create a file :

```
import java.io.File;
import java.io.IOException;
Public class Program2
{
  P.S.V.M (String[] args)
  {
    File ref = new File("D:/3CCA10/FILEIO/Demo.txt");
    boolean bool = ref.exists();
    if (bool==false)
    {
      try
      {
        ref.createNewFile();
        S.O.P ("File is created");
      }
      catch(IOException e)
      {
        e.printStackTrace();
      }
    }
    else
    {
      S.O.P (" File Already exists");
    }
  }
}
```

Pgm to delete File/Folder:

```
import java.io.File;
Public class Program3
{
   P.S.V.M (String[] args)
   {
      File ref = new File("D:/3ACC10/FILEIO/Demo.txt");
      boolean bool = ref.exists();
      if (bool == true)
      {
         ref.delete();
         S.O.P ("Deleted");
      }
      else
      {
         S.O.P("File/Folder Not Found");
      }
   }
}
```

Pgm to writing data to a text file



Steps:

- Create an object of class FileWriter
- Write data to FileWriter object using "write()" method
- Transfer data to actual file by calling "flush()" method
- close FileWriter object calling "close" method

```
import java.io.FileWriter;
import java.io.IOException;
Public class Pgm4
{
  P.S.V.M (String[] args)
  {
    FileWriter ref = null;
    try
    {
      ref = new FileWriter("D:/SCCA10/FILE10/Demo.txt");
      ref.write('ABC');
      ref.write("\n");
      ref.write("abc");
      ref.flush();
      S.O.P("Write completed");
    }
    catch(IOException e)
    {
      e.PrintStackTrace();
    }
    finally
    {
      try
      {
        ref.close();
      }
      catch(IOException e)
      {
        e.PrintStackTrace();
      }
    }
  }
}
```

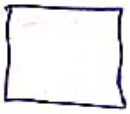
Pgm to reading data from a text file

Steps:

- Create an object of class FileReader.
- Read data from file using "read()" method.
- Close FileReader object using "close()" method.



.txt



.java

```
import java.io.FileReader;
import java.io.IOException;
Public class Program5
{
  Public static void main(String[] args)
  {
    FileReader ref=null;
    try
    {
      ref=new FileReader("D:/3CCA10/FILEIO/Demo.txt");
      int val=ref.read();
      while(val!=-1)
      {
        S.o.p((char)val);
        val=ref.read();
      }
    }
    catch(IOException e)
    {
      e.printStackTrace();
    }
    finally
    {
      try
      {
        ref.close();
      }
      catch(IOException e)
      {
        e.printStackTrace();
      }
    }
  }
}
```

Recursion

- Called method is calling back calling method
- The method is calling itself.
- Recursion if not controlled will lead to StackOverflowError.
- Programmer is responsible to prevent uncontrolled recursion by setting break points in program

Ex-18

```
class Recursion1
{
  static void test()
  {
    S.o.p ("test starts");
    disp();
    S.o.p("test ends");
  }
  static void disp()
  {
    S.o.p("disp starts");
    test();
    S.o.p("disp ends");
  }
  P.s.v.m(String[] args)
  {
    test();
  }
}
```

Ex-19

```
class Recursion2
{
  static void run()
  {
    S.o.p ("Execute run");
    run();
  }
  P.s.v.m (String[] args)
  {
    run();
  }
}
```

```
Class MultiplicationWithRecursion
{
   static void function(int num, int limit)
   {
      int prod = num * limit;
      S.O.P(num+" * "+limit+" = "+prod);
      limit++;
      if(limit<=10)
         function(num, limit);
   }
   P.S.V.M(String[] args)
   {
      function(2,1);
   }
}
```

```
Class ModulusWithRecursion
{
   Static void function(int num, int limit)
   {
      int res = num/2;
      S.O.P(num+"%"+2+" ="+res);
      num = num+2;
      if(num<=limit)
         function(num, limit);
   }
   P.S.V.M(String[] args)
   {
      function(2,10);
   }
}
```

1. Java prgm for prime number using recursion.
2.

```
*
* *
* * *
* * * *
* * * * *
```

```
Class FactorialUsingRecursion
{
   Static void function (Int num, Int fact)
   {
      fact = fact * num;
      num --;
      if (num>0)
         function (num, fact);
      else
         S.O.P (fact);
   }
   P.S.V.M (String[] args)
   {
      function(5, 1);
   }
}
```

Serialization and De-Serialization

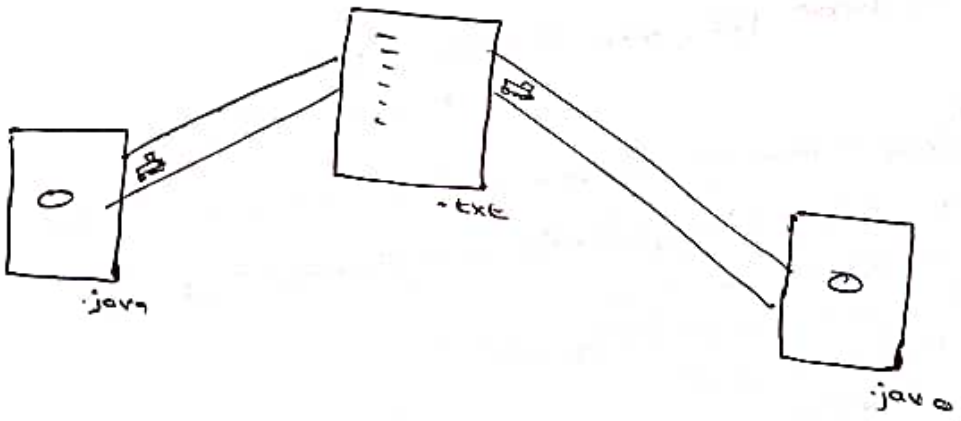
Serialization :

- It is the process of writing a Java object to a text file.
- During serialization, object is converted to a stream of ~~objects~~ Bytes.
- Serialization is also known as Marshalling.

Note: Serialization works only on Java objects of type Serializable.

De-Serialization :

- It is the process of reading a Java object from a text file.
- During De-serialization object is recreated from a stream of Byte
- De-Serialization is also known as Un Marshalling.



We can prevent serialization using the keyword transient.

Ex :

```
import java.io.Serializable;
Public class Player implements Serializable
{
    String name;
    int jerseyNo;
}
```

```
import java.io.FileOutputStream;
import java.io.ObjectOutputStream;
public class Sender
{
    P.S.V.M (String[] args) throws Exception
    {
        Player ref = new Player();
        ref.name = "Dhoni";
        ref.jerseyNo = 7;
        FileOutputStream fos = new FileOutputStream
                               ("D:/SCCAIO/FILEIO/Repo.txt");
        ObjectOutputStream oos = new ObjectOutputStream(fos);
        oos.objeWriteObject(ref);
        S.O.P ("Serialization Completed");
        oos.close();
        fos.close();
    }
}

import java.io FileInputStream;
import java.io objectInputStream;
Public class Receiver
{
    P.S.V.M (String[] args) throws Exception
    {
        FileInputStream fis = new FileInputStream
                              ("D/SCCAIO/FILEIO/Repo.txt");
        @Override
        ObjectInputStream ois = new objectInputStream(fis);
        Player ref = (Player) ois.readObject();
        S.O.P(ref.name);
        S.O.P (ref.jerseyNo);
        ois.close();
        fis.close();
    }
}
```

Threads

A thread is a light weight process.

Process:

A program or application that is currently under execution is called a process.

Light weight process:

Any process that consumes less CPU and less memory.

- Threads concept can be used by the Java programmer to implement multitasking at program level.
- Java language provides inbuilt support for threads.
- Whenever a Java program begins execution the JVM internally create 3 threads.
  1. Thread scheduler : Manage lifecycle of all threads in Java
  2. Garbage collector : Managing memory in Heap area
  3. Main Thread : Responsible for executing code written by programmer
- Garbage collector and Thread Scheduler are daemon threads
- As a Java programmer we can create threads in 2 ways.
  1. By extending Thread class
  2. By implementing Runnable interface.
- Using multithreading will help enhance the speed of execution of a program.

Extending Thread class

- Create a Java class that extends "Thread" class.
- Override run() method in Thread class. The run() method should contain the task to be executed by the thread.
- Create an object of Target class.
- Call start() method to begin the execution of Thread.

Ex:-

```
class DemoThread extends Thread
{
    @Override
    Public void run()
    {
        for(int i=0; i<5; i++)
        {
            S.o.P("DemoThread...");
        }
    }

Public class Pgm1
{
    P.S.V.m (String[] args)
    {
        S.O.P("Pgm starts");
        DemoThread dt = new DemoThread();
        dt.start();
        for (int i=0; i<5; i++)
        {
            S.o.P("MainThread...");
        }
        S.O.P("Pgm Ends");
    }
}
```

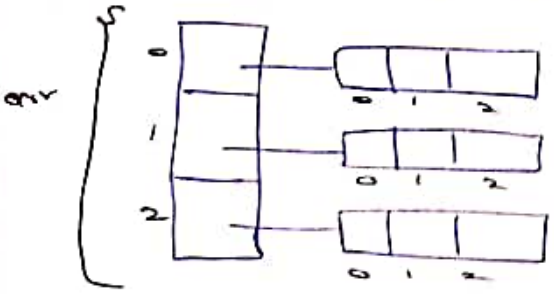
2Dimensional Arrays

```
import java.util.Scanner;
Public class prgm1
{
  P.S.V.M (String[] args)
  {
    Scanner scn = new Scanner(System.in);
    int[][] arr = new int[3][3];
    for(int i=0; i<arr.length; i++)
    {
      for(int j=0; j<arr[i].length; j++)
      {
        S.O.Pln("Enter value for"+i+","+j);
        arr[i][j] = scn.nextInt();
      }
    }
    S.O.Pln("The Array Elements are");
    for(int i=0; i<arr.length; i++)
    {
      for(int j=0; j<arr[i].length; j++)
      {
        S.O.P(arr[i][j]+" ");
      }
      S.O.Pln();
    }
  }
}
```

```
int[] arr = new int[5];
```




```
int[][] arr = new int[3][3];
```



Matrix Addition for 2D Array elements.

```
Public class MatrixAddition
{
  P.S.V.m(String[] args)
  {
    int[][] arr1 = {{1,2,3},{4,5,6},{7,8,9}};
    int[][] arr2 = {{3,2,1},{7,8,9},{4,6,5}};
    int[][] sum = new int[arr1.length][arr1.length];
    for(int i=0;i<sum.length;i++)
    {
      for(int j=0;j<sum[i].length;j++)
      {
        sum[i][j] = arr1[i][j] + arr2[i][j];
      }
    }

    for(int i=0;i<sum.length;i++)
    {
      for(int j=0;j<sum[i].length;j++)
      {
        S.o.P(sum[i][j] + "  ");
      }
      S.o.Pln();
    }
  }
}
```

O/P:

```
4   4   4
11  13  15
11  14  14
```

Display 2D Array element in Reverse Order

```
Public class Pgm2
{
  P.S.V.M (String[] args)
  {
    int[][] arr={{1,2,3},{4,5,6},{7,8,9}};
    S.O.Pln("The Array Elements are");
    for(int i=0; i<arr.length; i++)
    {
      for(int j=0; j<arr[i].length; j++)
      {
        S.O.P(arr[i][j]+" ");
      }
      S.O.Pln();
    }
    S.O.Pln("Rows Reverse");
    for(int i=0; i<arr.length; i++)
    {
      for(int j=arr[i].length-1; j>=0; j--)
      {
        S.O.P(arr[i][j]+" ");
      }
      S.O.Pln();
    }
  }
}
```

O/P :

The Array Elements are

1 2 3

4 5 6

7 8 9

Rows Reverse

3 2 1

6 5 4

9 8 7